



# 인공지능(AI)



## 1. AI 개념



## 01

### AI(인공지능) 개요

- 1.1 AI(인공지능) 개념

## 02

### AI 알고리즘

- 2.1 지도학습
- 3.2 비지도학습
- 2.3 강화학습

## 03

### Model 학습

- 1.1 합성 데이터
- 1.2 모델 평가

## 04

### 자연어처리

- 1.1 자연어처리 역사
- 1.2 Word Embedding
- 1.3 Attention

## 05

### 생성 AI

- 1.1 ChatGPT
- 1.2 Auto Encoder
- 1.3 VAE, GAN

## 06

### 최신 AI

- 1.1 CNN
- 1.2 연합학습
- 1.3 딥페이크, XAI

## 07

### 인공신경망

- 1.1 신경망 역사
- 1.2 퍼셉트론
- 1.3 경사하강법

## AI(인공지능) 개념

### □ 인공지능 용어 정리

- 대부분 인공지능(AI), 머신러닝(Machine Learning), 딥러닝(Deep Learning)을 비슷한 내용으로 이해하고 있지만, 인공지능은 가장 넓은 개념이며, 인공지능을 구현하는 대표적인 방법 중 하나가 바로 머신러닝
- 또한 딥러닝은 머신러닝의 여러 방법 중 하나의 방법론으로 인공신경망(Artificial Neural Network)의 한 종류



[인공지능, 머신러닝, 딥러닝]



# 인공지능(AI)



## 2. 인공지능 알고리즘1



## 1. 지도학습(Supervised Learning)

### 1) 분류(classification)

- 의사결정 트리(Decision Tree)
- 랜덤 포레스트(Random Forest)
- 앙상블
- KNN (K-Nearest Neighbor)
- SVM (Support Vector Machine)

### 2) 회귀(regression analysis)

- 선형 회귀(Linear Regression)
- 로지스틱 회귀(Logistic Regression)

## 2. 비지도학습

### 1) 군집

- K-MEANS
- DBSCAN

### 2) 차원축소

- PCA(Principal Component Analysis)

## 3. 강화학습

- Q-Learning
- MCTS

### □ 머신러닝 유형

#### ① 지도학습(Supervised Learning)

- 입력값과 결과값(정답 레이블)을 함께 주고 학습을 시키는 학습 기법

#### ② 준지도학습(Semi-Supervised Learning)

- 지도학습과 비지도 학습의 중간으로, 레이블이 달려있는 데이터와 레이블이 달려있지 않은 데이터(unlabeled data)를 동시에 사용해서 더 좋은 모델을 만드는 학습 기법

#### ③ 비지도학습(Unsupervised Learning)

- 결과값 없이 입력값만 주고 학습시키는 방법으로, 데이터를 탐색하여 내부 구조와 유사한 속성을 파악하는 학습 기법

#### ④ 강화학습(Reinforcement Learning)

- 결과값이 아닌 어떤 일을 잘했을 때 보상(Reward)을 주는 방식으로 어떤 행동(Action)이 최선인지를 학습 기법

## 지도학습(Supervised Learning)

### 1. 지도학습 개요

#### □ 지도학습 개념

- 입력값과 결과값(정답 레이블)을 함께 주고 학습을 시키는 방법으로, 분류/회귀 등 과거 데이터를 기반으로 앞으로 있을 이벤트를 예측
- 예를 들어, 신용카드 거래의 사기성이나 보험 가입자의 보험금 청구 가능성 여부 등을 예측

#### □ 지도학습 유형

##### ① 분류(Classification)

- 어떤 데이터에 대해 두 가지 중 하나로 분류하는 모델
- 예를들어, 스팸메일을 예측한다고 가정했을 경우 해당 메일이 스팸인가? 예 또는 아니오로 구분될수 있는 모델

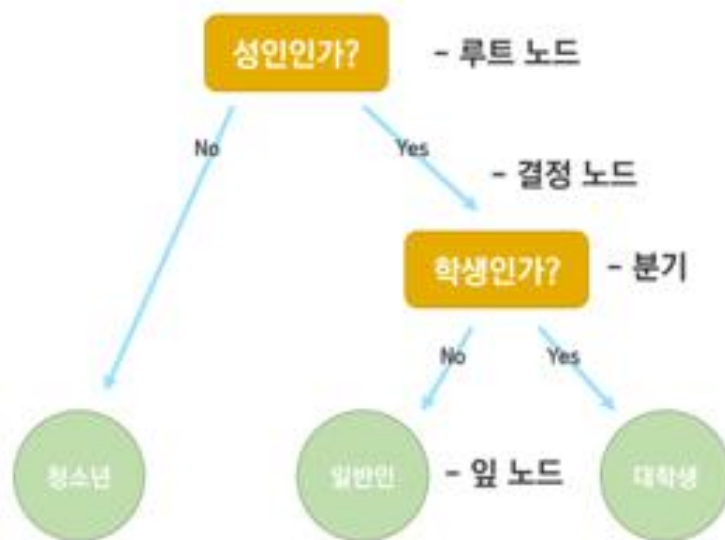
##### ② 회귀(Regression)

- 어떤 데이터들의 특징(Feature)을 기준으로 연속된 값을 예측하는 모델
- 주로 어떤 패턴이나 트렌드, 경향을 예측할 때 하용 하는 모델

## (1) 의사결정 트리(Decision Tree)

### □ 의사결정 트리 개념

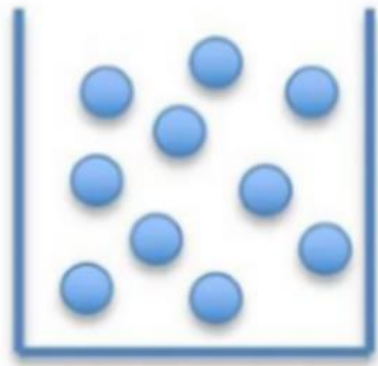
- 각 데이터들이 가진 속성들로부터 패턴을 찾아내서 분류하는 알고리즘 과제를 수행할 수 있도록 하는 지도학습 머신러닝 모델
- 의사결정트리는 전형적인 분류 모델이며 매우 직관적인 방법 중 하나로 다른 모델들과는 다르게 결과물이 시각적으로 읽히기 쉬운 형태로 나타나는 것이 장점 존재



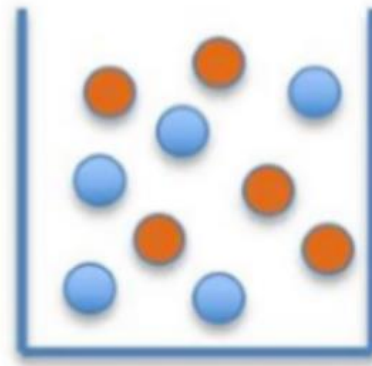
## (1) 의사결정 트리(Decision Tree)

### 1) 불순도(Impurity)

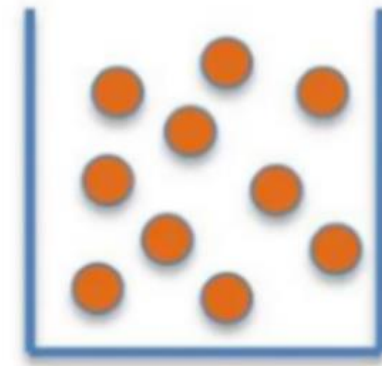
- 특정 공간에 서로 다른 개체들이 얼마나 포함되어 있는가를 의미
- 불순물이 어느정도 포함되어 있는지 확인할 수 있는 지표



항아리 1.  
순수도(purity) : 100%



항아리 2.



항아리 3.

- 불순도를 측정하는 지표에 엔트로피(Entropy), 지니계수(Gini Index) 존재

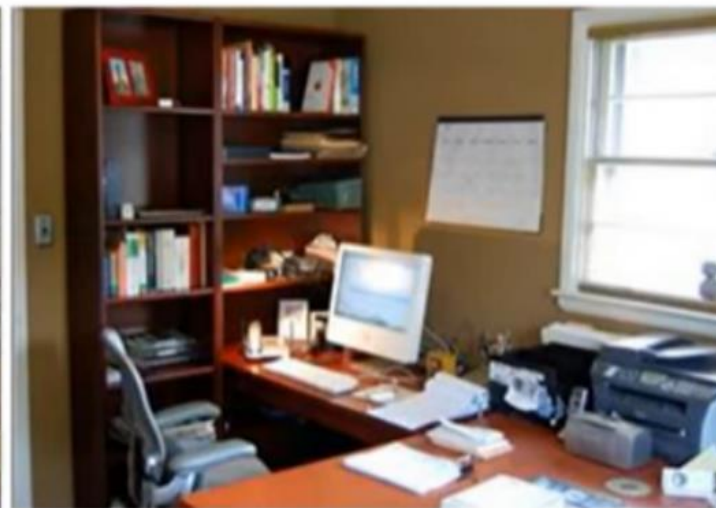
## (1) 의사결정 트리(Decision Tree)

### ① 엔트로피(Entropy) → ID3 알고리즘

- 무질서한 정도를 정량화(수치화)한 값
- 엔트로피 값이 0에 가까울수록 좋음



High Entropy (messy)



Low Entropy (Clean)

$$Entropy(S) = \sum_{i=1}^c p_i * I(x_i)$$

## (1) 의사결정 트리(Decision Tree)

### ② 지니계수(Gini Index) → CART 알고리즘

- 불순도를 측정하는 지표로서, 데이터의 통계적 분산정도를 정량화해서 표현한 값
- 0과 1사이의 값을 가짐
- 지니계수 값이 0에 가까울수록 좋음

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

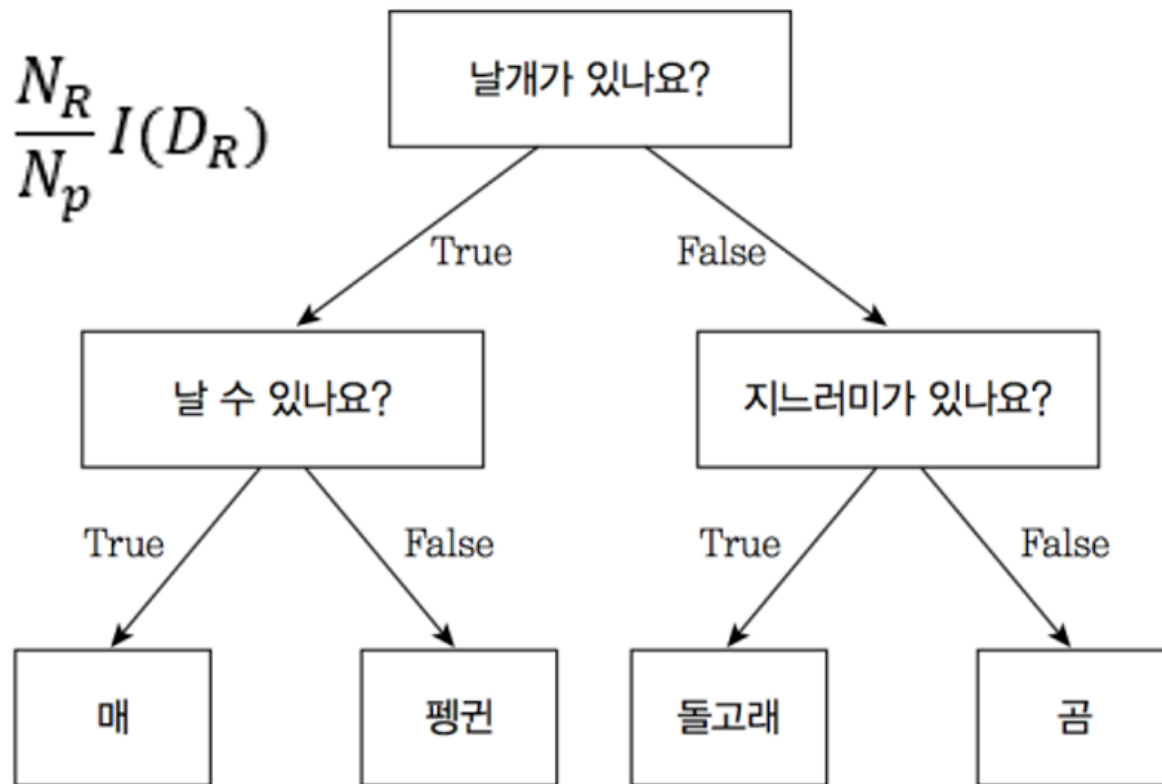


## (1) 의사결정 트리(Decision Tree)

### ③ 정보이득(Information Gain)

- 다음 노드로 이동 시 올바른 노드를 선택할 수 있도록 하는 수치
- 정보이득 값이 클 수록 올바른 선택

$$IG(D_p, f) = I(D_p) - \frac{N_L}{N_p} I(D_L) - \frac{N_R}{N_p} I(D_R)$$

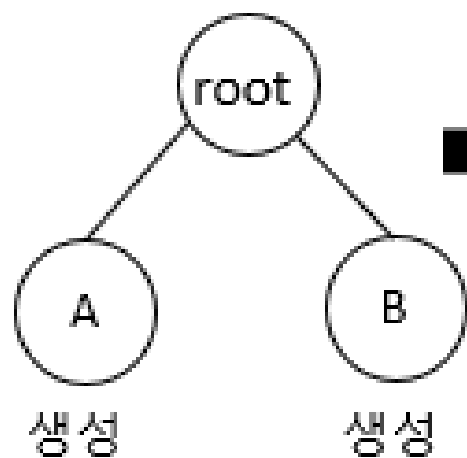




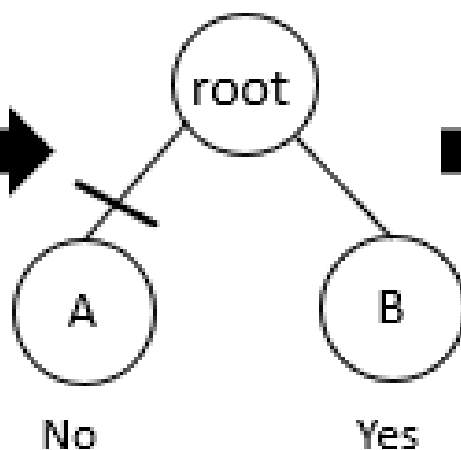
## (1) 의사결정 트리(Decision Tree)

### 2) 의사결정 트리 절차

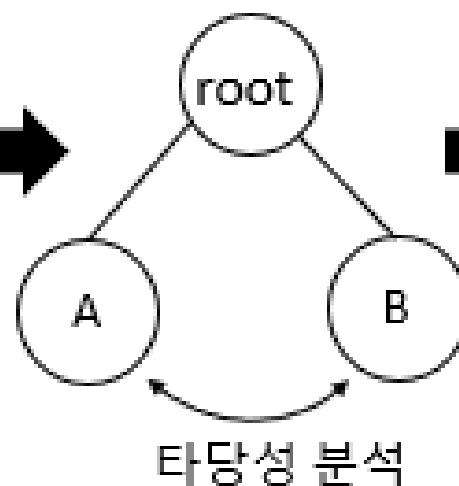
의사결정나무 형성



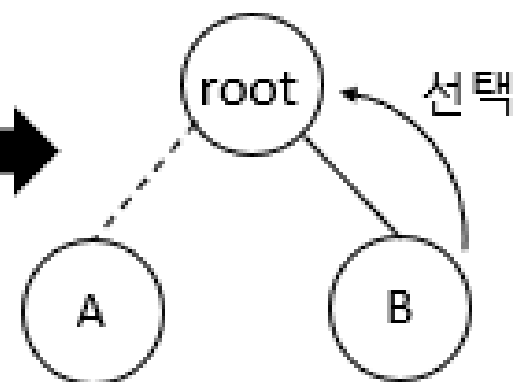
가지치기



타당성 평가

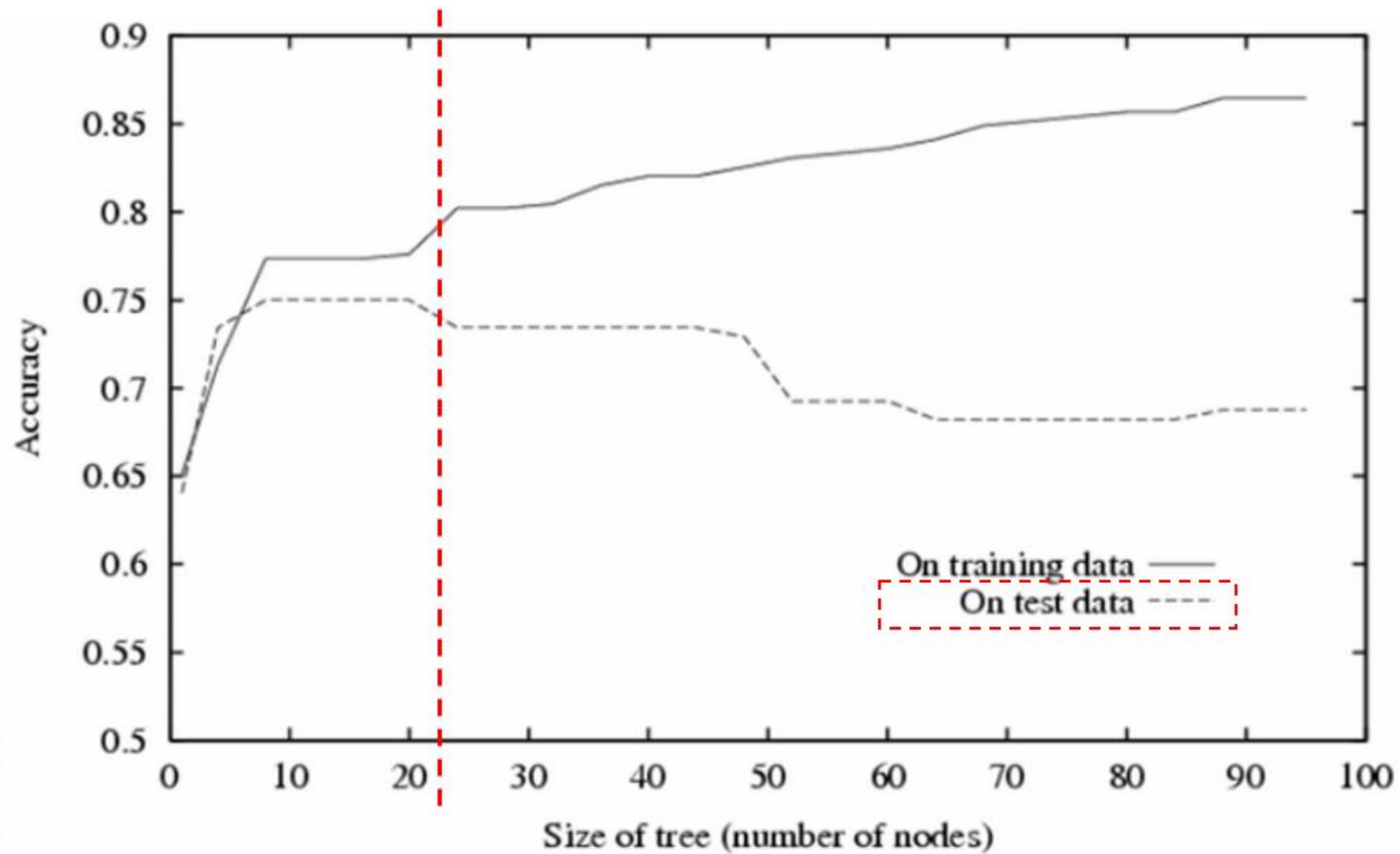


해석 및 예측



## (1) 의사결정 트리(Decision Tree)

### 2) 의사결정 트리 오버피팅(Overfitting)



## (2) 랜덤 포레스트 (Random Forest)

- **앙상블 기법** 중 하나로, 다수의 **임의성(randomness)**에 의해 decision tree를 형성하고, 각 트리가 분류한 결과에서 **투표**를 실시하여 가장 많이 득표한 결과를 최종 분류 결과로 선택하는 기법



decision tree

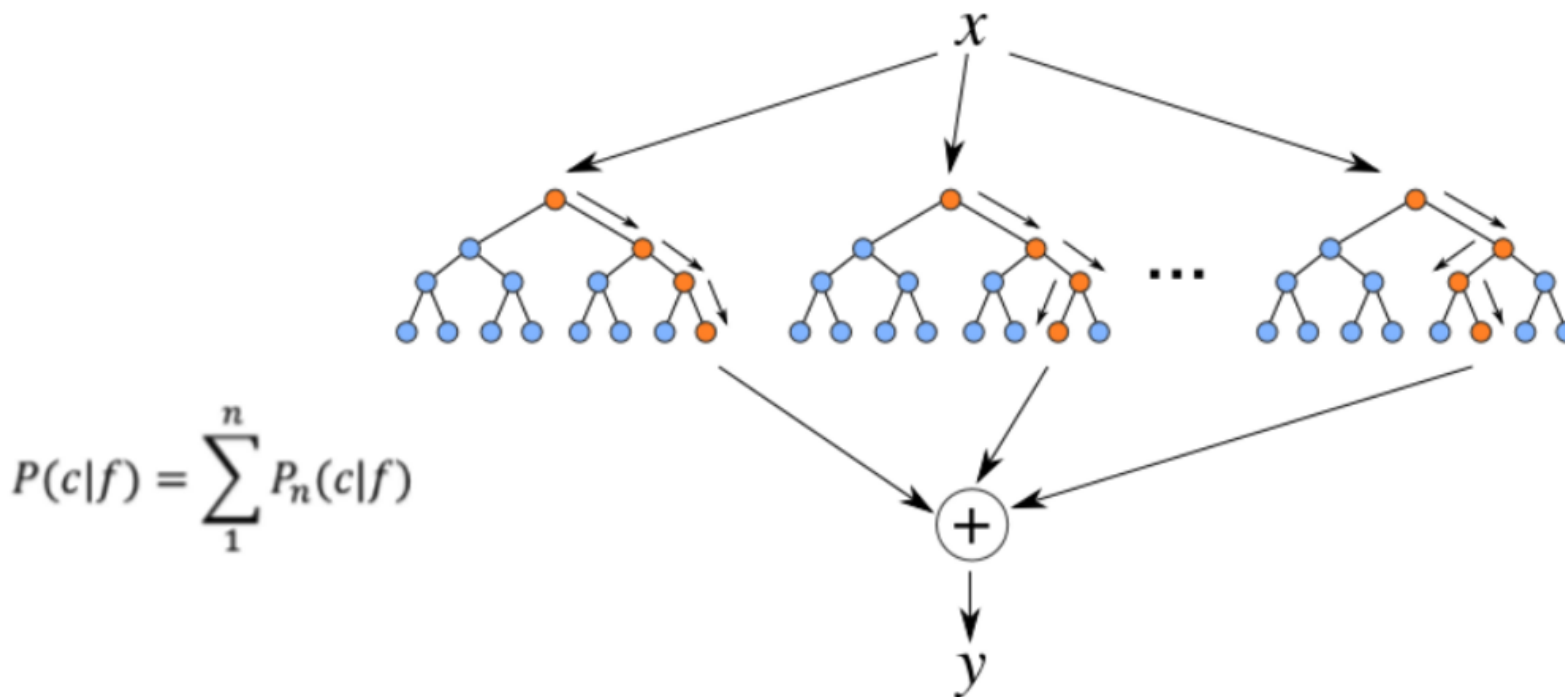


random forest

## (2) 랜덤 포레스트 (Random Forest)

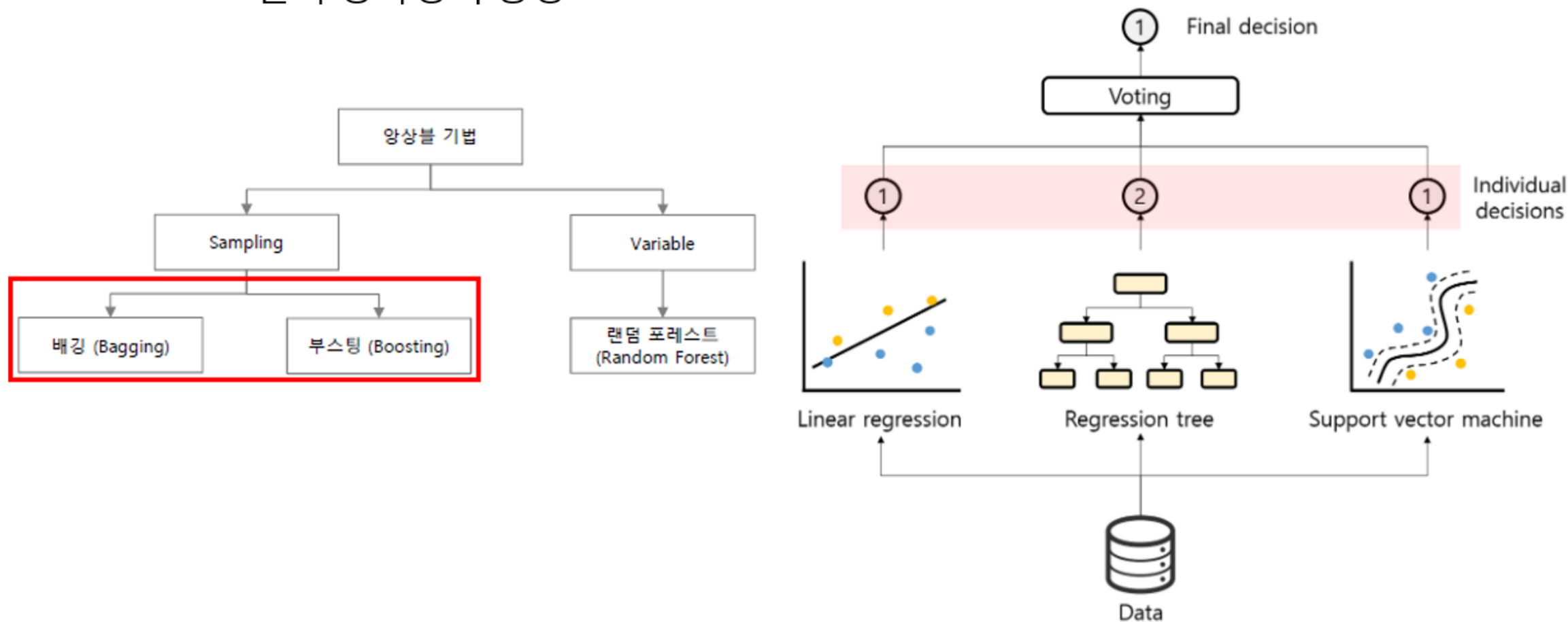
### 파라미터

- **n\_estimators**: 랜덤 포레스트 안의 의사 결정 트리 개수
- **max\_features**: 무작위로 선택할 Feature의 개수
- **max\_depth**: 각 결정 트리의 최대 깊이
- **bootstrap**: 데이터 샘플을 복원 추출할지 여부



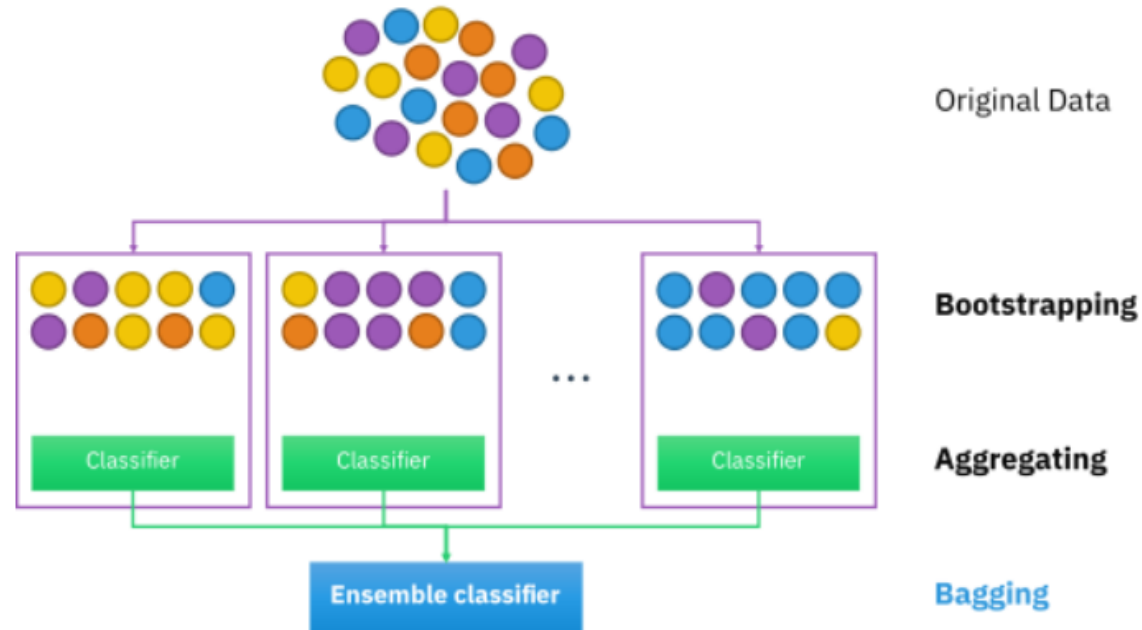
### (3) 앙상블 (ensemble)

- 다양한 학습 알고리즘들을 결합하여 성능을 향상시키기 위한 모델
- 모델의 정확성이 향상



## (4) 배깅 (bagging)

- bootstrapping + aggregating
- **bootstrapping**을 통해 train\_set 에서 **무작위 복원 추출**을 통해 모델을 생성하고 , 이러한 모델을 다수 생성하여 이들의 **출력 결과를 aggregating** 하는 알고리즘
- **Bootstrapping** : 전체 집합에서 **무작위 복원추출**을 통해 여러 부분집합을 만드는 행위
- **Aggregating** : '집계한다' 는 광범위한 용어로 평균이나 최빈값 등을 도출하는 행위



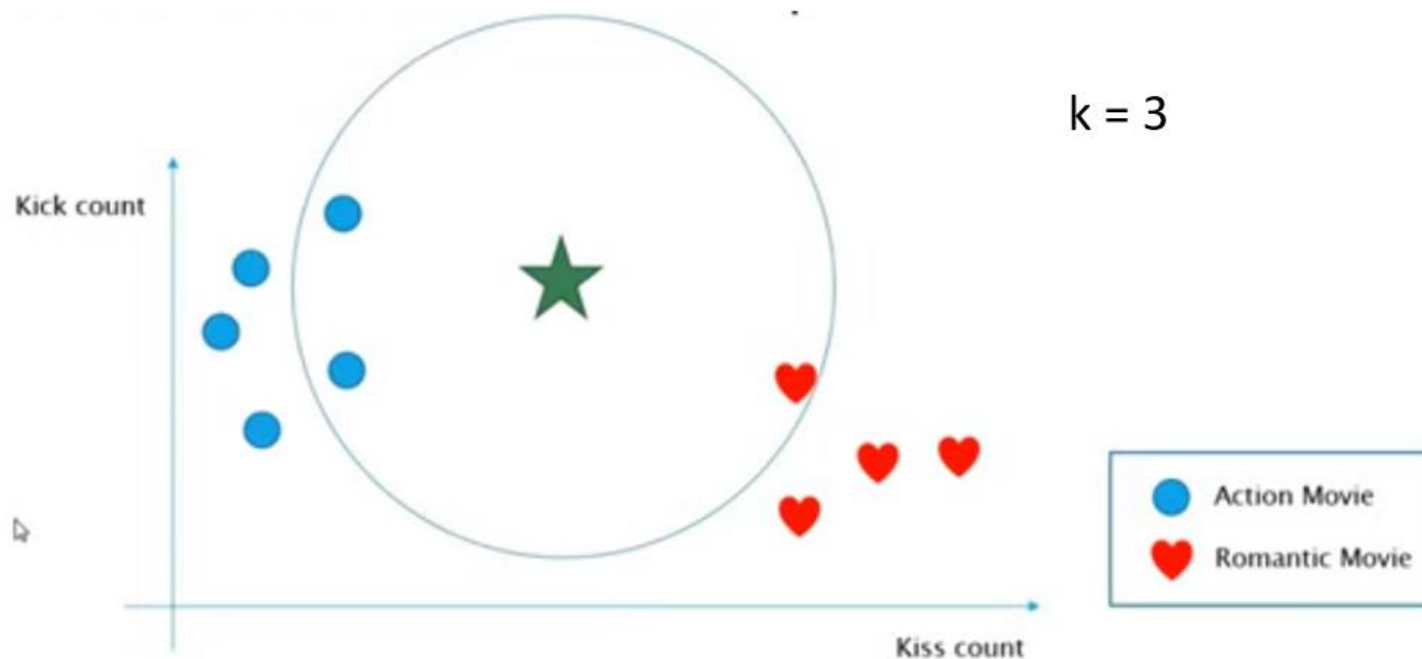
## (5) 부스팅 (Boosting)

- 약한 학습기(weak learner) 또는 잘못 분류된 개체들에 집중하여 가중치를 조정하여 반복적으로 오류를 개선해 나가는 알고리즘
- 오답에 대해 높은 가중치, 단 Outlier에 취약
- AdaBoost, Gradient Boost(GBM), XGBoost, LightGBM, CatBoost 등



## (6) KNN (K-Nearest Neighbor)

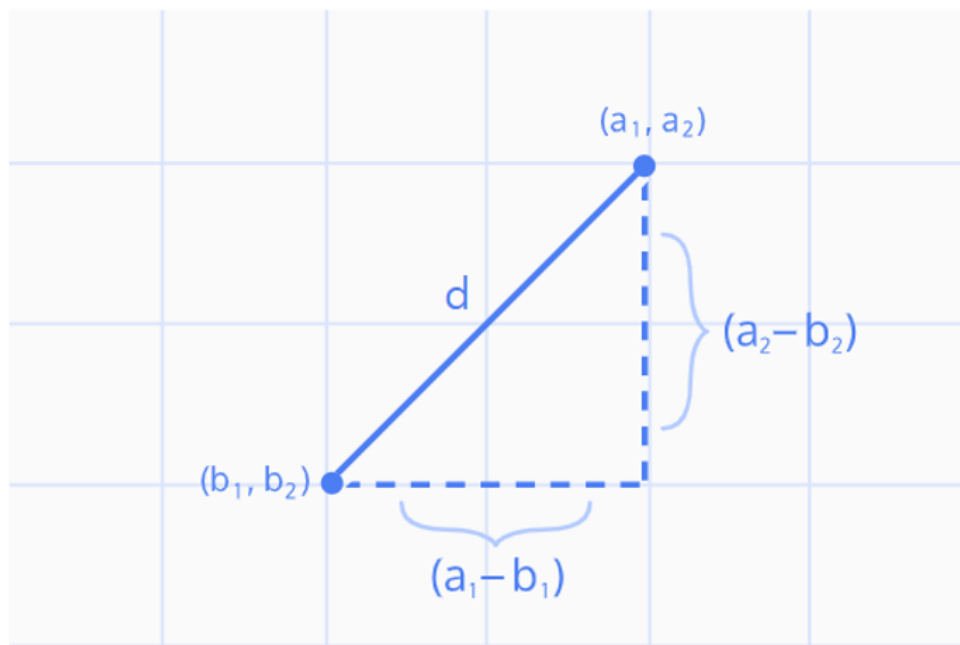
- 새로운 입력값을 기존의 모든 노드와 거리를 측정해 분류하는 알고리즘
- 가까운 거리 : 유클리드 거리, 맨하탄 거리
- 빈도 :  $k$ 의 개수
- 사례 : 암 진단, 이미지 처리, 글자 인식, 얼굴 인식, 영화, 음악, 상품 추천 알고리즘





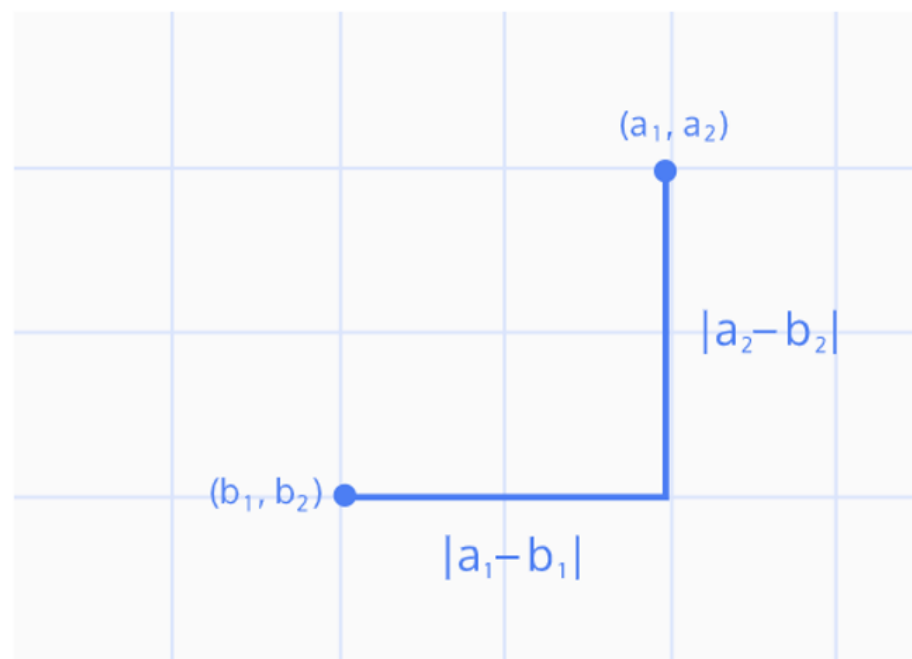
## (6) KNN (K-Nearest Neighbor)

- 유클리드 거리 (Euclidean Distance)
- 맨하탄 거리 (Manhattan Distance)



$$d = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

[유클리드 거리]

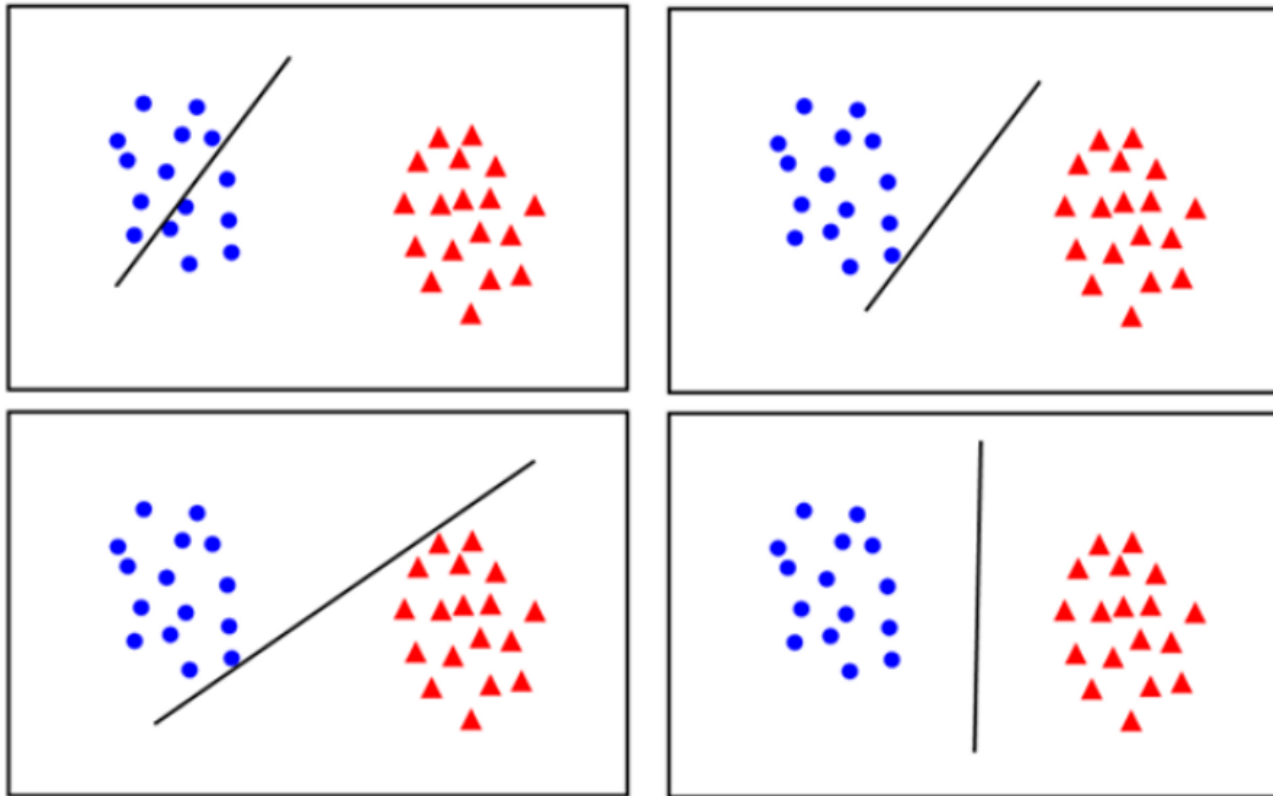


$$d = |a_1 - b_1| + |a_2 - b_2|$$

[맨하탄 거리]

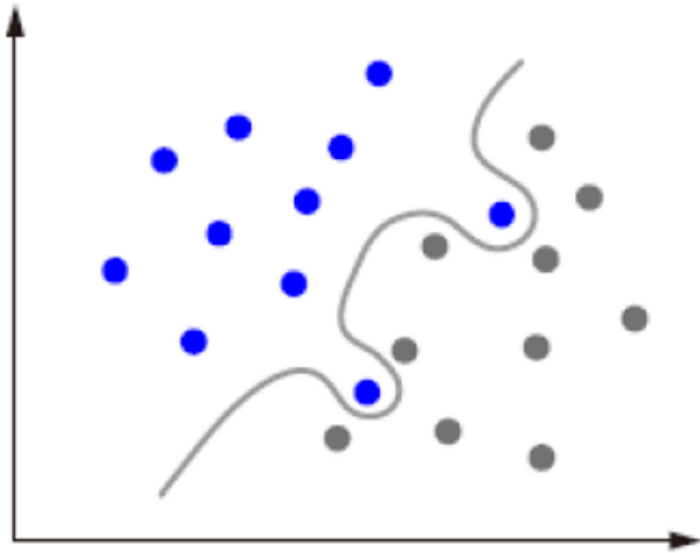
## (7) SVM (Support Vector Machine)

- 데이터를 분류하기 위해 두 개의 클래스로 나누어 Margin을 최대로 하는 결정직선(Decision Boundary)을 찾는 알고리즘
- 주어진 데이터가 어느 카테고리에 속할지 두 개로 분류하는 이진 선형 분류 모델



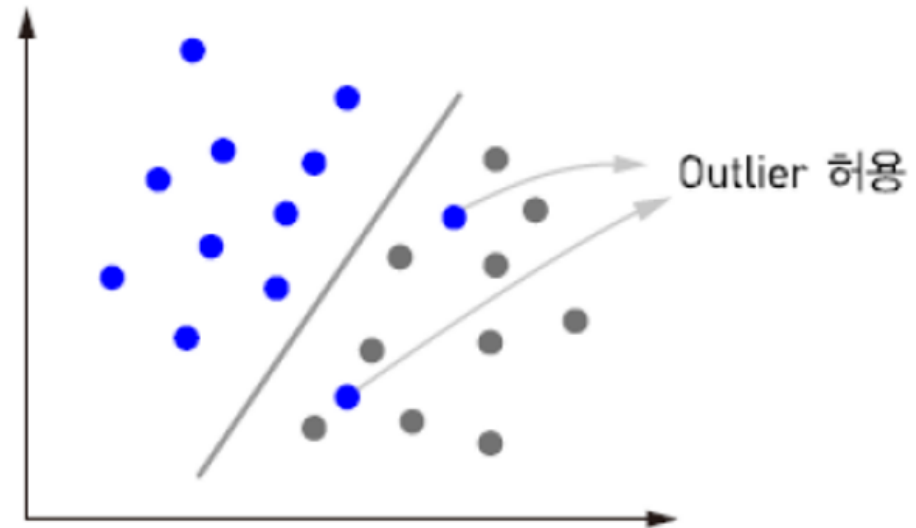
## (7) SVM (Support Vector Machine)

### Hard Margin SVM



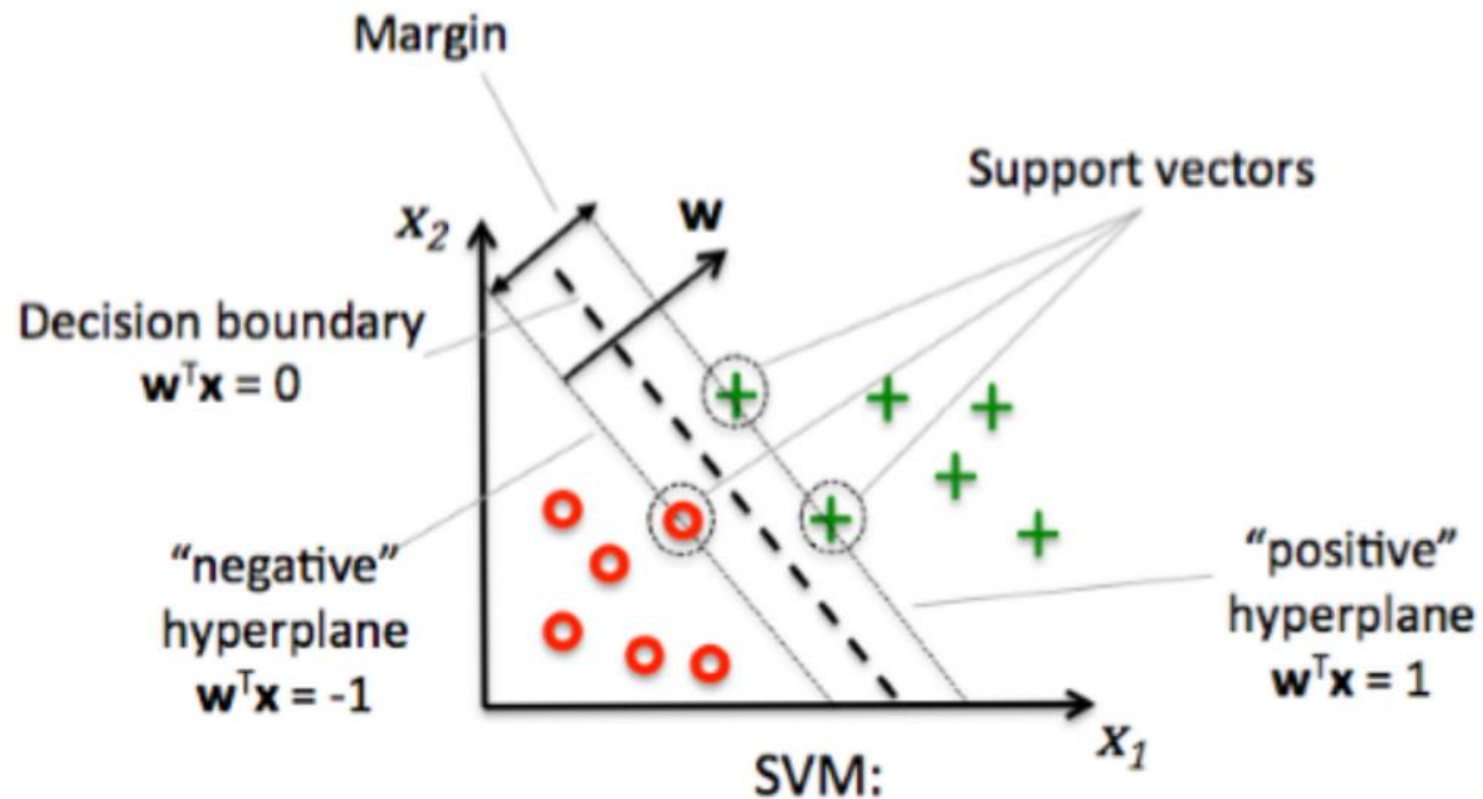
마진의 안쪽이나 바깥쪽에 절대로 잘못 분류된 분류를 허용하지 않는 SVM

### Soft Margin SVM



마진의 안쪽이나 바깥쪽에 잘못 분류된 분류를 허용하는 SVM

## (7) SVM (Support Vector Machine)



### 구성요소

Margin

Decision boundary

Support vectors

Hyperplan

감사합니다!



# 인공지능(AI)



## 2. 인공지능 알고리즘2

## 1. 지도학습(Supervised Learning)

### 1) 분류(classification)

- 의사결정 트리(Decision Tree)
- 랜덤 포레스트(Random Forest)
- 앙상블
- KNN (K-Nearest Neighbor)
- SVM (Support Vector Machine)

### 2) 회귀(regression analysis)

- 선형 회귀(Linear Regression)
- 로지스틱 회귀(Logistic Regression)

## 2. 비지도학습

### 1) 군집

- K-MEANS
- DBSCAN

### 2) 차원축소

- PCA(Principal Component Analysis)

## 3. 강화학습

- Q-Learning
- MCTS

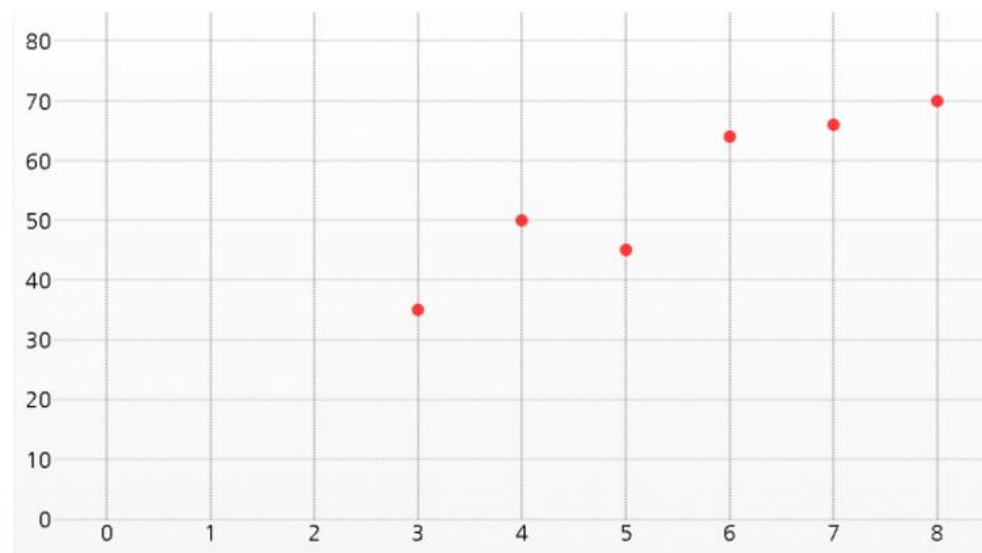
## (1) 선형회귀(Linear Regression Analysis)

### □ 선형회귀 분석 개념

- 알려진 데이터 종속변수  $y$  와 독립변수  $x$  와의 상관관계를 모델링(공식화)하여 알려지지 않은 데이터를 예측하는 기법
- 선형회귀 분석에서 종속변수란 우리가 알고 싶어 하는 결과값 이고, 독립변수란 이러한 결과값에 영향을 주는 입력값 임

<주중 공부시간에 따른 점수>

| 시간 | 획득점수 |
|----|------|
| 3  | 35   |
| 4  | 50   |
| 5  | 45   |
| 6  | 64   |
| 7  | 66   |
| 8  | 70   |



4.5시간 공부하면 획득점수는??



## (1) 선형회귀(Linear Regression Analysis)

**목적 : 오차의 평균을 최소화**할 수 있는 최적의 **회귀방정식**을 찾는 것

### ① 변수

- 독립변수 : 다른 변수에 영향을 미치는 변수, 원인
- 종속변수 : 다른 변수에 영향을 받는 변수, 결과

### ② 회귀 방정식

- 실제 값들을 가장 잘 설명할 수 있도록 직선을 만드는 방정식
- 변수 간의 관계를 함수식으로 정리할 수 있다면, 특정값 예측 용이
- 회귀 방정식은 실제값/예측값 간의 오차(잔차)를 최소로 만드는게 목적
- $Y = mx + b$  (기울기  $m$ , 절편  $b$ )

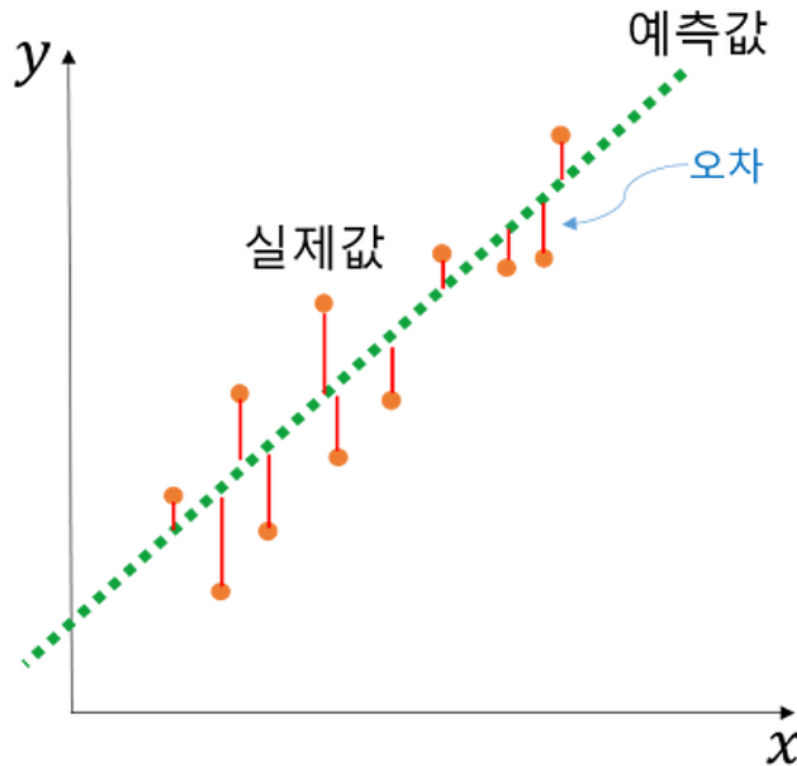
### ③ 최소 자승법(method of least squares) 또는 최소 제곱법

- 실제값과 예측값 간의 차이
- MSE(Mean Square of Errors)
- MAE(Mean Absolute of Errors)
- SSE(Error sum of squares)

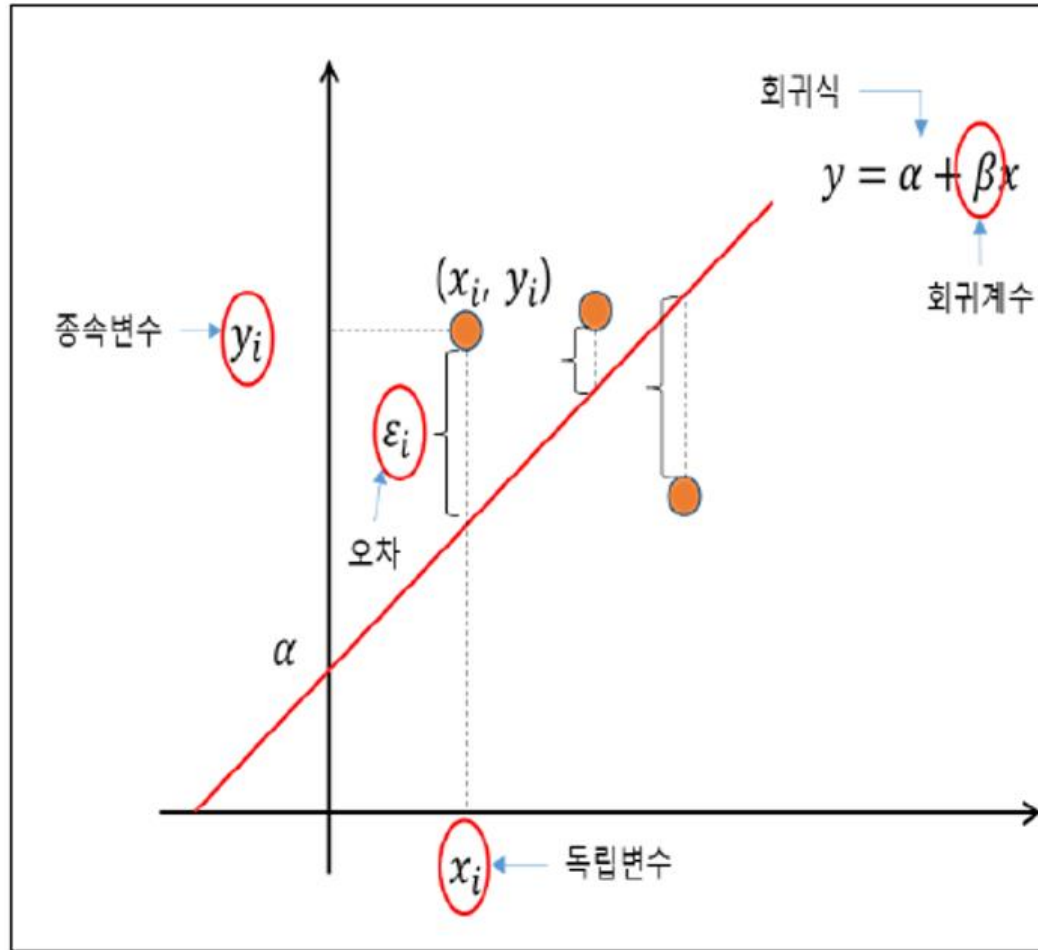
## (1) 선형회귀(Linear Regression Analysis)

### ④ 결정계수 ( $R^2$ , R-SQUARED)

- $x$ 가  $y$ 를 얼마나 잘 설명/예측하는가를 알려주는 통계량
- 최소 제곱법으로 나온 값이 결정계수



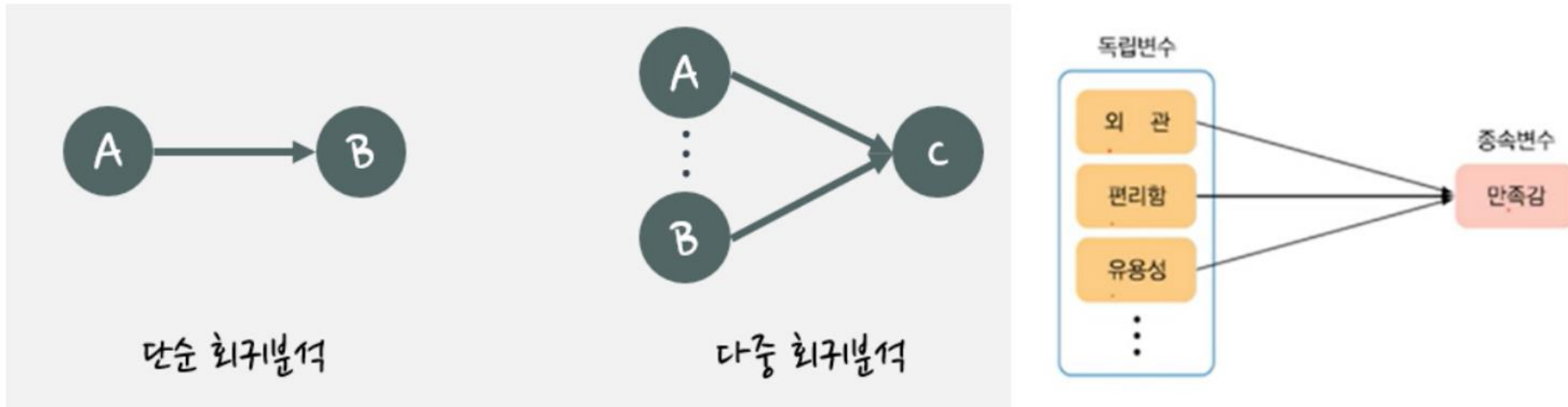
## (1) 선형회귀(Linear Regression Analysis)



| 요소    | 설명                                                             |
|-------|----------------------------------------------------------------|
| 독립변수  | -입력값이나 원인이 되는 변수                                               |
| 종속변수  | -독립변수에 의해 결과물이나 효과                                             |
| 회귀계수  | -단위 시간에 따라 변하는 양 (기울기)                                         |
| 최소자승법 | -산포도에 위치한 각 점에서 회귀선에 수직으로 이르는 값의 제곱의 합이 최소가 되는 선을 최적의 회귀선으로 추정 |
| 회귀방정식 | -회귀선의 수학적 함수로 표현한 식                                            |

## (2) 다중회귀(Multiple Regression Analysis)

- 종속변수에 영향을 미치는 독립변수가 여러 개인 경우에 분석하는 방법

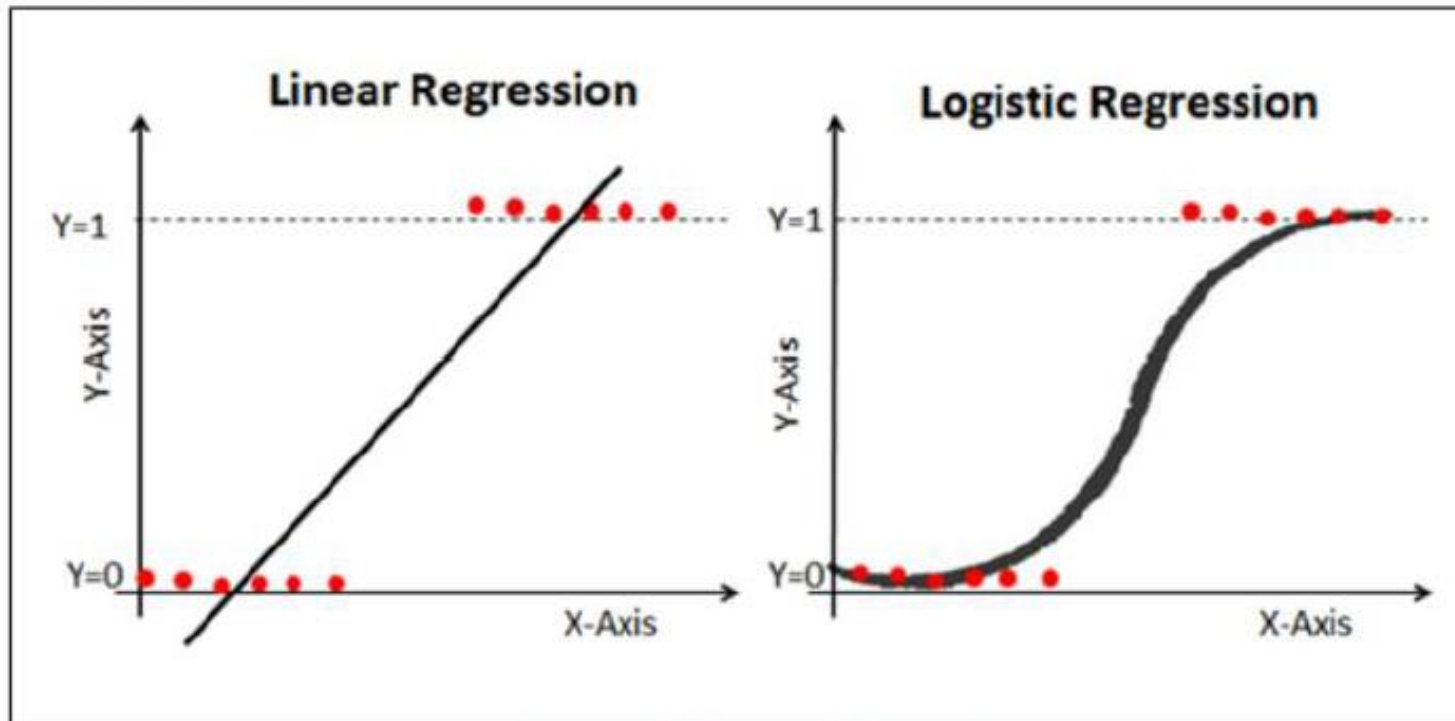


### (3) 로지스틱 회귀분석(Logistic Regression Analysis)

#### □ 로지스틱 회귀분석 개념

- 데이터가 어떤 범주에 속할 확률을 0에서 1 사이의 값으로 예측하고 그 확률에 따라 가능성이 더 높은 범주에 속하는 것으로 분류하는 알고리즘

#### □ 로지스틱 회귀분석 개념도



### (3) 로지스틱 회귀분석(Logistic Regression Analysis)

- odds : 어떤 일이 일어날 승산 값

$$\text{odds} = \frac{\text{사건 발생함}}{\text{사건 발생안함}} = \frac{p}{1-p}$$

- 만약 학생이 0.7 확률로 시험에 합격한다면, 떨어질 확률은 0.3으로 계산

$$\text{Odds of passing} = \frac{0.7}{0.3} = 2.\overline{33}$$

- log-odds를 **Sigmoid 함수**에 넣어서 0부터 1사이의 값으로 변환

$$\text{Log odds of passing} = \log(2.\overline{33}) = 0.847$$

- 확률이 50% 이상 이면 해당 사건이 발생, 이하는 안발생

## 비지도 학습(Unsupervised Learning)

### 1. 비지도학습 개요

#### □ 비지도학습 개념

- 지도 학습과는 달리 정답을 알려주지 않고 컴퓨터에게 결과값 없이 입력값만 주고 학습시키는 방법

#### □ 비지도학습 유형

##### ① 군집(Classification)

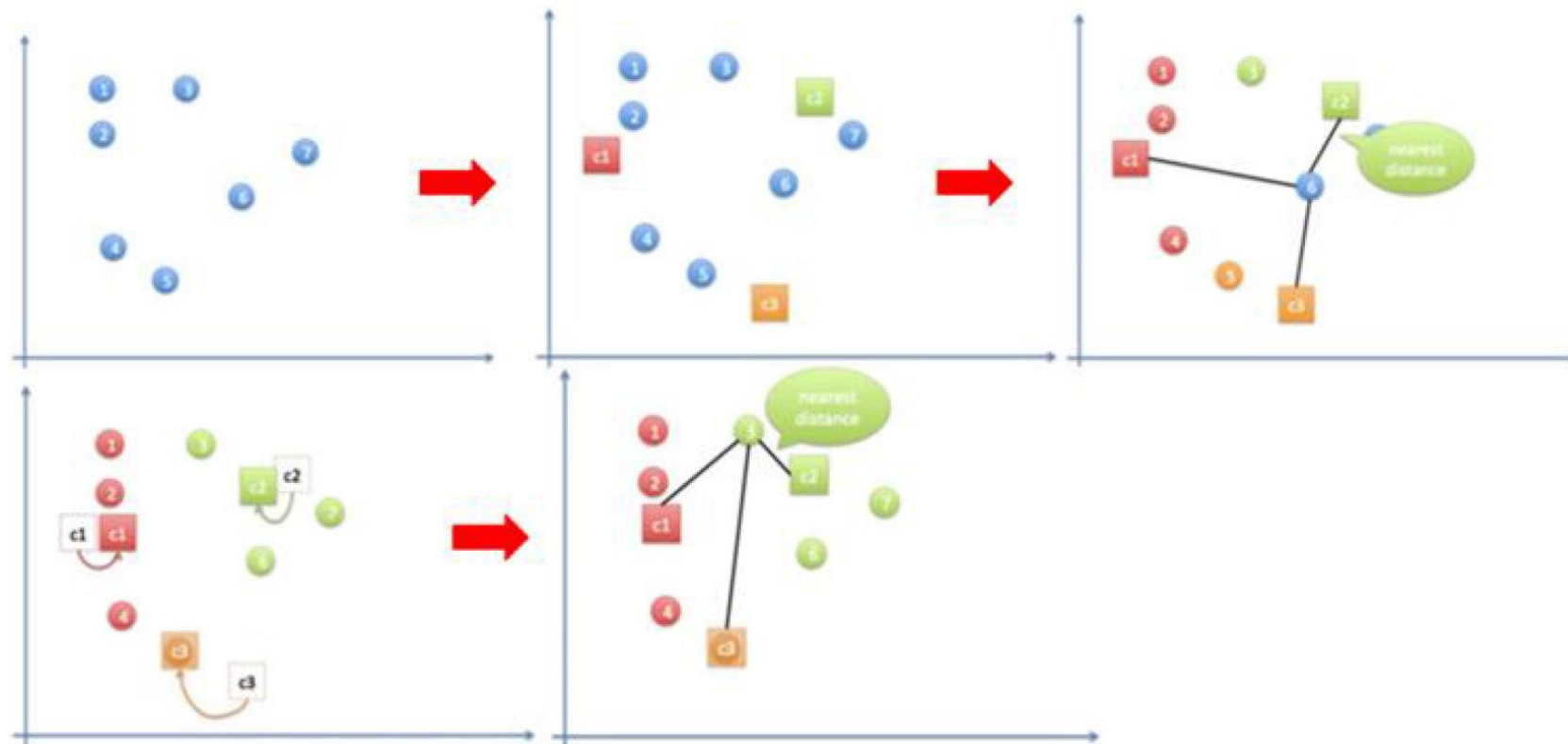
- 비슷한 것들끼리 군집으로 묶어주는 분석방법

##### ② 차원축소

- 데이터 입력값의 특성을 분석해서 출력값을 더 잘 설명해줄 수 있는 새로운 독립적인 입력 특성을 추출하는 방법

## (1) K-Means 알고리즘

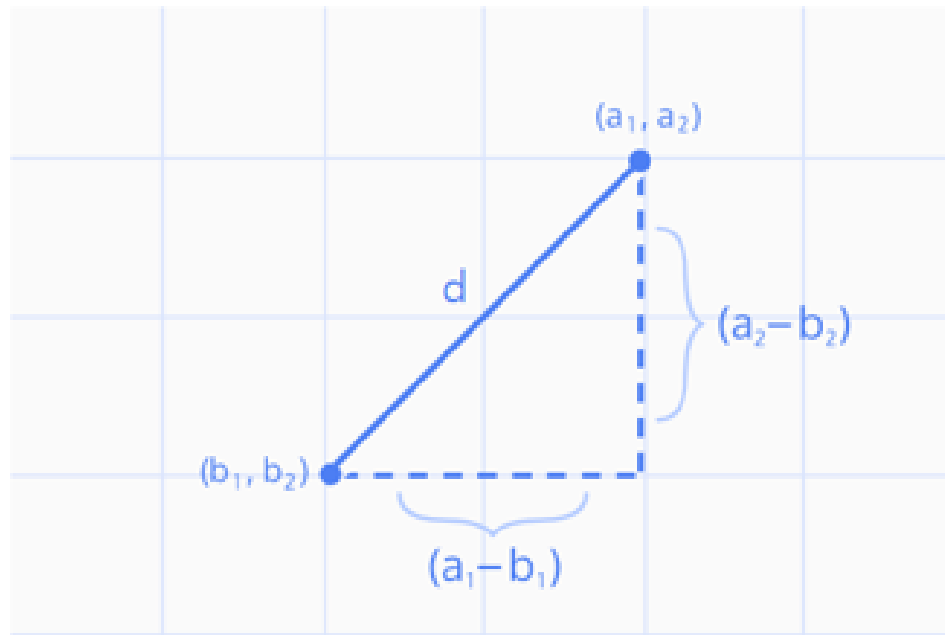
- 유클리드 거리 기반으로, 주어진 데이터를  $k$  개의 클러스터(군집)로 묶는 알고리즘
- 묶여진 군집 내에서 중심점을 이동하고, 이동한 중심을 기준으로 가장 가까운 그룹에 설정하여 군집 생성
- 이를 반복적으로 수행 후 만약 군집이 변함이 없다면 프로세스 종료





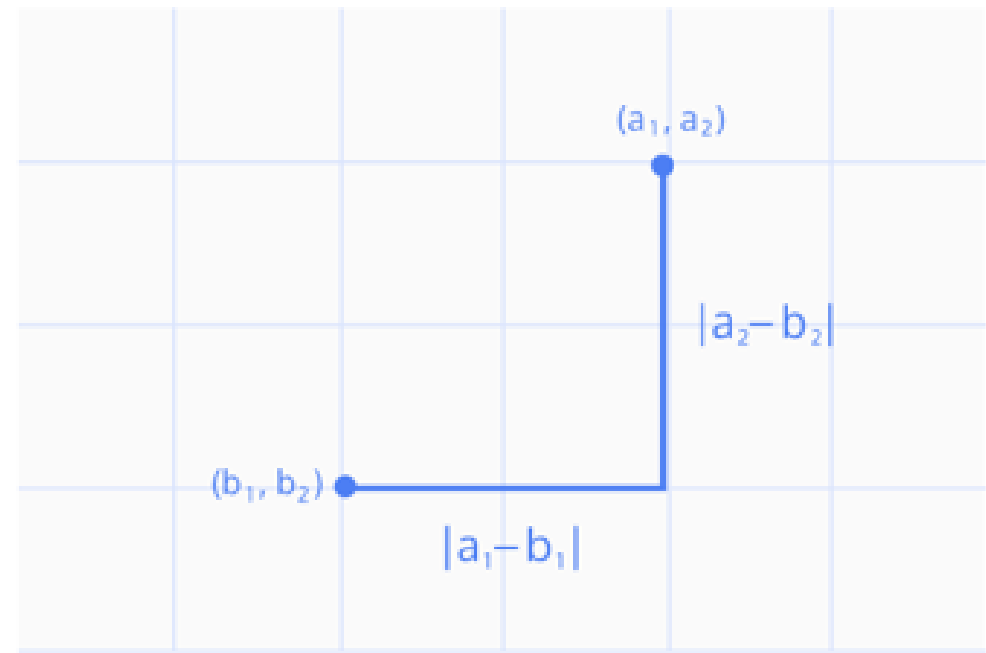
## (1) K-Means 알고리즘

- 유클리드 거리 (Euclidean Distance)
- 맨하탄 거리 (Manhattan Distance)



$$d = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

[유클리드 거리]



$$d = |a_1 - b_1| + |a_2 - b_2|$$

[맨하탄 거리]

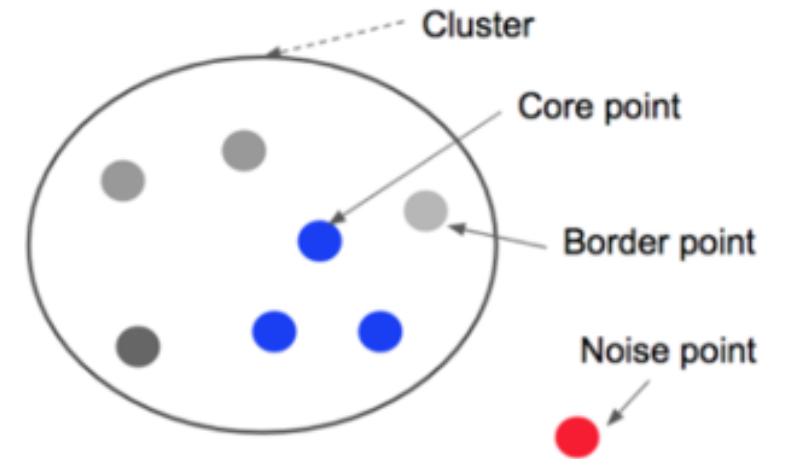
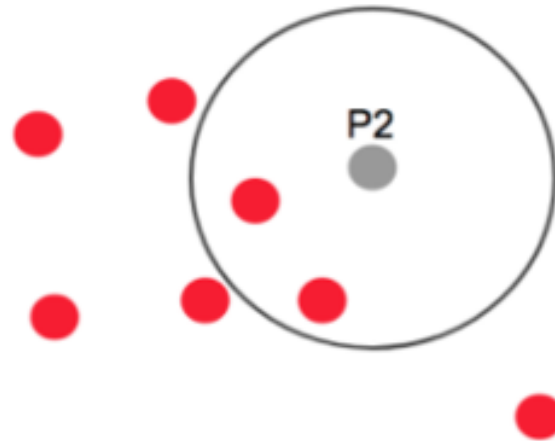
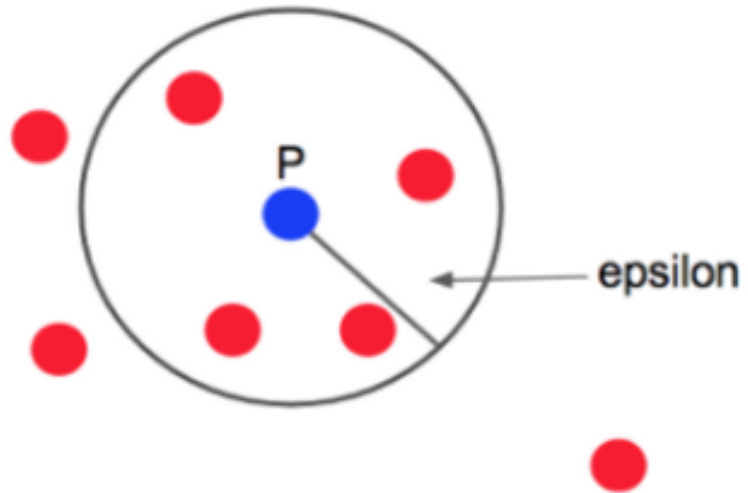
## (2) DBSCAN

- 밀도를 기반으로 하여 군집화하는 알고리즘
- K-means 이나 계층적 군집 알고리즘의 경우 데이터 간의 거리를 이용하여 클러스터를 나누는 데에 반해, DBSCAN 알고리즘은 데이터 포인트가 세밀하게 몰려 있어 밀도가 높은 부분을 군집화하는 방식

| 구성요소         | 설명                                                                       |
|--------------|--------------------------------------------------------------------------|
| Epsilon      | - 기준으로 부터의 거리                                                            |
| minPts       | - 반경내에 있는 점의 수                                                           |
| core point   | - 점을 중심으로 epsilon 반경내에 minPts 이상수의 점이 포함된 포인트                            |
| Border Point | - 군집의 중심이 되는 core point 는 되지 못하지만, 앞의 점 P 를 core point 로 하는 군집에는 속하는 포인트 |
| Nosie Point  | - 어느 군집에도 속하지 않는 outlier 가 되는 포인트                                        |

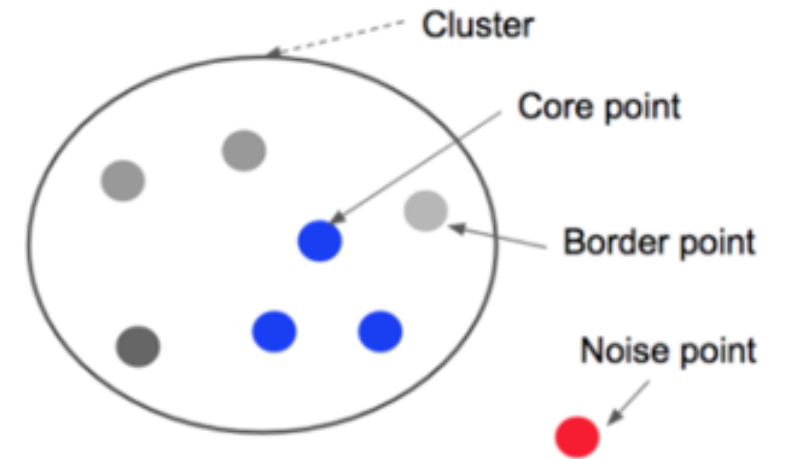
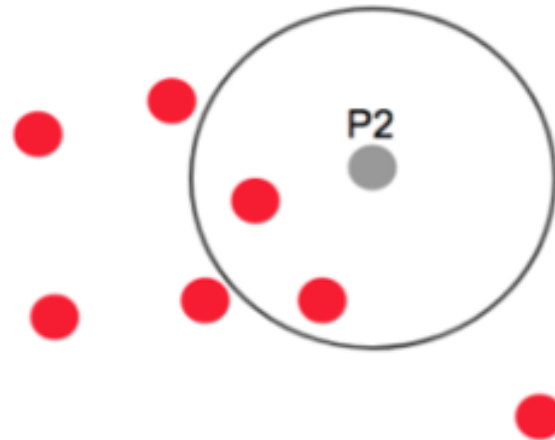
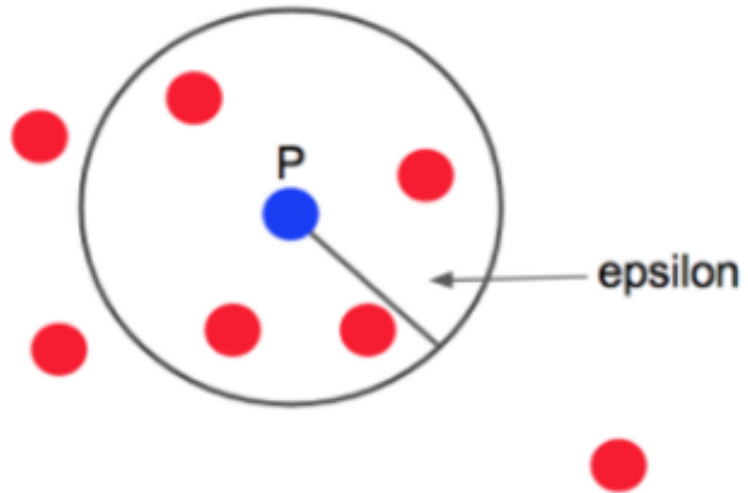
## (2) DBSCAN

<minPts = 4>



## (2) DBSCAN

<minPts = 4>



<개념도>

## (2) DBSCAN

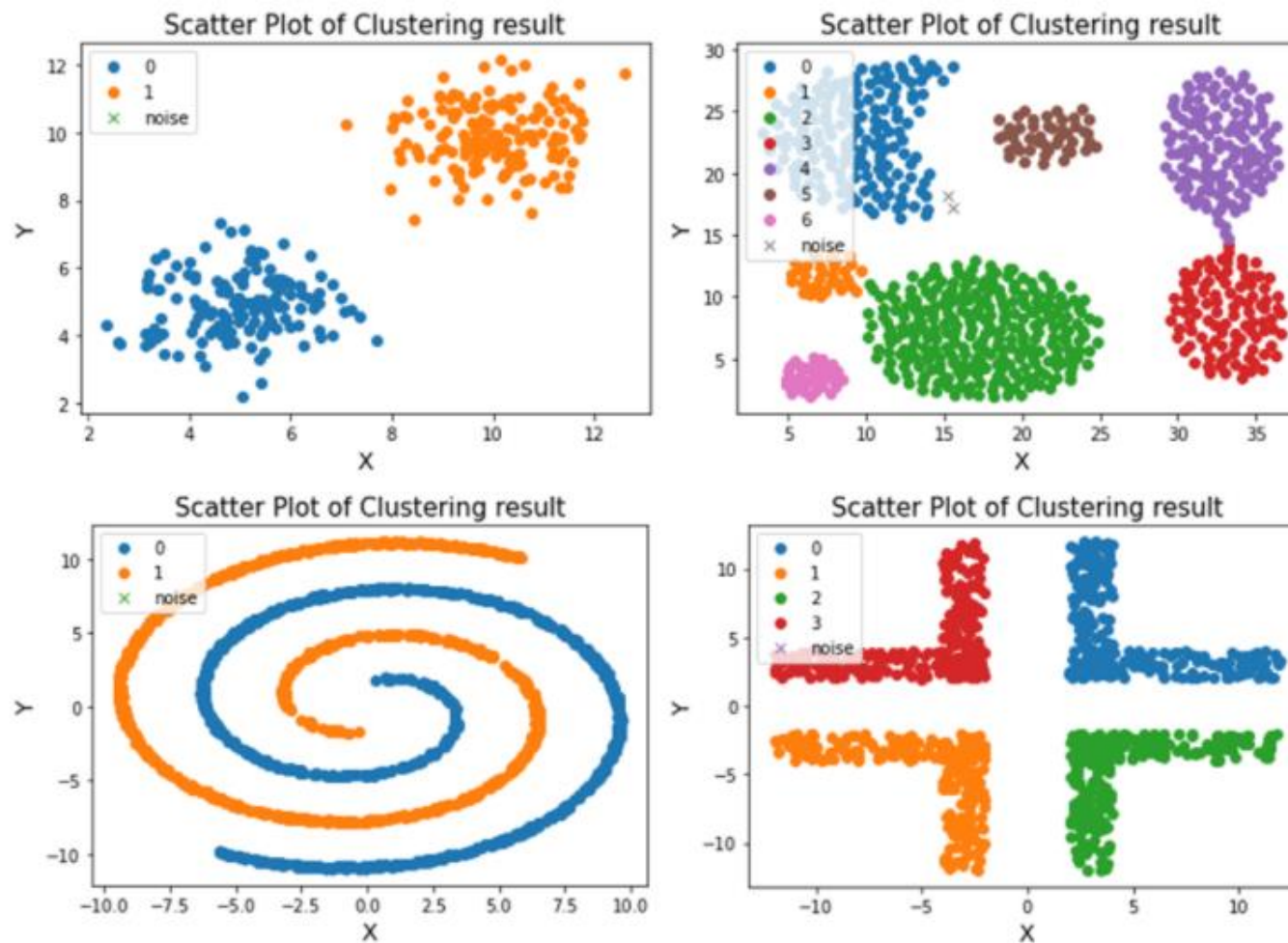


그림 4. DBSCAN 결과

## (1) PCA(Principal Component Analysis)

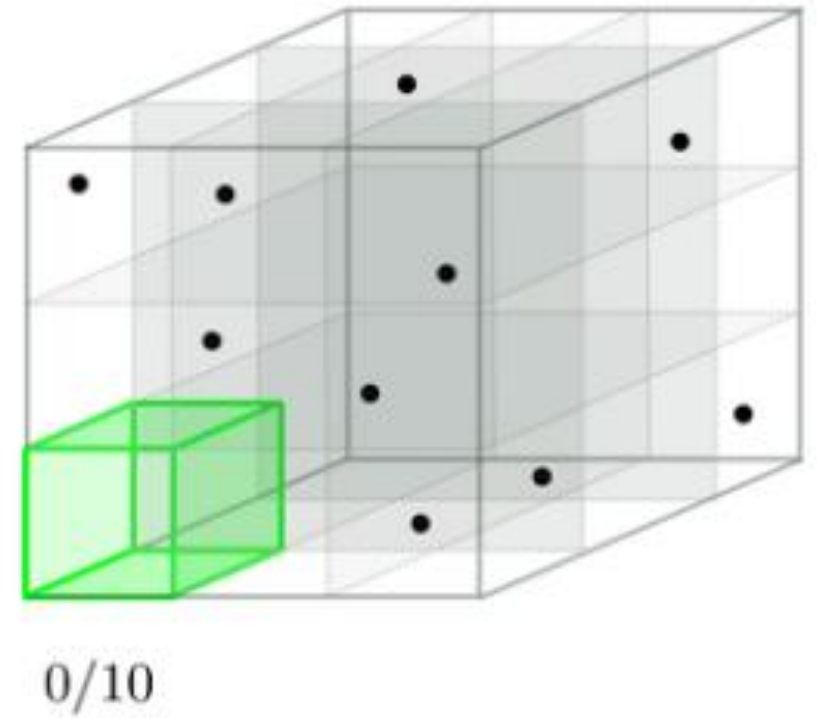
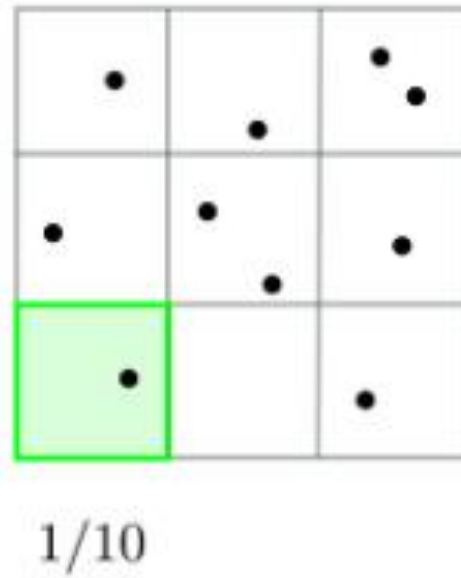
### □ PCA 등장배경

- 고차원의 데이터에서는 학습량이 기하급수적으로 증가하여 메모리 문제가 생기는데, 이를 차원의 저주
- 차원의 저주는 차원이 증가할 수록 성능이 비례하여 증가하지 않고, 학습 시간이 증가하는 현상
- 고차원의 데이터들은 의미를 제대로 표현하기 어렵기 때문에 이러한 데이터들을 저차원으로 축소를 해야 함

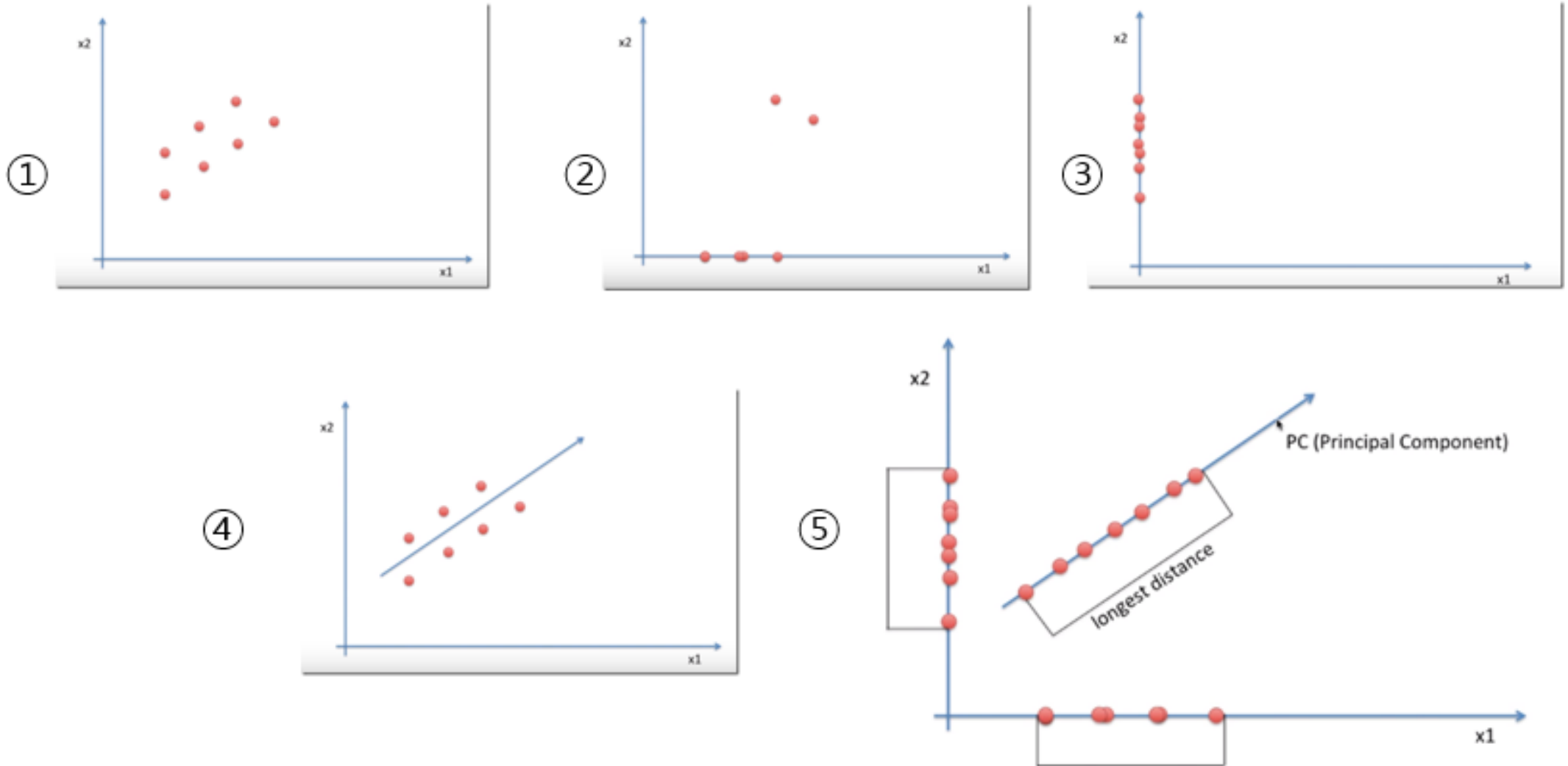
### □ PCA 개념

- 다차원 공간에 데이터가 분포되어 있을 때 가장 효율적인 최적의 축을 식별하여 차원을 축소하는 알고리즘
- 즉, 고차원의 데이터를 저차원으로 압축하는 변환하는 차원 축소기법

## (1) PCA(Principal Component Analysis)



# (1) PCA(Principal Component Analysis)





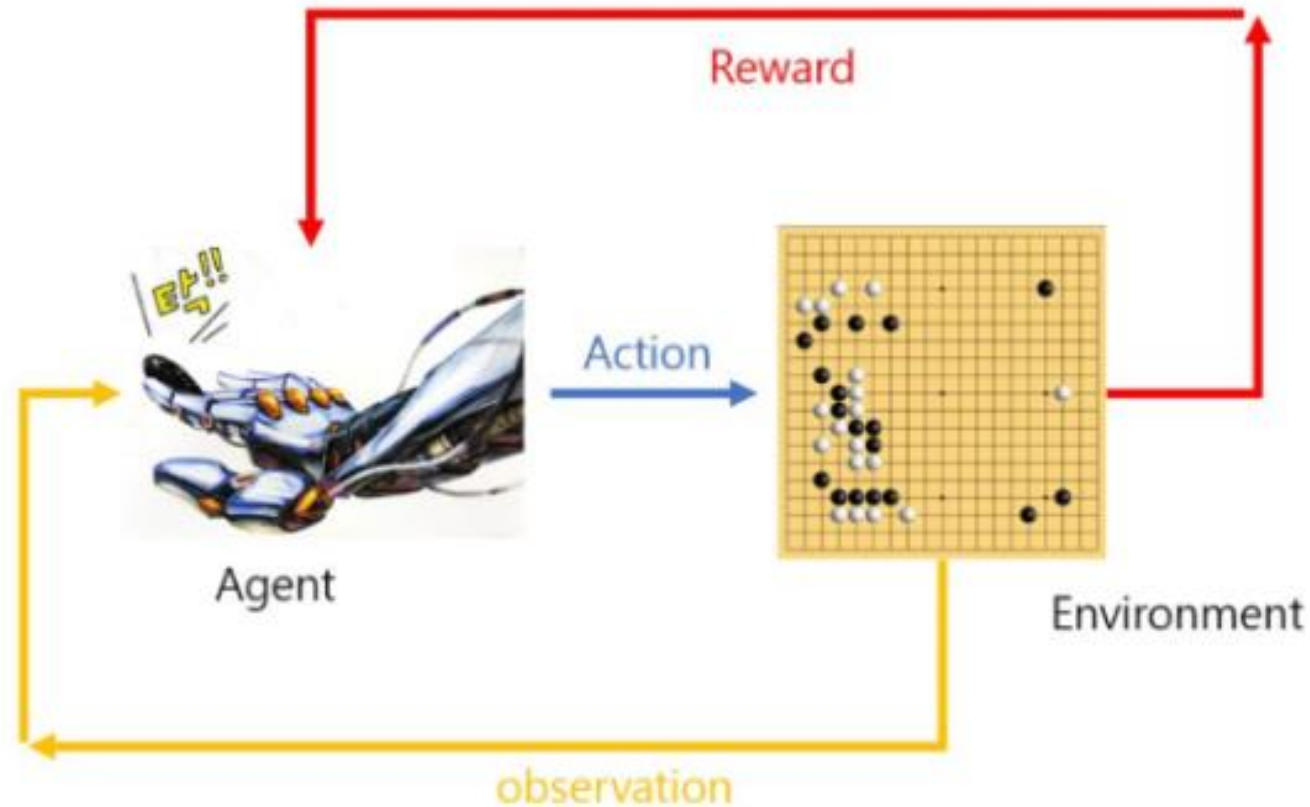
## (1) PCA(Principal Component Analysis)

| 구성요소                    | 설명                                             |
|-------------------------|------------------------------------------------|
| 분산(variance)            | - 데이터가 평균으로부터 얼마나 떨어져 있는가                      |
| 공분산(Covariance)         | - 점들의 그룹과 다른 점들의 그룹과의 관계                       |
| 고유 벡터<br>(Eigen vector) | - 어떤 벡터에 선형 변환을 했을 때, 방향의 변화 없이 벡터의 크기만 변하는 벡터 |
| 고유 값(Eigen value)       | - scaled size                                  |

- Eigen value 가 높으면, 분산 즉 퍼져있는 정도가 높으므로, 공분산행렬에서 Eigen value 가 가장 높은 녀석의 Eigen vector 값을 기준으로 점들을 옮겨주면 PCA 가 완료됨

## 강화학습(Reinforcement Learning)

- 주변 상태에 따라 어떤 행동을 할지 판단을 내리는 주체인 에이전트가 있고, 에이전트가 속한 환경 존재하여 에이전트가 행동을 하면 그에 따라 상태가 바뀌게 되고, 보상을 받을 수도 있음
- 강화학습은 에이전트(agent), 환경(environment), 상태(state), 행동(action), 보상(reward) 등으로 구성



## (1) Q-Learning

- Agent가 특정 상황에서 **보상을 최대화** 하는 최적의 Policy을 찾는 방법
- **Q-Value(행동가치함수)** : Action을 취했을 때, 종료되기까지 reward의 총합 예측값
- "Q"라는 단어는 현재 상태에서 취한 행동의 보상에 대한 quality 를 의미



- ① 위 그림에서 start 지점에서 goal 지점까지 간다고 가정할 때 start 지점에서 취할 수 있는 행동은 위, 아래, 왼쪽, 오른쪽으로 이동하는 것 네가지
- ② 이때의 행동을 action 이라고 하고, 현재의 위치(1, 1)를 state(상태)라고 함
- ③ 만약 start 지점에서 왼쪽 또는 위로 가는 action 은 Goal 지점까지 갈수 없으므로 올바른 행동이 아님
- ④ q-learning 은 보상을 통해 현재 state 에서 어떤 action 을 취하는게 좋은지 학습하는 과정
- ⑤ Q-Value(행동가치함수)는 Action 을 취했을 때, 종료되기까지 reward 의 총합 예측 값으로 Agent 가 Action 할 때마다 보상을 받아 Q 값이 갱신됨

## (2) MCTS (Monte Carlo Tree Search)

### □ MCTS 개념

- 시뮬레이션을 통해 가장 승률이 좋은 행동을 하도록 하는 알고리즘
- MCTS 는 알파고에 사용된 알고리즘으로 현재 바둑, 체스 등의 모든 보드 게임 알고리즘에서 사용됨

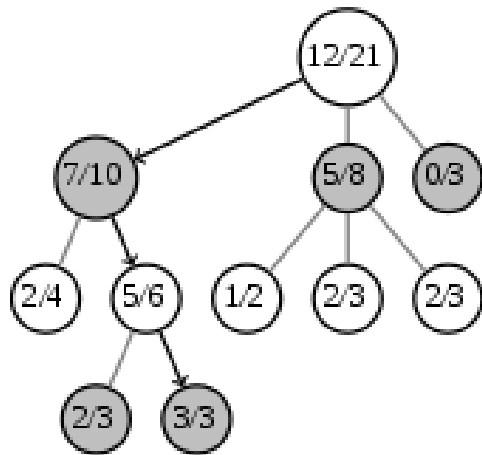
### □ MCTS 조건

| 조건 | 설명           |
|----|--------------|
| 1  | 최대, 최소 값이 존재 |
| 2  | 게임 규칙의 존재    |
| 3  | 게임 길이 제한     |

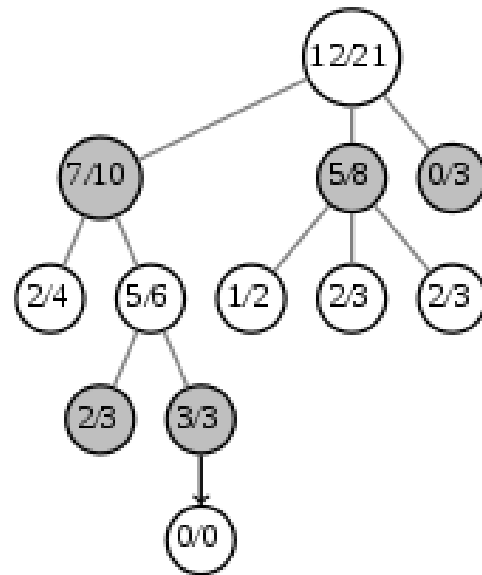
- MCTS 는 다음 3 가지 조건을 만족할 때 사용 가능

## (2) MCTS (Monte Carlo Tree Search)

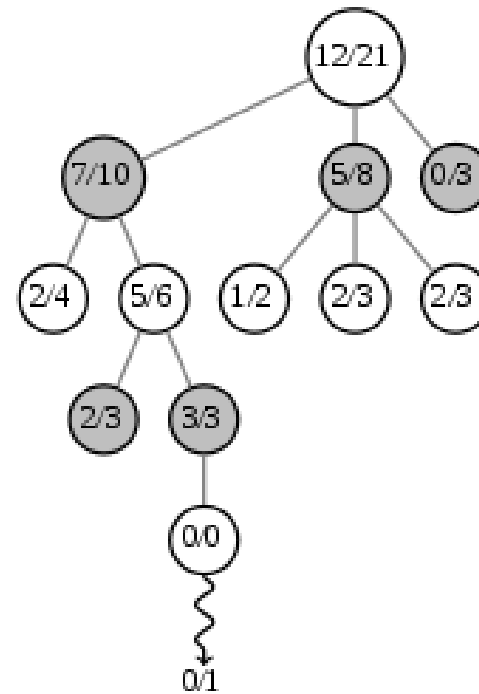
Selection



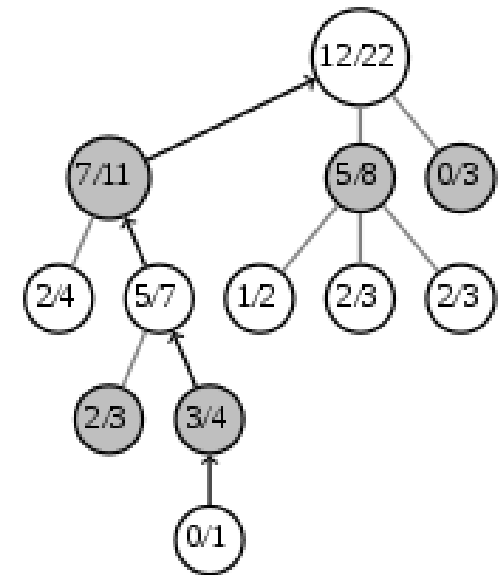
Expansion



Simulation



Backpropagation



- Selection(선택), Expansion(확장), Simulation(시뮬레이션), Backpropagation(역전파)

감사합니다!



# 인공지능(AI)



## 3. Model 학습

## 1. 학습 데이터

- 1) 합성데이터(Synthetic Data)
- 2) 차원의 저주(The curse of dimensionality)
- 3) 오버피팅(overfitting)
- 4) 언더피팅(underfitting)

## 2. 모델평가

- 1) Cross Validation
- 2) 혼동행렬(Confusion Matrix)



## (1) 합성데이터(Synthetic Data)

- 인공지능 학습 시, 의미 있는 데이터 확보 어려움
- **개인정보를 보호하면서, 불충분한 데이터로 극복**
- 정의 : 실제 데이터에서 존재하는 **통계, 패턴을 모방**하여 인공적으로 만들어진 **가짜 데이터**



합성데이터 생성 및 활용.(자료=금융보안원)

## (1) 합성데이터(Synthetic Data)

### 나. 합성 데이터 유형

| 유형        | 핵심                     | 설명                                                           |
|-----------|------------------------|--------------------------------------------------------------|
| 완전 합성 데이터 | 정보 보호 측면 가장 강력한 보안성 제공 | 공개하려고 하는 데이터에 측정된 실제 데이터가 하나도 없이 모두 가상으로 생성된 데이터로만 이루어진 데이터  |
| 부분 합성 데이터 | 민감한 정보에 관한 변수 대치       | 공개하려는 변수들 중 일부만을 선택하여 합성 데이터로 대체한 데이터                        |
| 복합 합성 데이터 | 실제 데이터와 합성 데이터 혼합      | 합성 데이터와 실제 데이터를 모두 이용하여 또 다른 일부 변수들의 값을 다시 도출하는 방법으로 생성된 데이터 |

- 개인정보를 포함하는 실제 데이터를 기반으로, 통계적 모형에 의해 임의로 생성된 가상 데이터

### 다. 합성데이터 평가지표

- 데이터의 효용성을 측정하고, 원 데이터와 유사성을 비교하는 품질지표 기법

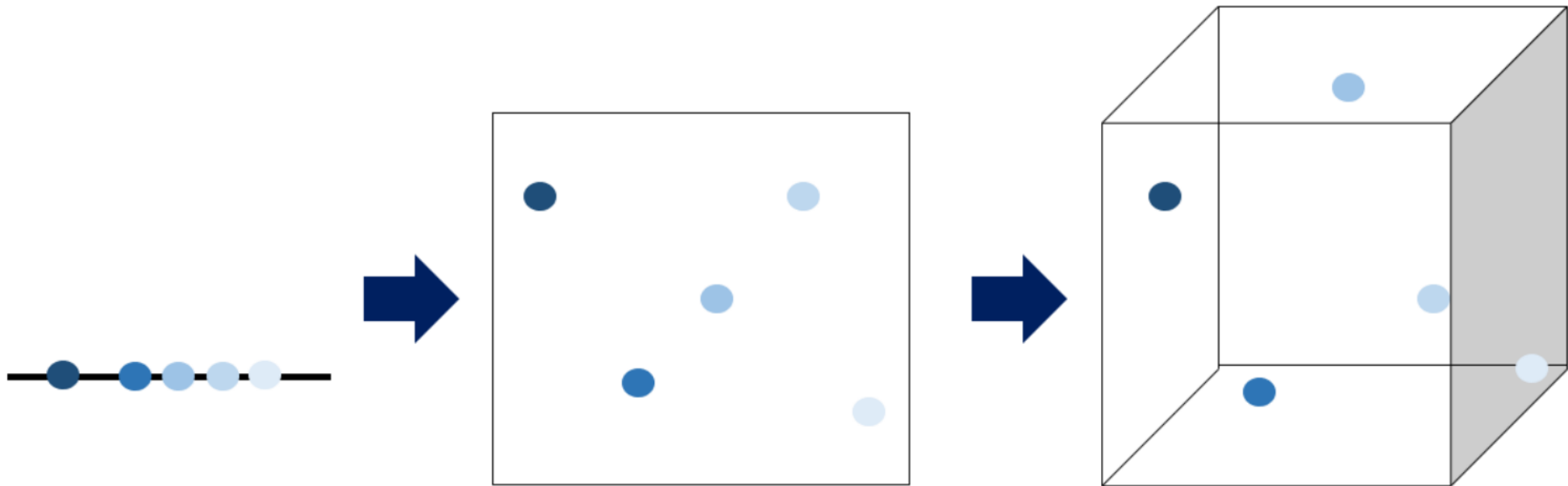
① **분산 비교** : 실제 데이터와 합성데이터 간의 분산을 비교/측정

② **헬링거 거리** : 실제 데이터와 합성데이터 사이의 거리 측정

③ **식별력**(Distinguishability) : 개발된 모델에서 데이터가 실제인지, 합성인지 결정하는 척도

## (2) 차원의 저주(The curse of dimensionality)

- 차원이란 : 0차원, 1차원, 2차원, 3차원, ...
- 정의 : 데이터 학습을 위해 차원이 증가하면서 학습 및 성능이 안 좋아지는 현상
- 메모리 사용량, 학습 시간, 성능 저하
- 해결 : PCA, LDA 등 차원축소



### (3) 오버피팅(overfitting)

- Training Set에 편중되어 Training Set에서의 오차는 작게 나타나지만 실제 Test Set에서는 오차가 커지는 과적합 현상
- 해결방안 : 배치 정규화(Batch Normalization), 일반화(Regularization), Cross Validation (교차 검증), Drop Out 등

열심히 뉴럴넷에게 고양이



를 가르쳤더니..



똥똥하니까 고양이가님

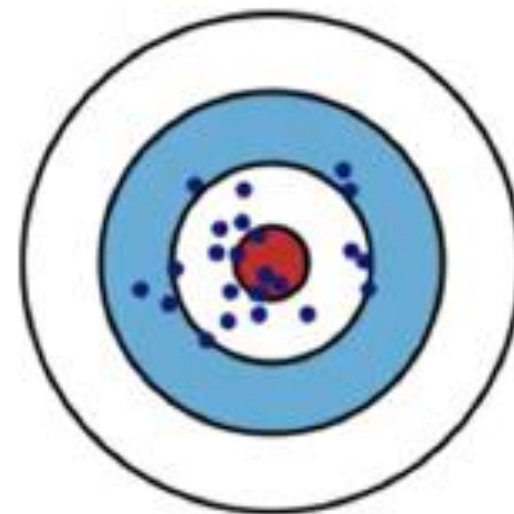


갈색이니까 고양이가님



귀쳐졌으니까 고양이가님

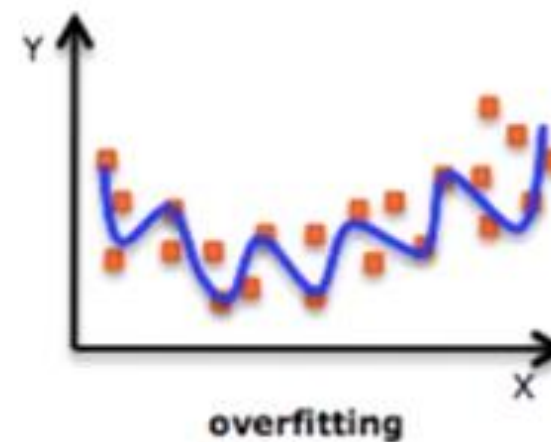
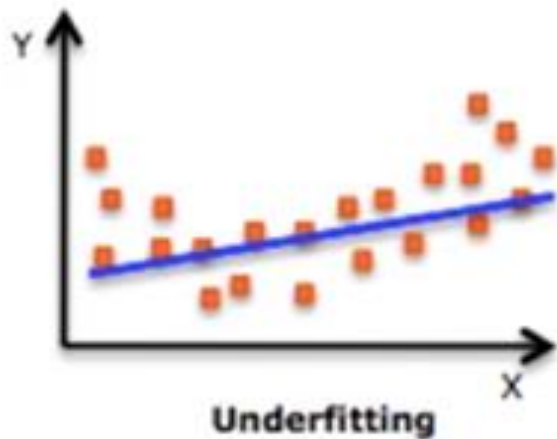
High Variance



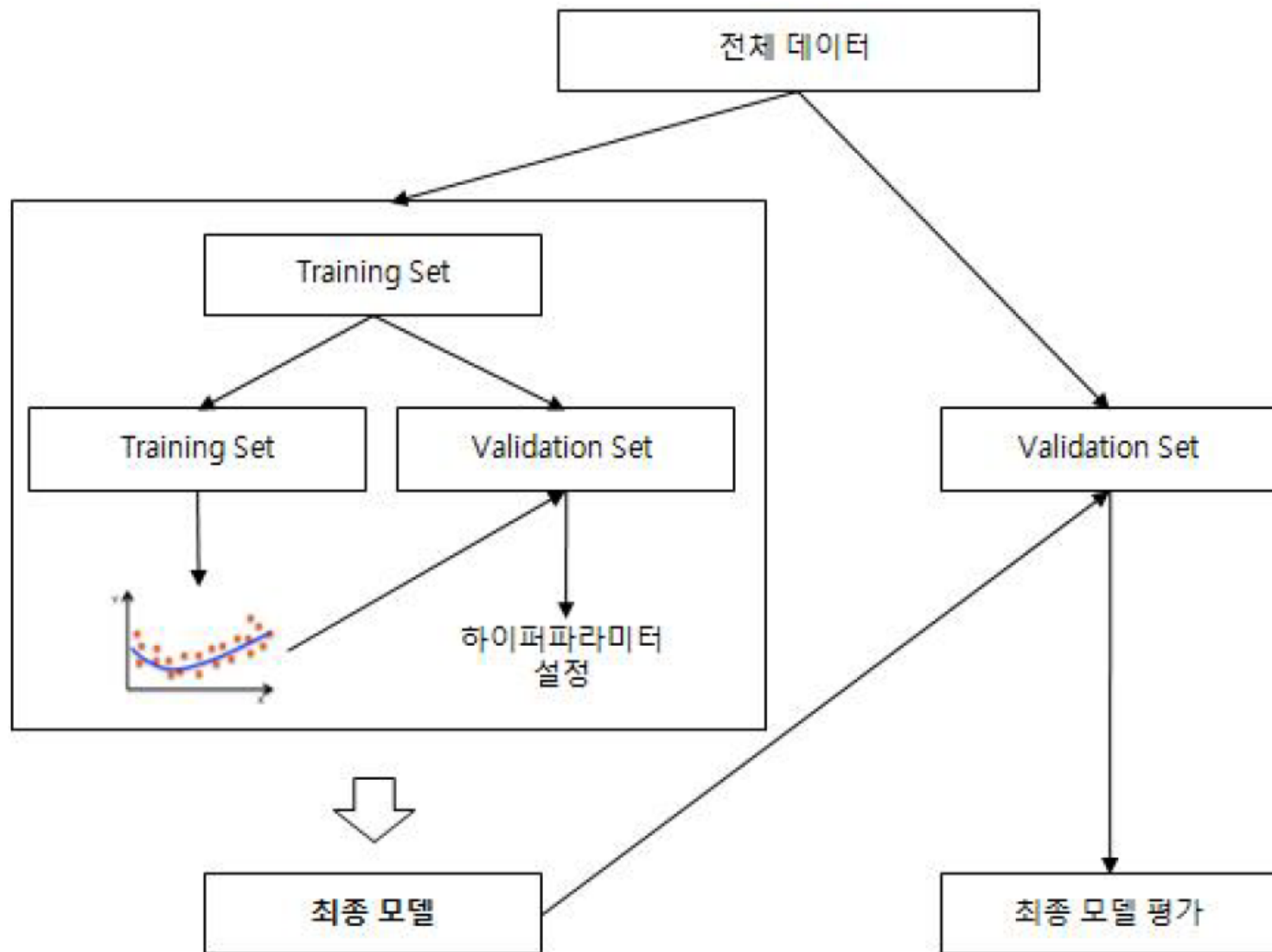
융통성이라곤 눈곱 만큼도 없다! **Overfitting**

### (3) 오버피팅(overfitting)

- 데이터 수집시 단편화 된 방법으로 인한 학습으로 적정 수준의 학습이 부족하여 실제 성능이 떨어지는 현상
- 해결방안 : feature더 많이 반영, variance가 높은 모델 사용 (의사결정나무, KNN, SVM)



## (1) 교차 검증(Cross Validation)



## (1) 교차 검증(Cross Validation)

- 오버피팅 방지하기 위해 Train Set 과 Validation Set 을 지정하여 검증하는 방법
- 학습데이터는 Training data, validation data, test data 이고 Training data 는 모델을 만드는데 사용하는 데이터이고 validation data 는 만든 모델이 잘 예측하는지 검증하기 위한 데이터이고, 마지막으로 Test data 는 최종 모델의 성능을 테스트하기 위한 test 데이터



## (1) 교차 검증(Cross Validation)

### ☐ holdout cross validation



- 가장 간단한 방법으로 데이터셋을 train data 와 validation data 로 나누어 사용하는 기법
- 보통 데이터셋을 8 : 2 로 나누어 많이 사용하지만 7 : 3, 9 : 1 비율로도 나누어 많이 사용



## (1) 교차 검증(Cross Validation)

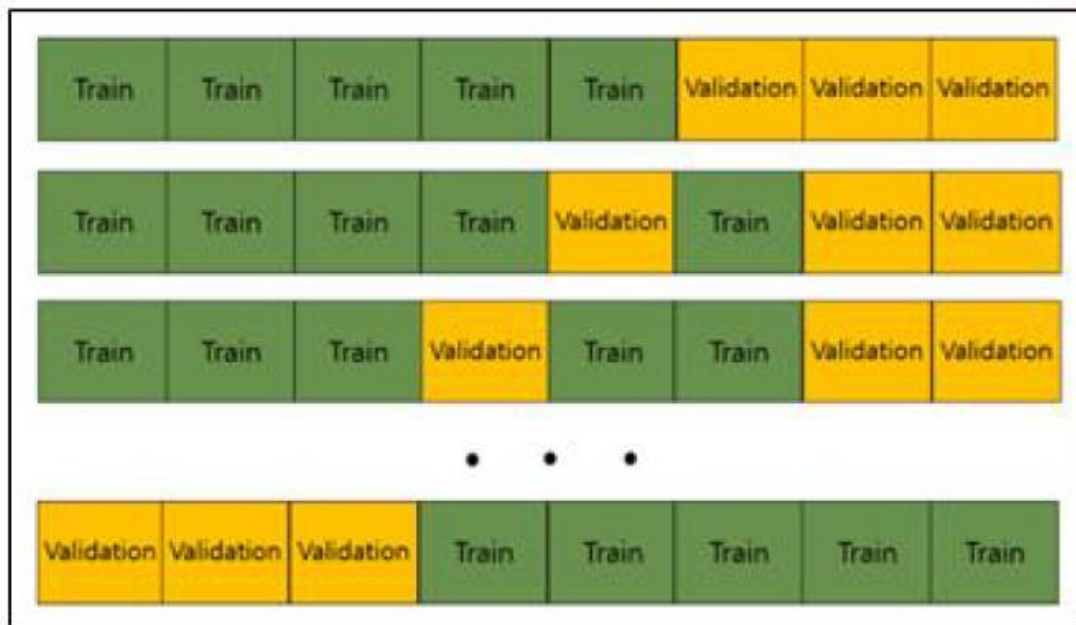
### □ k-fold cross validation



- 가장 일반적으로 사용하는 방법으로, 데이터를 K 개의 그룹으로 나누어 그 그룹 중에서 하나를 추출하여 validation set 으로 사용하는 기법

## (1) 교차 검증(Cross Validation)

### □ Leave-p-out cross validation

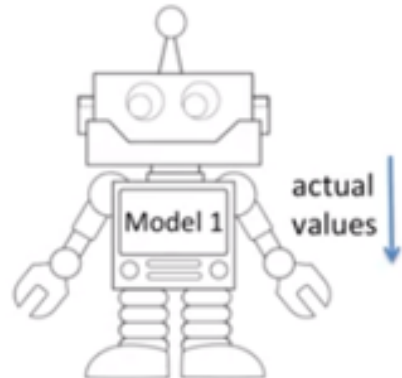


- 데이터 셋에서  $p$  개의 데이터를 추출하고, 남은 데이터를 validation data 로 사용하는 기법
- 예를 들어 Leave-3-out Cross Validation 은 데이터에서 3 개를 추출하여 검증데이터로 이용
- 그리고 나머지 데이터는 훈련 데이터로 사용하여 검증 결과를 도출
- Leave-p-out Cross Validation 는 계산 시간이 오래 걸린다는 단점이 있어 잘 사용하지 않음.

## (2) 혼동행렬(Confusion matrix)

- 모델에 의한 예측이 실제와 같은지 다른지 성능 평가하기 위한 matrix

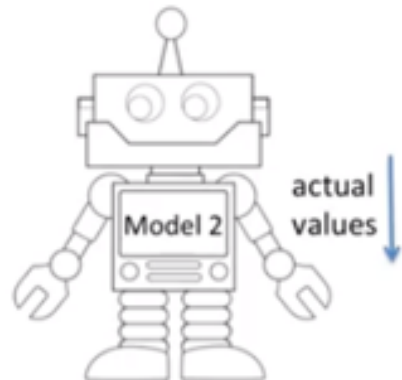
SVM



The diagram shows a robot labeled 'Model 1' on the left. A blue arrow labeled 'actual values' points from the robot to a confusion matrix table on the right. Above the table, a blue arrow labeled 'predictions' points to the right.

|   | A  | B | C | D |
|---|----|---|---|---|
| A | 10 | 0 | 0 | 0 |
| B | 0  | 5 | 3 | 2 |
| C | 0  | 1 | 8 | 1 |
| D | 0  | 1 | 0 | 9 |

K-means



The diagram shows a robot labeled 'Model 2' on the left. A blue arrow labeled 'actual values' points from the robot to a confusion matrix table on the right. Above the table, a blue arrow labeled 'predictions' points to the right.

|   | A | B | C | D |
|---|---|---|---|---|
| A | 8 | 2 | 0 | 0 |
| B | 1 | 7 | 0 | 2 |
| C | 0 | 0 | 9 | 1 |
| D | 2 | 3 | 0 | 5 |

## (2) 혼동행렬(Confusion matrix)

|    |    | 실제 |    |
|----|----|----|----|
|    |    | 양성 | 음성 |
| 예측 | 양성 | 정답 | 오답 |
|    | 음성 | 오답 | 정답 |

|    |    | 실제                              |                                |
|----|----|---------------------------------|--------------------------------|
|    |    | 양성                              | 음성                             |
| 예측 | 양성 | True Positive                   | False Positive<br>Type I Error |
|    | 음성 | False Negative<br>Type II Error | True Negative                  |

- 1) TP(True Positive) - 맞다고 예측했는데, 실제 맞았을 경우 (정답)
- 2) FP(False Positive) - 맞다고 예측했는데, 실제 틀렸을 경우 (오답)
- 3) TN(True Negative) - 틀렸다고 예측했는데, 실제 틀렸을 경우 (정답)
- 4) FN(False Negative) - 틀렸다고 예측했는데, 실제 맞았을 경우 (오답)

## (2) 혼동행렬(Confusion matrix)

| 지표              | 수식                                                  | 설명                                                                                                                                              |
|-----------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Accuracy (정확도)  | $\frac{TP + TN}{TP + FN + FP + TN}$                 | - 전체 예측 건수에서 정답을 맞힌 건수의 비율                                                                                                                      |
| Precision (정밀도) | $\frac{TP}{TP + FP}$                                | - 내가 예측한 값 중에서 정답이 있을 경우<br>- 1에 가까울수록 좋음                                                                                                       |
| Recall (재현도)    | $\frac{TP}{TP + FN}$                                | - 실제로 정답이 True 인 것들 중에서 분류기가 True 로 예측한 비<br>- 즉, True 가 발생하는 확률이 적을 때 사용하면 좋음                                                                  |
| F1 Score        | $\frac{2 * Precision * Recall}{Precision + Recall}$ | - Precision 과 Recall 의 조화평균<br>- recall, precision 둘 중 하나가 0에 가깝게 낮을 때 지표에 그것이 잘 반영되도록, 다시말해 두 지표를 모두 균형 있게 반영하여 모델의 성능이 좋지 않다는 것을 잘 확인하기 위해 사용 |

감사합니다!



# 인공지능(AI)



## 4. 자연어처리

# **1. 자연어처리 패러다임 변화**

## **2. 워드 임베딩**

- 1) 원-핫 인코딩(One-hot Encoding)
- 2) 워드투벡(Word2Vec)

## **3. RNN(Recurrent Neural Network)**

## **4. Attention**

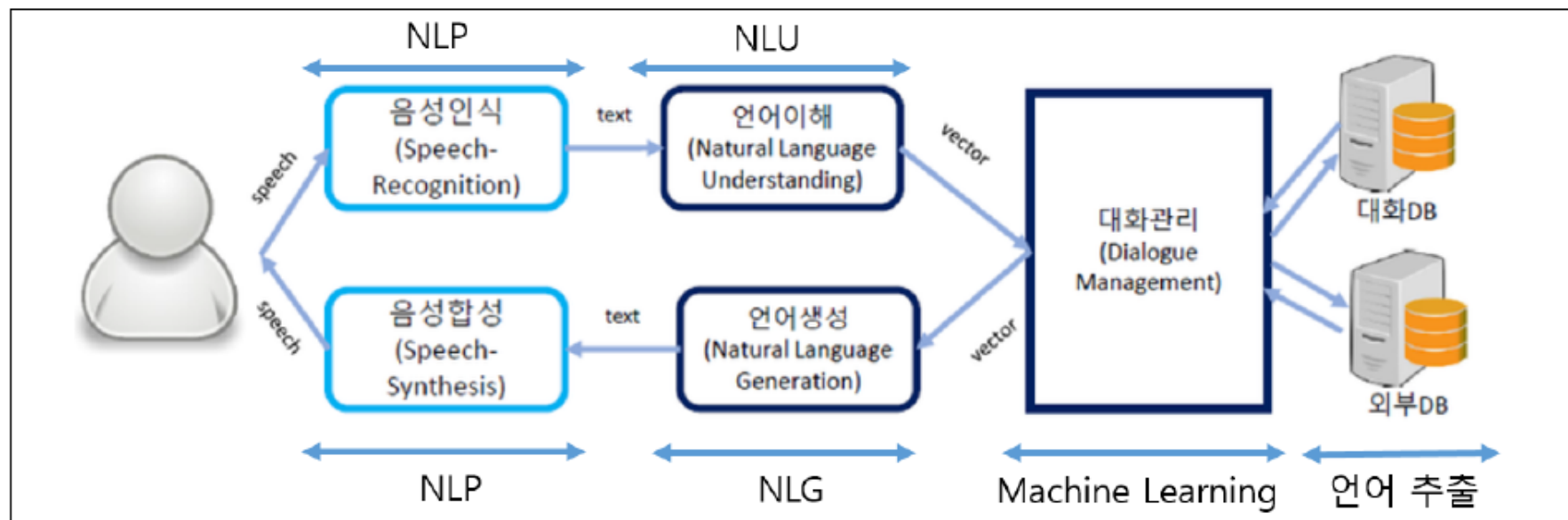
## **5. Transformer**



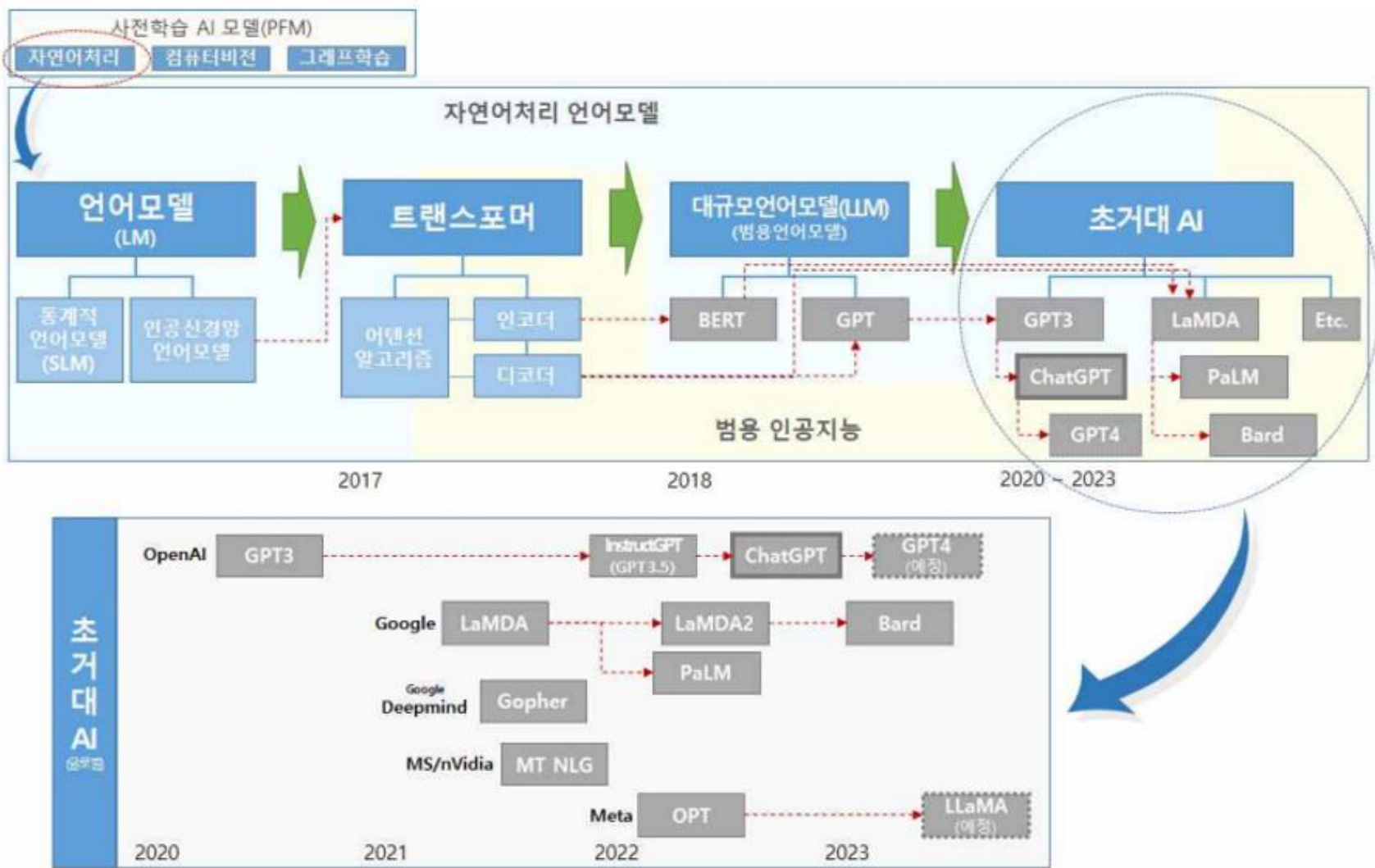
## (1) 자연어처리 패러다임 변화

### NLP(Natural Language Processing, 자연어 처리)

- 컴퓨터가 인간의 언어를 이해하고 상호 작용하게 만드는 과정으로 인공지능의 핵심 기술(자연어 처리)
- NLU : 자연어를 기계가 이해할 수 있도록 변환
- NLG : 기계의 언어를 자연어로 만들어 내는 과정



# (1) 자연어처리 패러다임 변화



[그림2-1] 언어모델의 개념 정의와 변화양상

## ( 2 ) Word Embedding

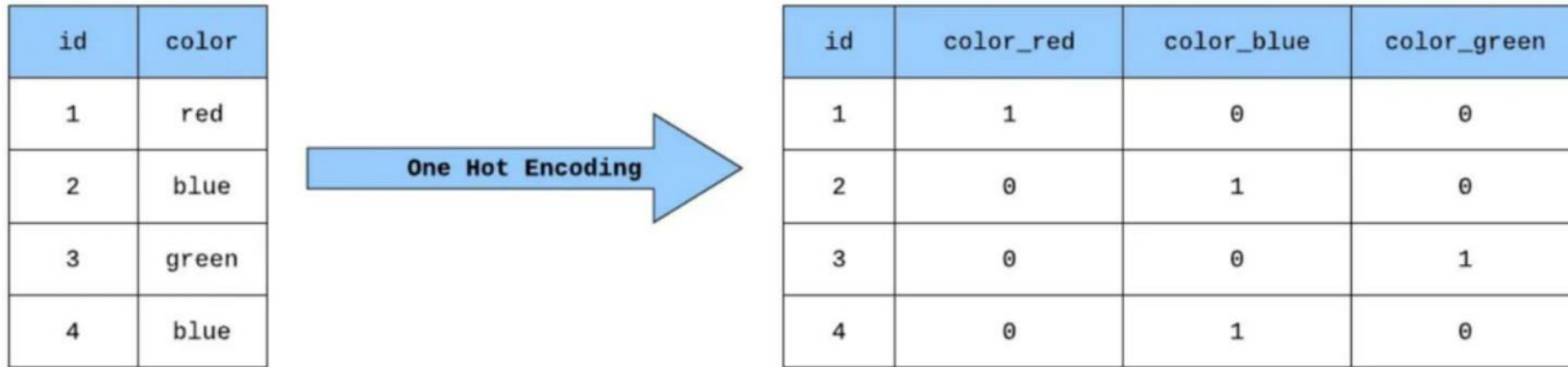
### □ 원핫 인코딩 개념

- 인간의 언어(자연어)는 수치화 되어 있지 않은 데이터이기 때문에 컴퓨터가 이해하거나, 바로 사용할 수가 없음
- 원핫 인코딩은 자연어 처리에서 문자를 숫자로 바꾸는 기법 중 하나로, 범주형 데이터를 수치형으로 변환하기 위해 하나의 데이터만 1, 나머지는 0 으로 변환하는 벡터 표현 방식

### □ 원핫 인코딩 이해

- 원핫 인코딩은 데이터를 수많은 0 과 한 개의 1 의 값으로 데이터를 구별하는 인코딩 기법
- 예를 들어, 글자가 적힌 종이를 0 부터 9 까지의 숫자값으로 분류 한다면 숫자값들은 아래와 같은 값들로 이루어질 수 있음

## ( 2 ) Word Embedding

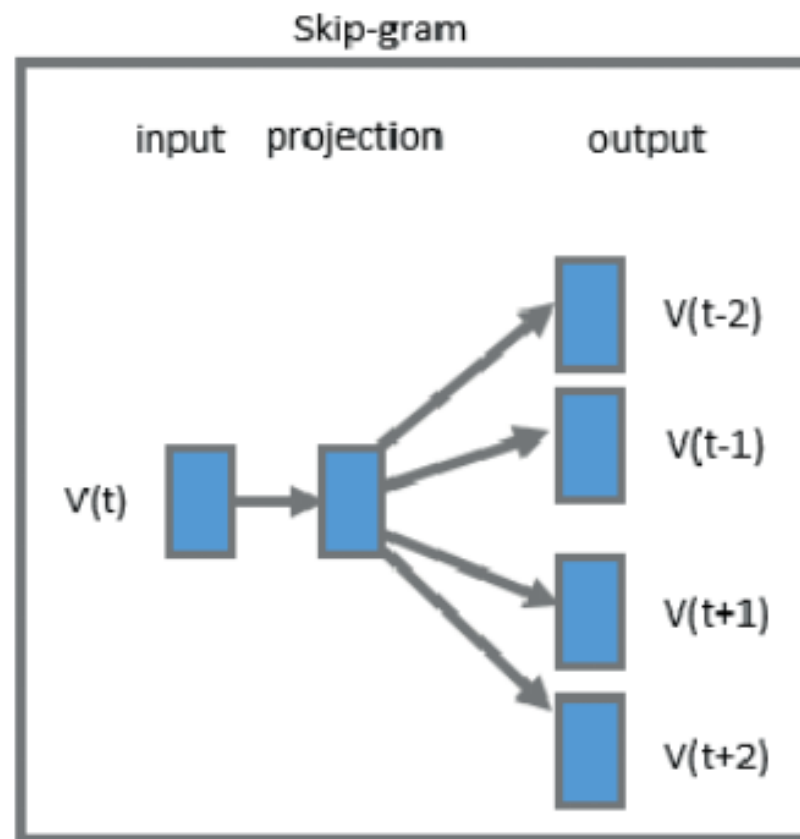
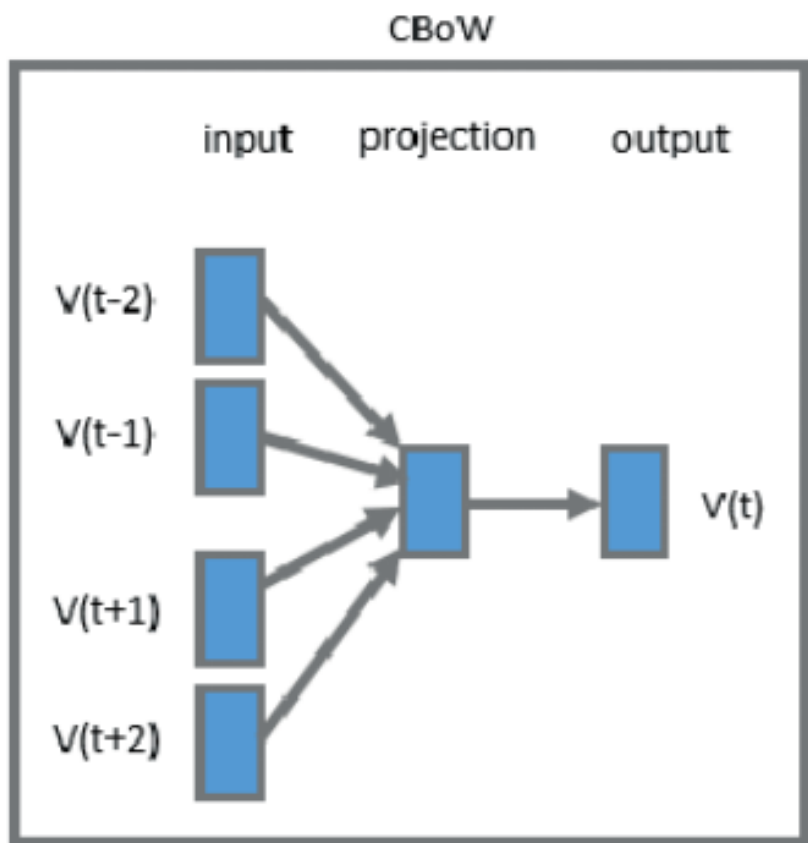


- 위 그림과 같이 **0으로 구성된 벡터에** , 몇개의 **1의 값으로 해당 데이터의 값을 구별**하는 것이 원핫 인코딩 임.
- 원핫 인코딩은 매우 기초적이면서도 아주 흔히 사용되는 기법으로 , 딥러닝 뿐만 아니라 데이터 마이닝, 자연어 처리 등 아직도 많은 분야에서 사용되고 있음.
- **Sparse Representation (희소 표현) = sparse vector**  
 벡터 또는 행렬(matrix)의 값이 대부분이 0으로 표현되는 상황

## ( 2 ) Word Embedding

### □ Word2Vec 개념

- 자연어처리를 위해 단어의 내포된 의미를 단어 주변의 단어들을 조합하여 벡터로 표현한 뒤 어떠한 상황에서 그 단어를 사용할 때 가장 적합한지를 예측하는 확률 모델



### ( 3 ) RNN(Recurrent Neural Network)

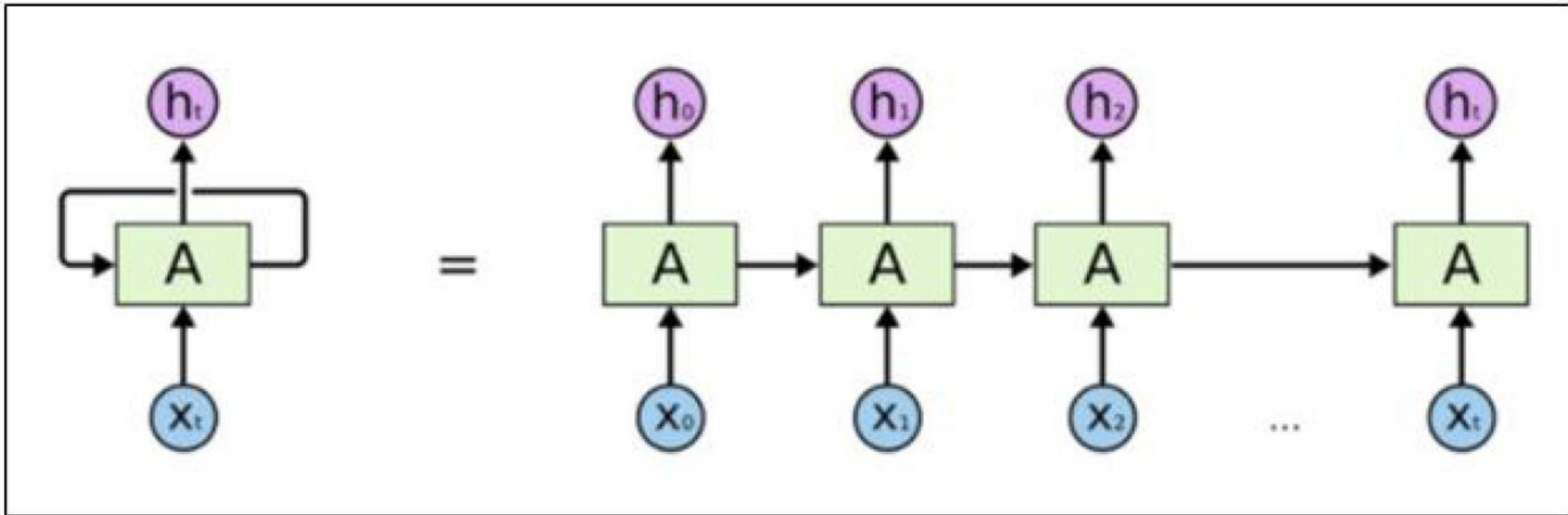
---

#### □ RNN 개념

- 시간의 흐름이 있는 sequence data 를 분석하는 신경망
- 신경망에 새로운 정보가 들어오면 기억을 조금씩 수정하여 전체를 요약하는 정보 출력
- Sequence data 의 대표적인 형태로는 '문장'과 같은 데이터가 있는데, 현재의 단어만으로 의미를 해석하는 것이 아니라 앞 단어와의 관계를 통해서 현재 단어의 의미를 해석
- 이외에도 손글씨, 음성 신호, 센서가 감지한 데이터, 주식 등을 처리하는데 자주 활용

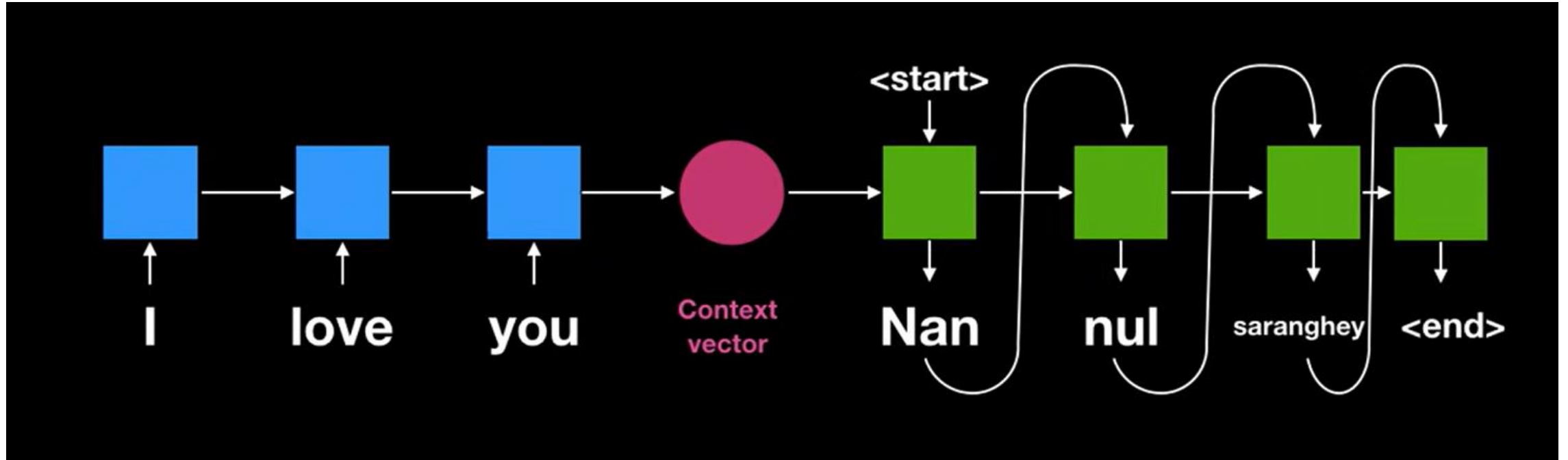
### ( 3 ) RNN(Recurrent Neural Network)

#### □ RNN 구성도



- 먼저 입력이 들어오면 첫번째 기억이 만들어지고, 다음 입력이 들어오면 기존의 기억과 새로운 입력을 참고하여 새 기억을 생성
- RNN 은 이 기억을 바탕으로 출력을 만들어 낸다는 개념

### ( 3 ) RNN(Recurrent Neural Network)



- RNN 의 장점은 이전의 데이터를 기억하여 다음 데이터에 활용할 수 있다는 것
- 이는 데이터들의 간격이 크지 않을 경우에 유용하며, 만약 많은 데이터들이 들어오고 데이터들이 복잡하고 많아지게 되면 뒤로 갈수록 앞쪽 데이터의 기억이 점점 작아져서 의미가 사라지는 문제가 발생
- 이를 장기 의존성 문제라고 하고 이를 해결하기 위해 LSTM, GRU 알고리즘이 등장

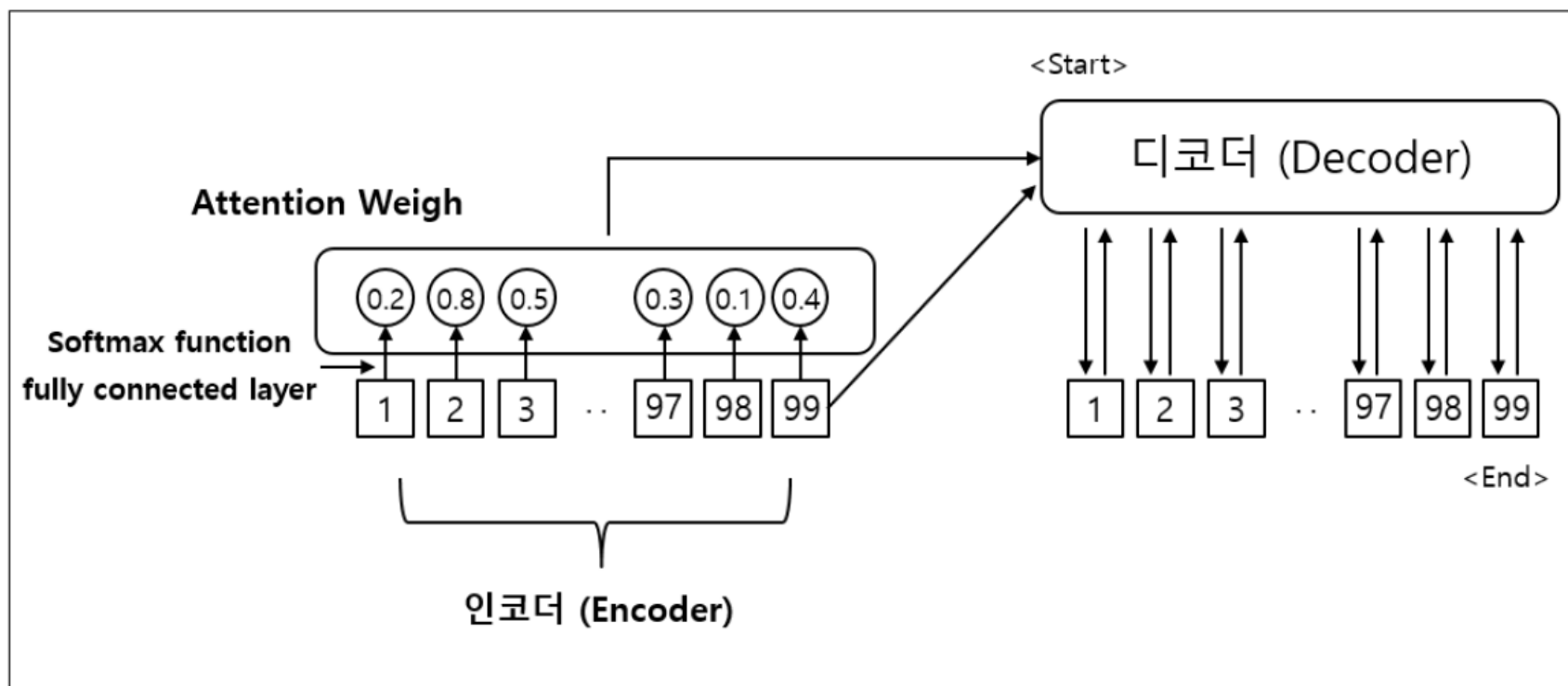


### ( 3 ) Attention

#### □ Attention 매커니즘 개념

- 입력 문장이 긴 상황에서는 번역 품질이 떨어지는 Vanishing Gradient 현상이 나타났고, 이런 현상을 보정하기 위해 중요한 단어에 집중하여 Decoder 에 바로 전달하는 기법
- 입력 시퀀스가 길어지면 출력 시퀀스의 정확도가 떨어지는 것을 보정해주기 위한 등장

#### □ Attention 매커니즘



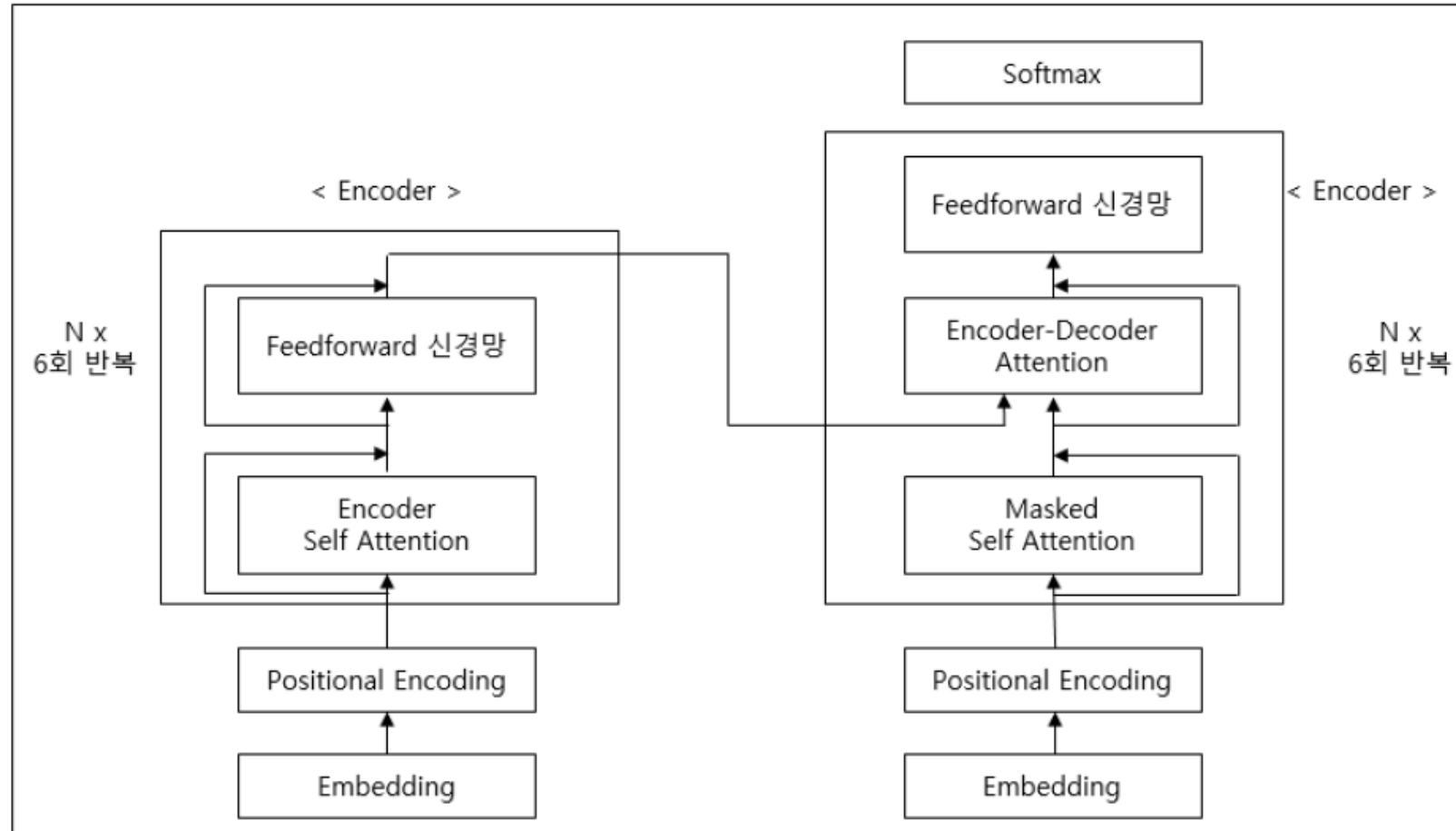
### ( 3 ) Attention

$$Attention(Q, K, V) = softmax_k(\frac{QK^T}{\sqrt{d_k}})V$$

| 구분        | 구성요소                  | 설명                                                                       |
|-----------|-----------------------|--------------------------------------------------------------------------|
| Encoder   | fully connected layer | - 이전 계층에서의 값에서 2 차원 벡터의 행렬을 1 차원 배열로 평탄화 하고, state 값의 score 계산           |
|           | Softmax function      | - 입력받은 값을 출력으로 0~1 사이의 값으로 모두 정규화하며 출력 값들의 총합은 항상 1 이 되는 특성을 가진 함수       |
|           | Attention Weigh       | - fully connected layer 에서 나온 가중치의 값을 Softmax 활성화함수에서 나온값, 즉 노드의 가중치를 칭함 |
| 데이터 학습 측면 | Query(Q)              | - t 시점의 디코더 셀에서의 은닉 상태                                                   |
|           | Keys(K)               | - 모든 시점의 인코더 셀의 은닉 상태들                                                   |
|           | Values(V)             | - 모든 시점의 인코더 셀의 은닉 상태들                                                   |

- 전체 입력 문장을 전부 다 동일한 비율로 참고하는 것이 아니라, 해당 시점에서 예측해야 할 단어와 연관이 있는 입력 단어 부분을 좀 더 집중(attention)해서 가중치를 부여함

### ( 3 ) Transformer



- RNN(Recurrent Neural Network)을 탈피하고 seq2seq 의 인코더-디코더 모델과 셀프 어텐션 (Self Attention) 구조만으로 구현한 자연어 처리 모델

### ( 3 ) Transformer

| 구분      | 구성요소                                   | 설명                                                                                                                                           |
|---------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Encoder | 포지셔널 인코딩<br>(Positional Encoding)      | <ul style="list-style-type: none"> <li>- 입력단어의 위치 값 추가</li> <li>- 사인, 코사인 함수 이용</li> <li>- RNN 미 적용으로 인한 단어 위치 문제해결</li> </ul>               |
|         | 인코더 셀프 어텐션<br>(Encoder Self-Attention) | <ul style="list-style-type: none"> <li>- 멀티 헤드 셀프 어텐션 : 입력 토큰 병렬처리</li> <li>- Query=Key=Value</li> <li>- 6 개의 인코더가 이전 인코더의 어텐션 참조</li> </ul> |
|         | 피드 포워드 신경망<br>(Feed Forward NN)        | <ul style="list-style-type: none"> <li>- Position-Wise 완전 연결망</li> <li>- 잔차(Residual) 연결이용 및 정규화 수행</li> </ul>                               |
| Decoder | 마스크드 셀프 어텐션 (Masked Self-Attention)    | <ul style="list-style-type: none"> <li>- 멀티 헤드 셀프 어텐션 : 입력 토큰 병렬처리</li> <li>- Query=Key=Value - 현재 이후 단어 마스킹 처리</li> </ul>                   |
|         | 인코더-디코더 어텐션                            | <ul style="list-style-type: none"> <li>- 인코더 어텐션과 디코더 어텐션 결합 사용</li> <li>- 인코더 셀프 어텐션=Key=Value</li> <li>- 디코더 셀프 어텐션=Query</li> </ul>       |
|         | Softmax                                | - 디코더 출력을 벡터화하여 Softmax 통해 결과값 출력                                                                                                            |

### ( 3 ) BERT, GPT-3

| 구 분  | BERT                                                                                                                                                                                                                                     | GPT-3                                                                                                                                                                                                                                |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 개념도  | <p>BERT (Ours) diagram showing a bidirectional flow between input tokens (<math>E_1, E_2, \dots, E_N</math>) and output tokens (<math>T_1, T_2, \dots, T_N</math>) through multiple layers of Transformer blocks (<math>Trm</math>).</p> | <p>OpenAI GPT diagram showing a unidirectional flow from input tokens (<math>E_1, E_2, \dots, E_N</math>) to output tokens (<math>T_1, T_2, \dots, T_N</math>) through multiple layers of Transformer blocks (<math>Trm</math>).</p> |
| 개발주체 | Google                                                                                                                                                                                                                                   | OpenAI                                                                                                                                                                                                                               |
| 모델구조 | 트랜스포머의 Encoder 만 사용                                                                                                                                                                                                                      | 트랜스포머의 Decoder 만 사용                                                                                                                                                                                                                  |
| 주요특징 | fine tuning                                                                                                                                                                                                                              | few shot Learning                                                                                                                                                                                                                    |
| 핵심기능 | 문장의 의미 추출                                                                                                                                                                                                                                | 문장 생성                                                                                                                                                                                                                                |
| 사례   | <p>어제 카페 갔었어 <span style="border: 1px solid black; padding: 0 5px;"> </span> 사람 많더라</p> <p>←→</p>                                                                                                                                        | <p>어제 카페 갔었어 거기 사람 많더라</p> <p>→</p>                                                                                                                                                                                                  |

- BERT 와 BPT-3 가 성능을 가지고 경쟁하는 것처럼 보이지만 실제 Transformer 로 부터 Encoder 및 Decoder 부분만 사용하는 언어모델 임.

감사합니다!



# 인공지능(AI)



## 5. 생성 AI

1. LLM(Large Langue Model)
2. 파운데이션모델(Foundation Model)
3. ChatGPT
4. GAN(Generative Adversarial Networks)



## (1) LLM(Large Langue Model)

- 대규모 데이터 세트에서 얻은 지식을 기반으로 텍스트와 다양한 콘텐츠를 인식하고 요약, 번역, 예측, 생성할 수 있는 딥 러닝 알고리즘
- 대규모 언어 모델은 트랜스포머 모델의 가장 성공적인 애플리케이션 중 하나

| 기술요소                 | 설명                                                                                           |
|----------------------|----------------------------------------------------------------------------------------------|
| Pre training         | - 많은 데이터를 학습하여, 다양한 모델을 만들수 있는 기본 학습                                                         |
| fine tuning          | - 기존에 학습되어져 있는 모델을 기반으로 아키텍처를 새로운 목적(나의 이미지 데이터에 맞게)변형하고 이미 학습된 모델 Weights로 부터 학습을 업데이트하는 방법 |
| Word Embedding       | - 단어의 의미를 숫자 형식으로 표현하는 데 사용되는 알고리즘                                                           |
| Attention Mechanisms | - 입력 텍스트의 특정 부분에 집중할 수 있도록 가중치를 설정하는 기술                                                      |
| Transformers         | - 문장 속 단어와 같은 순차 데이터 내의 관계를 추적해 맥락과 의미를 학습하는 신경망                                             |

## (2) 파운데이션(Foundation) 모델

- 레이블이 지정되지 않은 광범위한 데이터 집합
- 대규모 언어 모델을 사용자가 원하는 목적에 맞게 학습 시키기 위해, 방대한 양의 데이터를 사용하여 사전에 학습되어 있는 일종의 반 제품 형태의 AI 기초 모델
- 비지도 학습 기반의 초거대 언어모델, 초거대학습, AI 민주화

| 구분    | 핵심 기술       | 설명                                                       |
|-------|-------------|----------------------------------------------------------|
| 구현기술  | - 파인튜닝      | - 사전 학습한 모든 가중치와 더불어 하위 문제를 위한 최소한의 가중치를 추가해서 모델을 추가로 학습 |
|       | - 자기지도학습    | - 최소한의 데이터만으로 스스로 규칙을 찾아 분석하는 AI                         |
|       | - YOLO      | - 객체인식을 위한 AI을 구현하기 위한 AI 파운데이션 모델                       |
|       | - 포지셔널 인코딩  | - AI 파운데이션의 대표 모델인 트랜스포머의 주요기술로, 단어의 위치 정보를 알기 위한 기술.    |
| 구현 모델 | - Google    | - BERT, LAMDA, Switch Transformer                        |
|       | - Open AI   | - GPT-3, DALL-E2, Codex                                  |
|       | - Microsoft | - LLaMA(오픈소스)                                            |

### (3) ChatGPT

- OpenAI 에서 GPT 알고리즘을 통해 사전 학습된(**Pre-trained**) 모델에서 **PPO, RLHF, 강화 학습** 기반의 대화형 생성 AI모델 또는 챗봇

| 구분              | 기술요소                                             | 설명                                                                                                                                     |
|-----------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 데이터 수집 측면       | Pre-training                                     | <ul style="list-style-type: none"> <li>- 사전학습 된 모델</li> <li>- 사이즈가 큰 데이터로 이미 학습이 되어 있는 모델</li> </ul>                                   |
|                 | LLM(Large Language Model)                        | <ul style="list-style-type: none"> <li>- 대규모 데이터에서 얻은 지식을 기반으로 요약, 번역, 예측, 생성할 수 있는 딥 러닝 알고리즘</li> </ul>                               |
| 보상모델 생성 및 학습 측면 | Fine-tuning                                      | <ul style="list-style-type: none"> <li>- 큰 데이터셋으로 사전 학습된 모델을 가져와서 새로운 작업 또는 도메인에서 더 특화된 모델로 조정하는 과정</li> </ul>                         |
|                 | RLHF(Reinforcement Learning From Human Feedback) | <ul style="list-style-type: none"> <li>- 인간이 의도하는 정책을 학습시키기 위해 사람 (Labeler)이 선별한 적은 양의 샘플 데이터셋으로 학습하는 방법</li> </ul>                    |
|                 | Transformer                                      | <ul style="list-style-type: none"> <li>- Encoder, Decoder, Self Attention, Masked Self Attention 등으로 구성되어 병렬 처리하는 자연어 처리 모델</li> </ul> |
| 최적화 측면          | 강화학습                                             | <ul style="list-style-type: none"> <li>- 모델이 생성한 여러 답변 후보들 중 랭킹을 매겨 점수화한 데이터셋을 수집하여 Reward Model 학습</li> </ul>                         |
|                 | PPO (Proximal Policy Optimization)               | <ul style="list-style-type: none"> <li>- SFT 모델에 여러 사용자들의 입력을 주고, Reward Model 과 함께 상호 작용하면서 강화학습</li> </ul>                           |

# ChatGPT

- 대화형 모델

# ChatGPT

- Generative
- 생성 AI

# ChatGPT

- **Pre-trained** 사전 학습한
- **LLM : 거대언어모델**
- **5조 개 문서, 3천억 단어**

# ChatGPT

- **Transformer** 딥러닝 모델
- 다음 단어를 확률로 예측

### (3) ChatGPT

---

#### 할루시네이션(hallucination)

- **환각**이라는 의미, 잘못된 정보를 사실처럼 말하는 문제
- ChatGPT가 학습한 데이터 중 **한국어 데이터는 극소량**
- ChatGPT의 대답은 정답이 아니라 **'정답일 확률이 가장 높은 답변'** 선택
- 모르면 모른다고 하면 되는데, 왜 이렇게 그럴싸한 답변을 내놓는 걸까요?
- **Transformer** 모델 때문!



## (3) ChatGPT

+ New chat

Today

추천 도서 목록

Previous 7 Days

도서관 변화 예측

현재 시간 확인.

New chat

미래 도서관 변화

New chat

안녕, ChatGPT!

 Junsam Kim

...

 GPT-3.5

 GPT-4

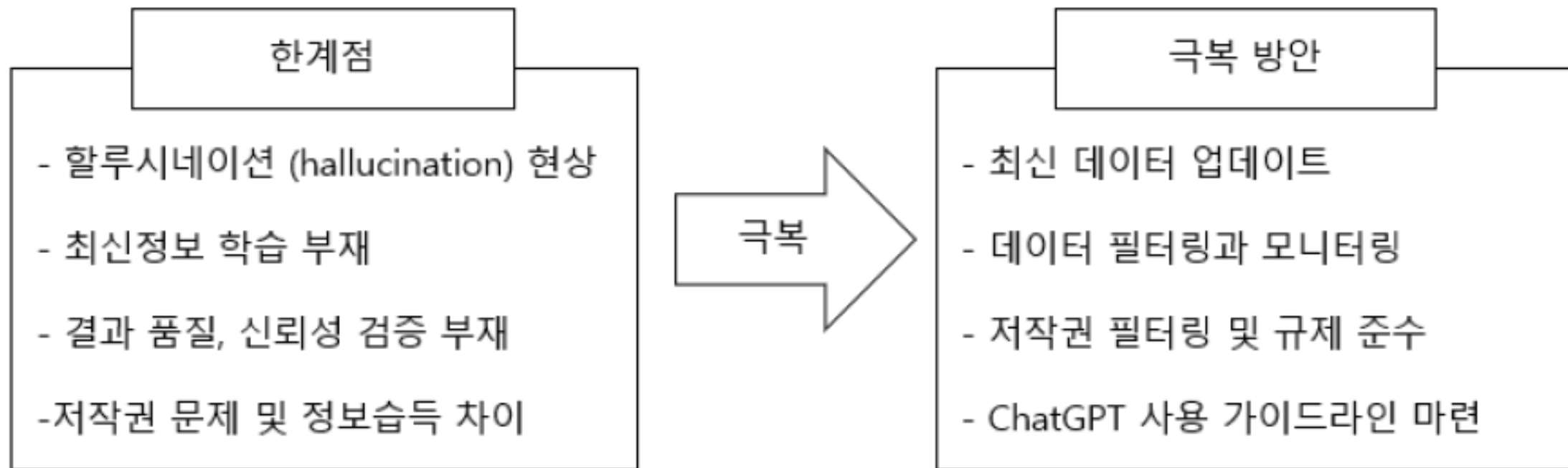
ChatGPT **PLUS**

Send a message.



ChatGPT may produce inaccurate information about people, places, or facts. [ChatGPT May 24 Version](#)

### (3) ChatGPT



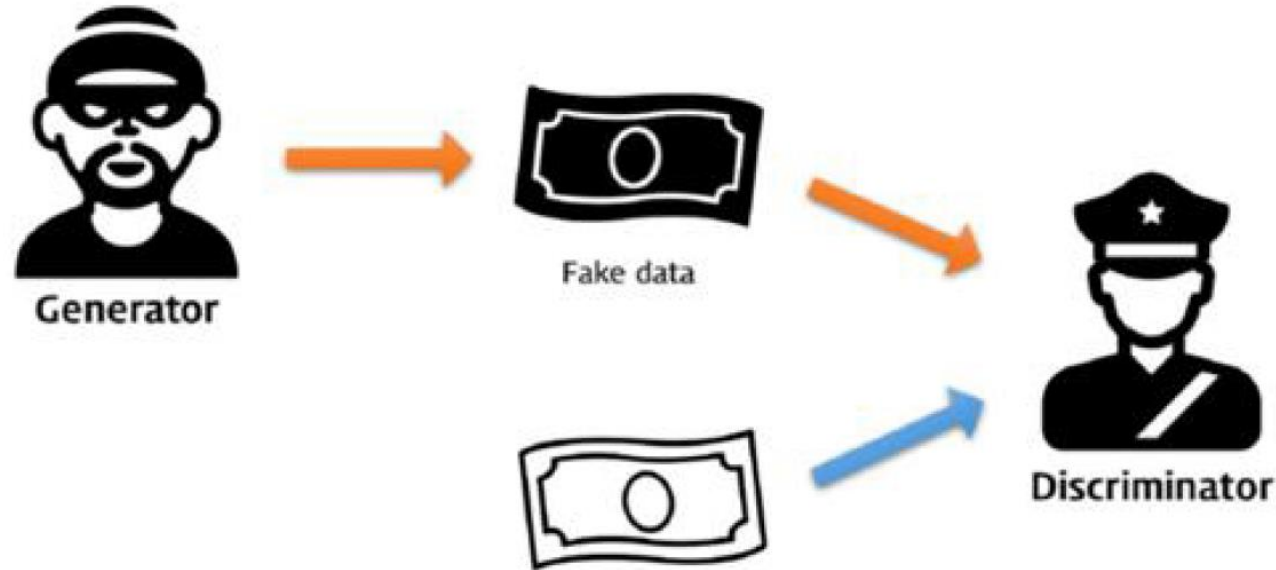
- ChatGPT의 대답은 정답이 아니라 '정답일 확률이 가장 높을 답변' 선택하는 알고리즘으로, **할루시네이션 (hallucination) 현상** 등과 같은 한계점 존재
- Transformer 및 RLHF를 통해 ChatGPT의 성능이 극대화되었으나, 알고리즘 자체 문제점으로 인해 기술 및 사회적 측면으로 극복해야 할 과제 존재
- ChatGPT와 같은 초거대 언어모델이 갖는 한계점을 타파하고 발전하기 위한 지속적인 R&D 및 투자가 필요

#### (4) GAN(Generative Adversarial Networks)

---

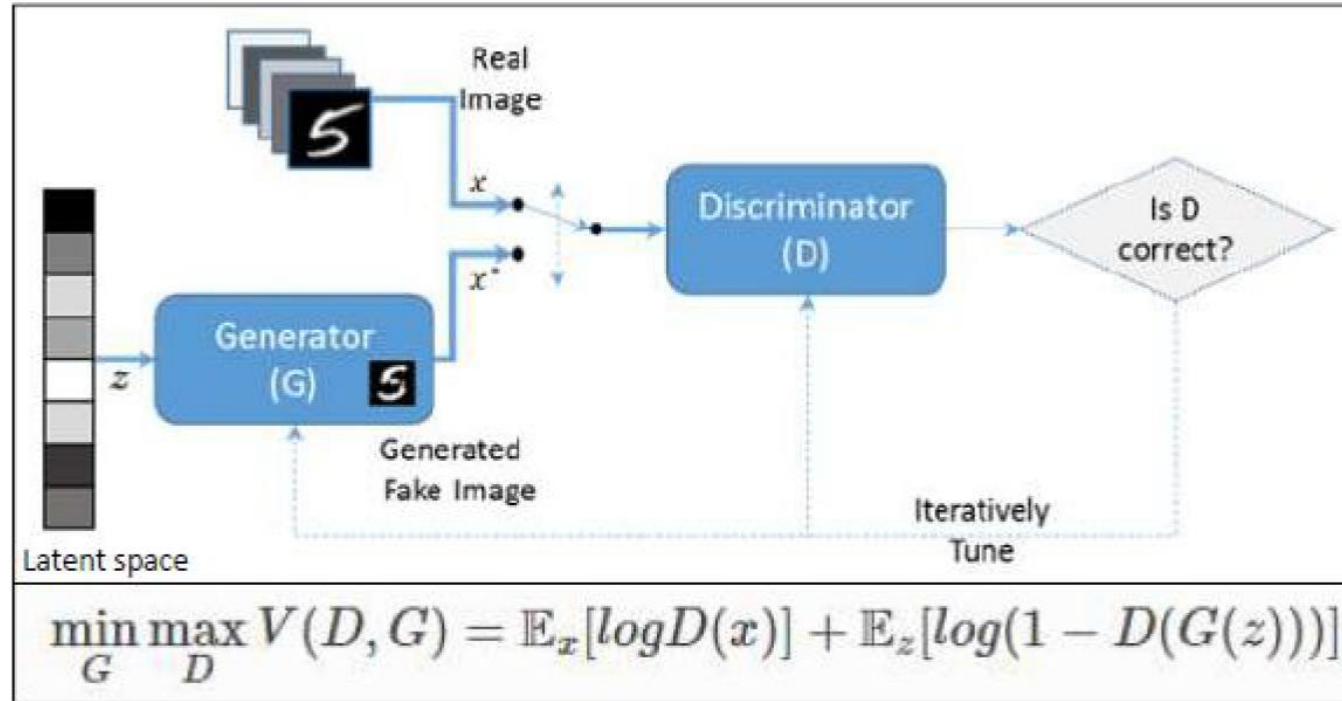
- Generative 은 "생성적인, 생산하는"이라는 뜻으로 GAN 모델 안에서의 의미는 이미지를 생성한다라는 의미
- Adversarial 은 "적대적인" 이르는 뜻으로, 서로 경쟁하면서 무엇인가를 좋게 하는 의미. 즉, 이미지를 만들긴 하는데, 서로 경쟁하면서 좋게 만든다 라는 의미
- 스스로 이미지를 만드는 생성자 (Generator)와 이미지를 감별하는 감별자 (Discriminator)간 대립 과정을 통해 훈련 타깃을 생성해 내는 인공지능 기술

#### (4) GAN(Generative Adversarial Networks)



- GAN 을 설명할 때 흔히 사용되는 비유법은 지폐위조범과 경찰의 예시임
- 지폐 위조범에게 Generator 라는 역할을 부여하고 경찰은 Discriminator 라는 역할을 부여.
- 위조범은 경찰의 단속을 피하기 위해서 더욱 정교한 가짜 지폐를 만들어내고, 경찰은 점점 더 정교한 기법으로 지폐를 판별해내는 방법을 개발함
- 이처럼 각각의 역할을 가진 두 개의 모델을 통해 진짜같은 가짜를 생성해내는 능력을 키워주는 것이 GAN 의 핵심 아이디어이고, 그래서 적대적이라는 용어가 붙은 이유임

#### (4) GAN(Generative Adversarial Networks)



- ① 훈련 데이터(진짜 데이터)를 가지고 Discriminator(판별기)를 학습. 이를 통해, 훈련데이터의 분포를 얻게 됨
- ② Latent sample(잠재된 공간에서의 샘플 데이터)를 통해 노이즈한 값을 Generator(생성기)에 주고, 생성기에서는 이미지를 생성. 물론, 처음에는 굉장히 허접한 데이터를 생성하지만 분포가 점점 훈련데이터 분포에 따라갈수록 진짜 이미지와 비슷한 이미지를 생성
- ③ 생성된 이미지(가짜 데이터)는 Discriminator(판별기)에서 가짜 데이터와 진짜 데이터를 구별하게 되고, 오차 역전파를 통해 Generator(생성기)를 좀 더 진짜 데이터로 만들 수 있는 방향으로 학습
- ④ 이와 같은 방법을 반복하게 되면, 생성기는 결국 진짜 데이터와 유사한 데이터가 생성 됨

#### (4) GAN(Generative Adversarial Networks)

| 구분   | 주요 기술                       | 설명                                                                    |
|------|-----------------------------|-----------------------------------------------------------------------|
| 신경망  | 감별자<br>(Discriminator)      | - 입력으로 주어진 데이터가 학습 데이터에 포함된 진짜인지, 생성자가 만들어낸 가짜인지 판별 역할                |
|      | 생성자<br>(Generator)          | - 랜덤 노이즈를 입력받아 학습 데이터와 유사한 패턴으로 변환하여 데이터를 생성 기술                       |
|      | 노이즈<br>(Latent variable)    | - 생성자가 가짜 데이터를 만들기 위해 입력하는 랜덤 데이터, 랜덤 노이즈                             |
| 적용이론 | 내쉬 균형<br>(Nash Equilibrium) | - 생성자와 판별자가 합리적 판단을 하는 수학적 균형점을 찾는다는 이론<br>- 2 개의 신경망이 서로 충돌시 발생한 차이점 |
|      | Mini-max problem            | - 예상되는 최대 손실을 최소화 시키기 위해 사용하는 의사결정이론의 한 방법                            |

감사합니다!



# 인공지능(AI)



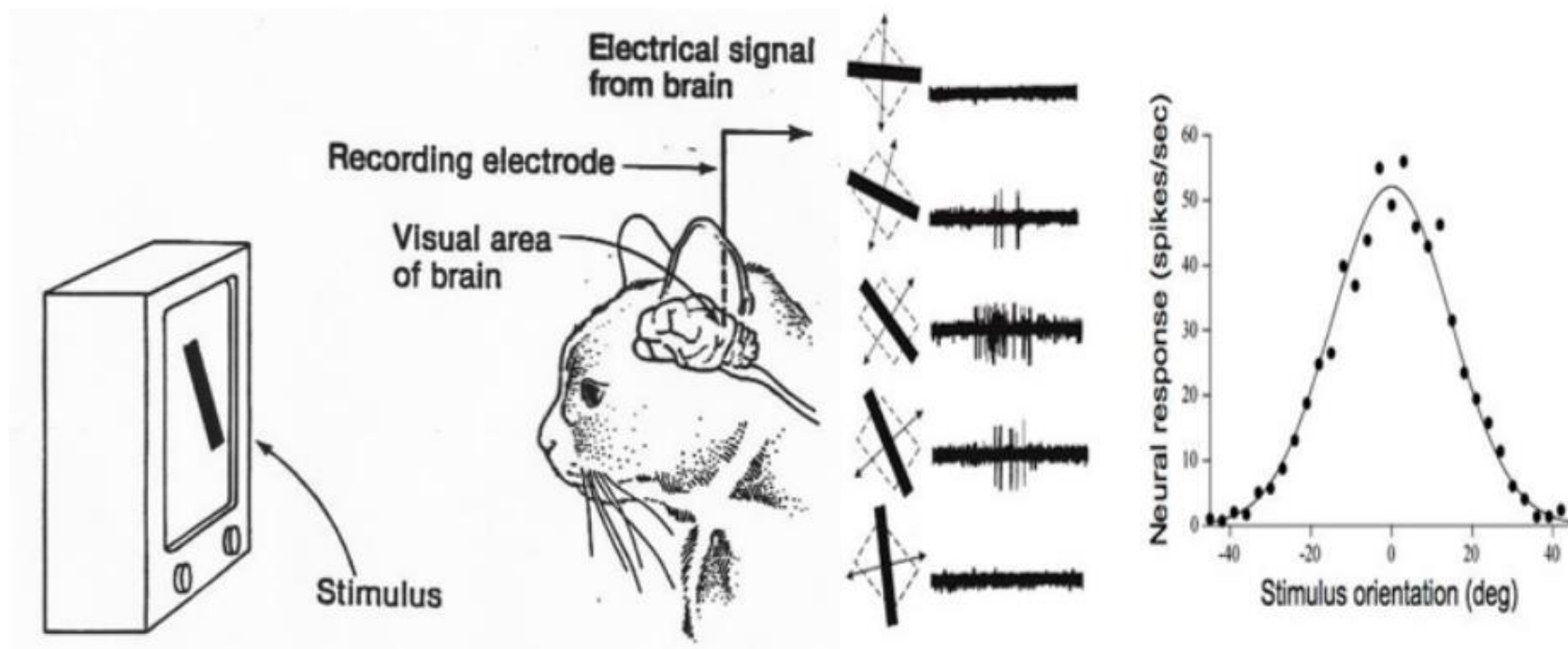
## 6. 최신 AI



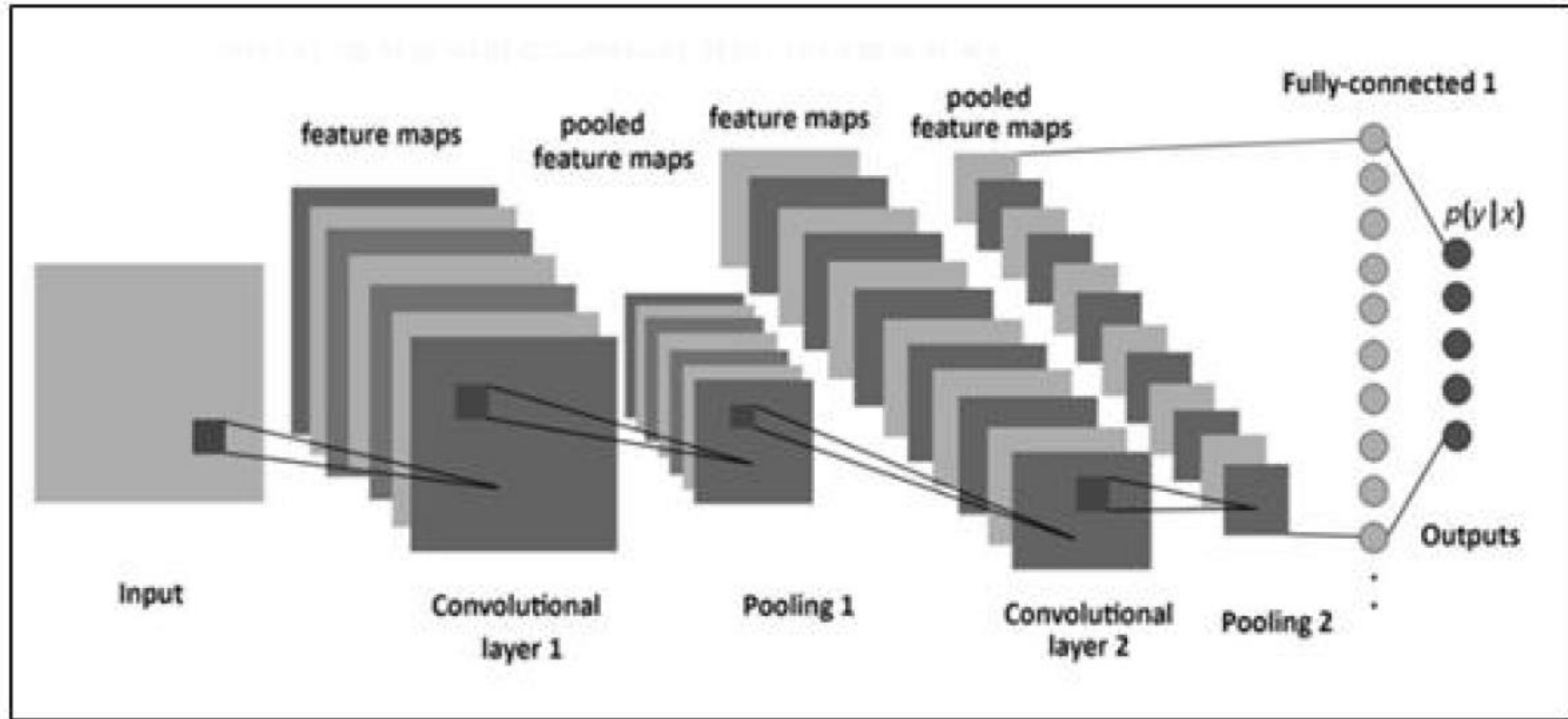
1. CNN(Convolutional Neural Network)
2. 전이학습(Transfer Learning)
3. 연합학습(Federated Learning)
4. 딥페이크(Deepfake)
5. XAI(Explainable AI)

## (1) CNN(Convolutional Neural Network)

- 생물의 시신경이 동작하는 원리를 이용하여 Convolution, Pooling 반복하여 이미지 데이터 처리에 적합한 구조로 만들어진 딥러닝 알고리즘
- CNN은 고양이가 특정 이미지를 보는 데에서 착안하여 고양이가 화면을 볼 때 위치에 따라 자극 받는 뇌의 위치가 다른 것을 보고 개발함. 즉, image 전체를 보는 것이 아니라 부분을 보는 것이 핵심



## (1) CNN(Convolutional Neural Network)

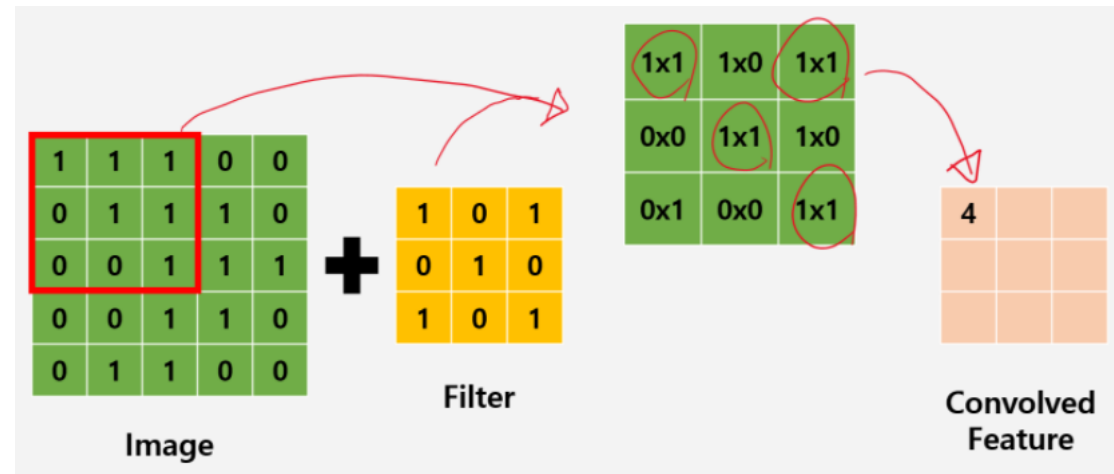
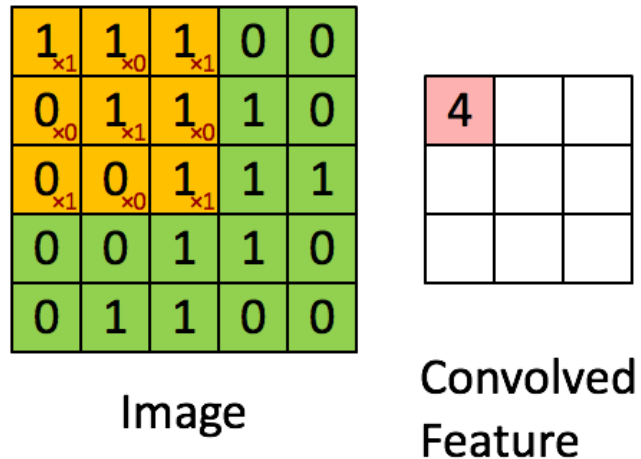


- CNN 은 Convolution 과 Pooling 을 통해 특징을 찾고, 그 특징을 Fully-connected 신경망에 보는 과정

## (1) CNN(Convolutional Neural Network)

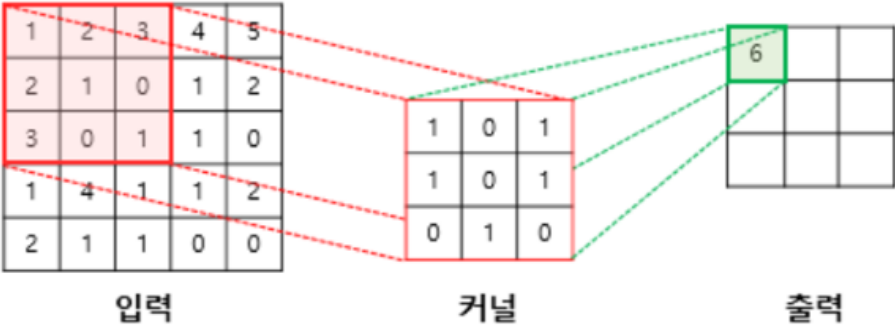
## ① convolutional layer (합성곱)

- 원본 데이터 : 5x5
- Filter : 3x3 (이미지를 한번에 보는 영역)
- stride : 1 (위치이동)



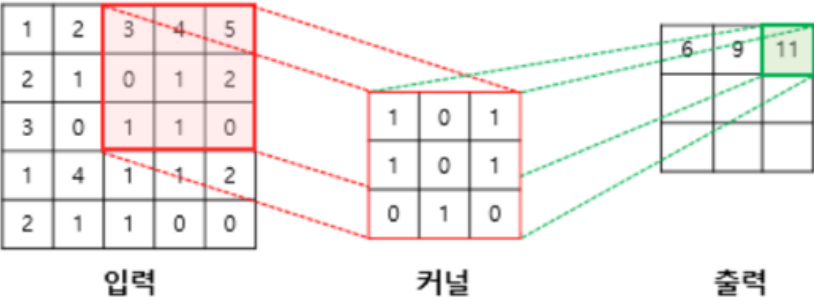
# (1) CNN(Convolutional Neural Network)

## 1. 첫번째 스텝

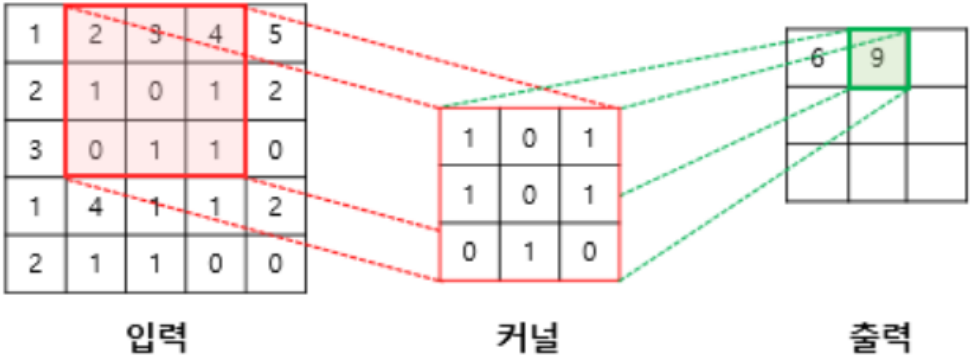


$$(1 \times 1) + (2 \times 0) + (3 \times 1) + (2 \times 1) + (1 \times 0) + (0 \times 1) + (3 \times 0) + (0 \times 1) + (1 \times 0) = 6$$

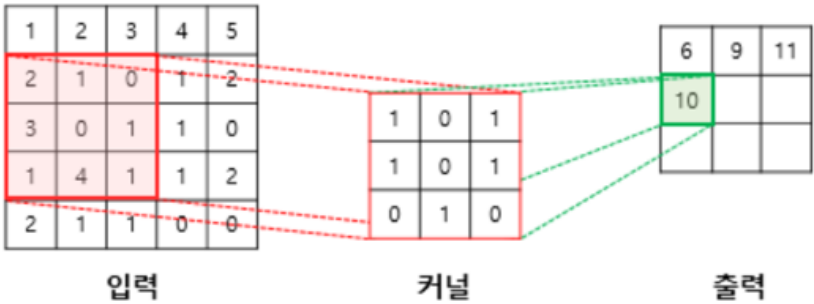
## 3. 세번째 스텝



## 2. 두번째 스텝



## 4. 네번째 스텝



$$(2 \times 1) + (1 \times 0) + (0 \times 1) + (3 \times 1) + (0 \times 0) + (1 \times 1) + (1 \times 0) + (4 \times 1) + (1 \times 0) = 10$$

## (1) CNN(Convolutional Neural Network)

- Padding : 데이터 손실을 줄이기 위한 방법
- zero-padding : 가장자리에 0의 값을 갖는 픽셀을 추가
- Convolution처리 + 활성화함수

$$\begin{array}{|c|c|c|c|} \hline 0 & 1 & 7 & 5 \\ \hline 5 & 5 & 6 & 6 \\ \hline 5 & 3 & 3 & 0 \\ \hline 1 & 1 & 1 & 2 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline 1 & 0 & 0 \\ \hline 1 & 2 & 1 \\ \hline 1 & 2 & 3 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 41 & 33 \\ \hline 25 & 23 \\ \hline \end{array}$$

패딩이 적용되지 않은 합성곱 연산

$$\begin{array}{|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 1 & 7 & 5 & 0 \\ \hline 0 & 5 & 5 & 6 & 6 & 0 \\ \hline 0 & 5 & 3 & 3 & 0 & 0 \\ \hline 0 & 1 & 1 & 1 & 2 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} \otimes \begin{array}{|c|c|c|} \hline 1 & 0 & 0 \\ \hline 1 & 2 & 1 \\ \hline 1 & 2 & 3 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 26 & 42 & 55 & 35 \\ \hline 34 & 41 & 33 & 28 \\ \hline 18 & 25 & 23 & 14 \\ \hline 3 & 9 & 8 & 8 \\ \hline \end{array}$$

zero-padding이 적용된 합성곱 연산 (우).

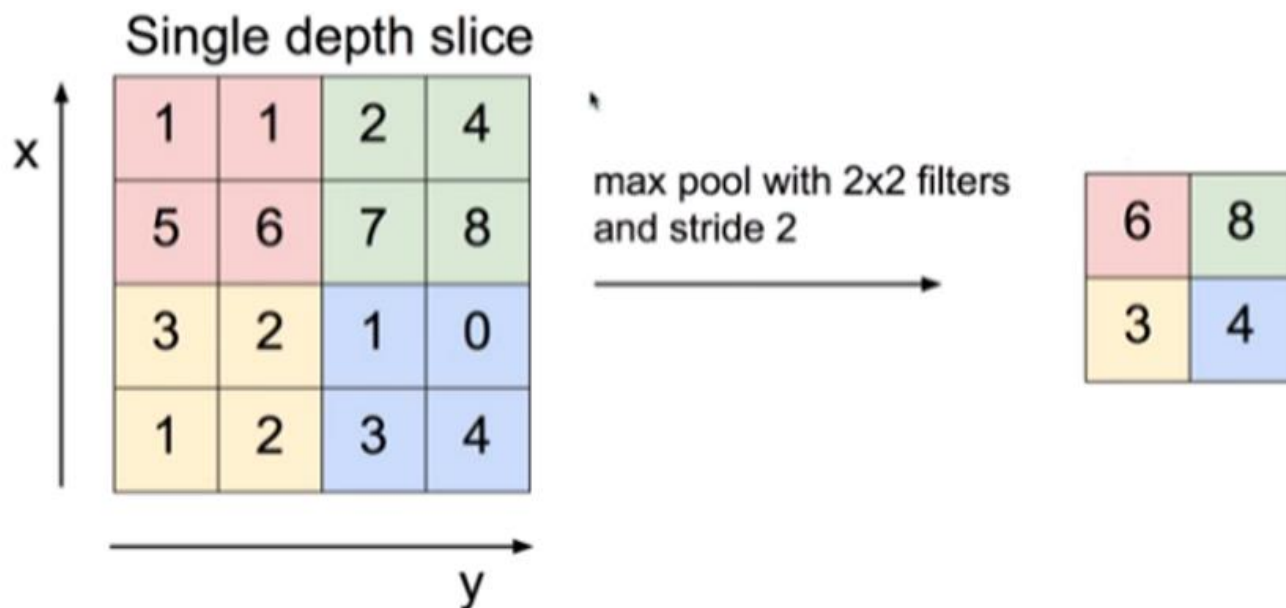
## (1) CNN(Convolutional Neural Network)

### ② Pooling Layer

- convolution layer을 resizing하여 새로운 layer를 얻는 방법
- pooling을 통해 최대한 파라미터를 줄여 효율적인 feature를 추출 가능
- Max Pooling : 레이어 중의 최대값 선택

### MAX POOLING

그림 : 4x4  
Filter : 2x2  
Stride : 2



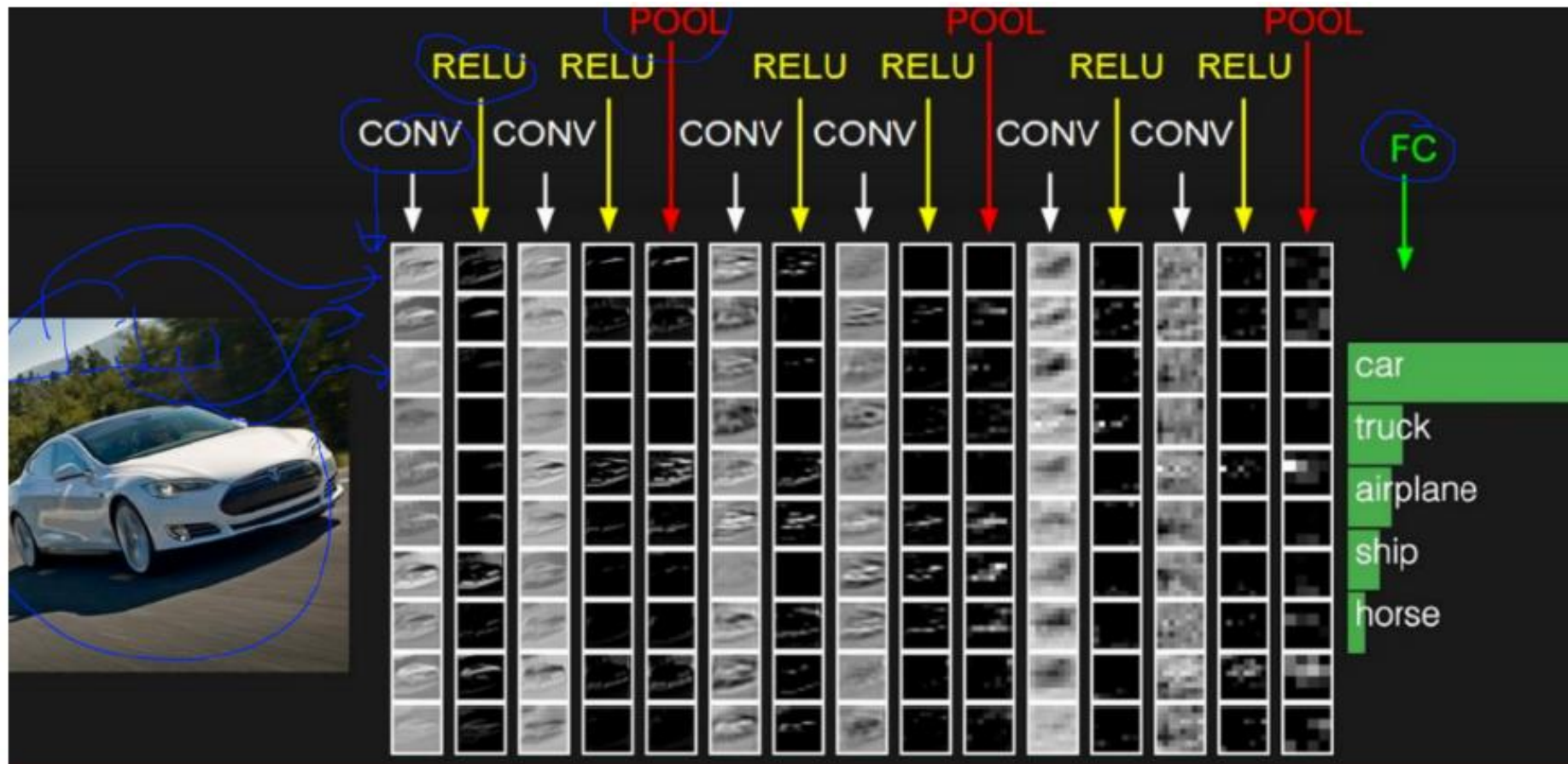
각 pixel마다 최댓값을 뽑아낸다. (max pooling)



## (1) CNN(Convolutional Neural Network)

### ③ Fully Connected Layer (FC)

- 이전 레이어의 모든 노드가 다음 레이어의 모든 노드에 연결하여 특징 점을 기반으로 이미지를 분류하는 레이어



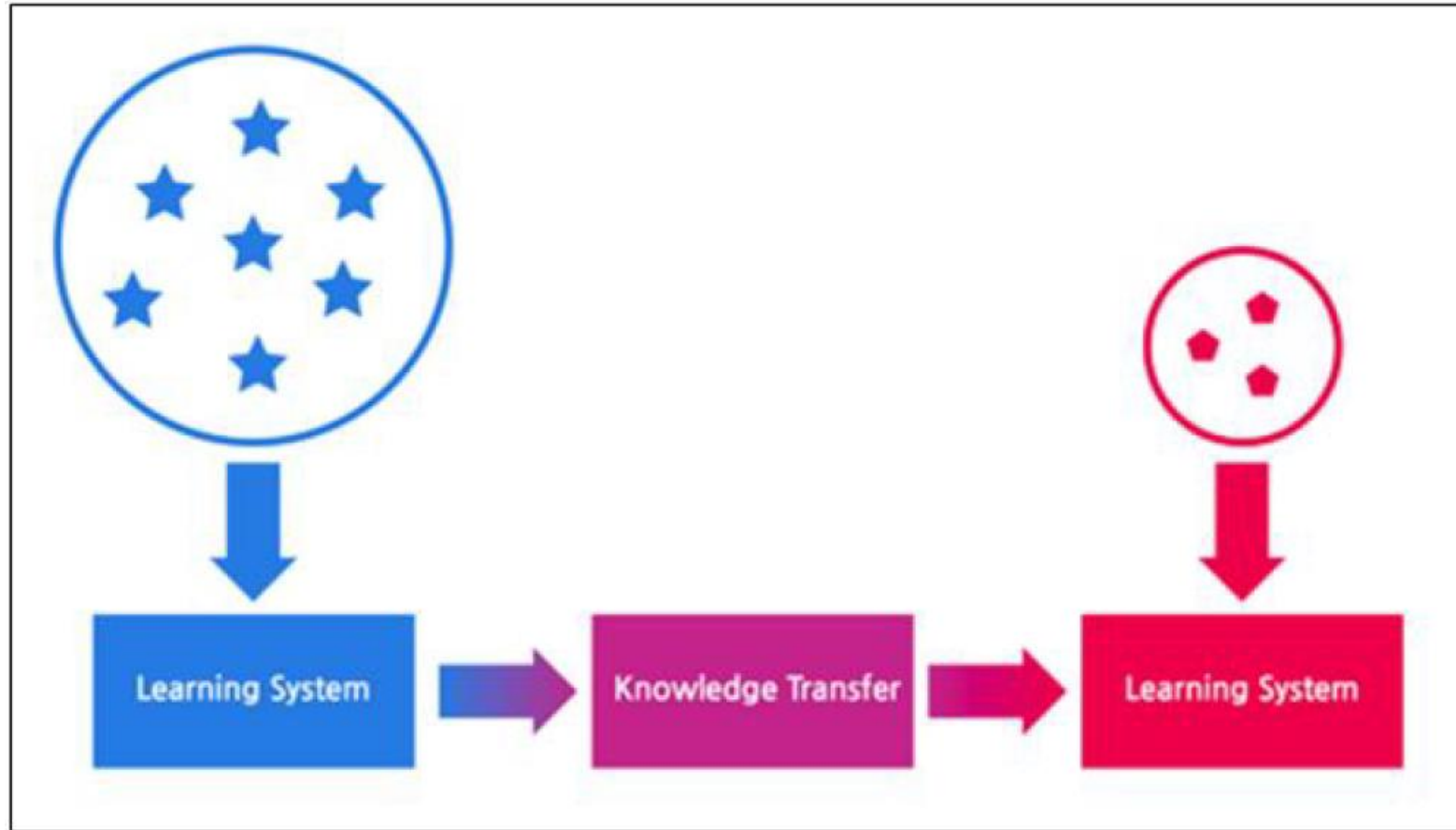


## (2) 전이학습(Transfer Learning)

---

- 학습 데이터가 부족한 분야의 모델 구축을 위해 데이터가 풍부한 분야에서 훈련된 모델을 재사용하는 알고리즘
- 특정 분야에서 학습된 신경망의 일부 능력을 유사하거나 전혀 새로운 분야에서 사용되는 신경망의 학습에 이용하는 것을 의미. 따라서 기존의 만들어진 모델을 사용하여 새로운 모델을 만들시 학습을 빠르게 하여, 예측을 더 높일 수 있음

## (2) 전이학습(Transfer Learning)



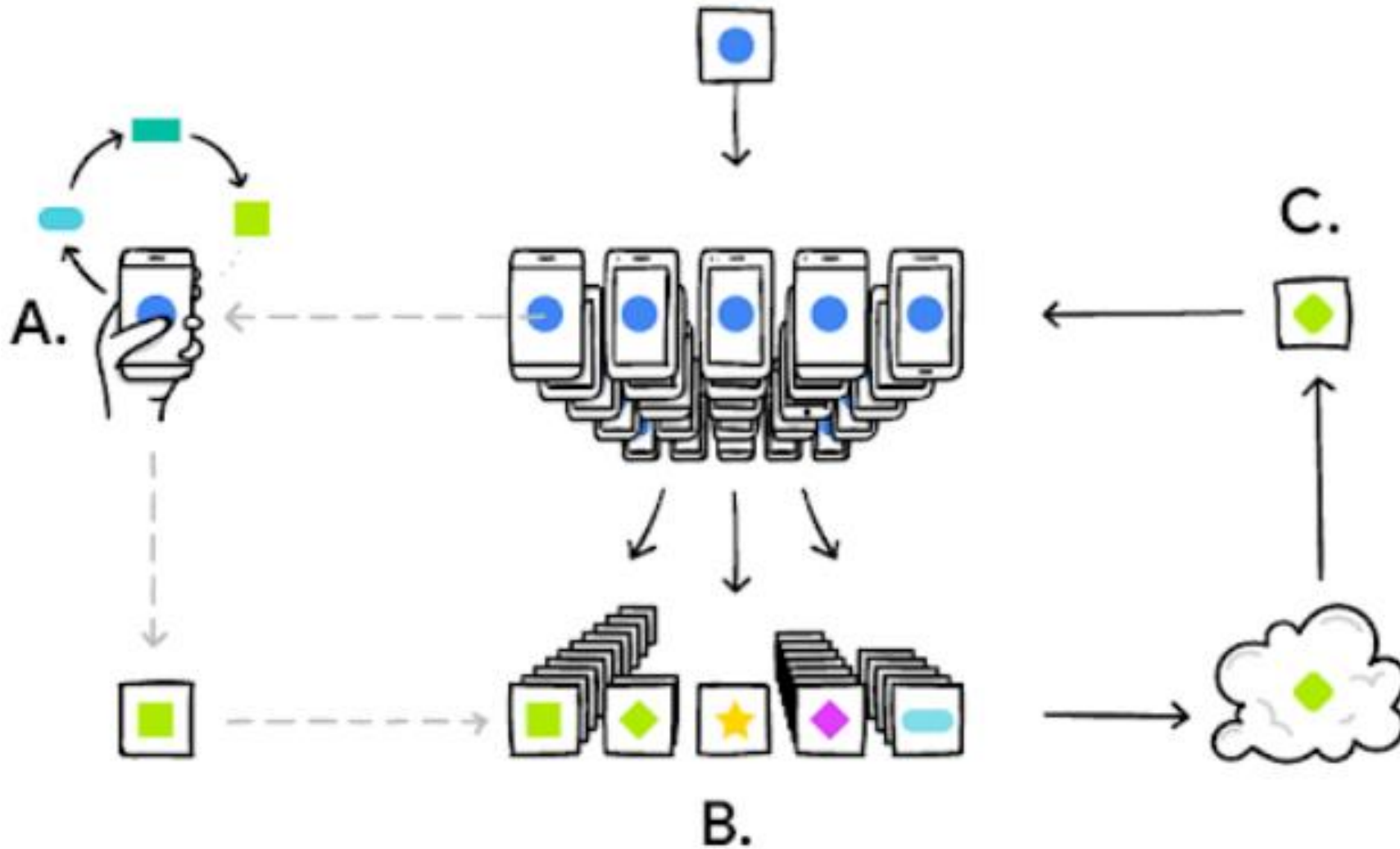
- 전이학습은 예를들어, 사과 깎는 방법을 익힌 AI에게 배를 깎도록 하거나 체스를 익힌 AI에게 장기를 두게하거나 비가 올 확률을 예측하는 AI에게 눈이 올 확률을 예측하는 기법

## (2) 전이학습(Transfer Learning)

| 지표                                    | 설명                                                                                                                                                  |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pre-trained Model</b><br>(사전학습 모델) | <ul style="list-style-type: none"><li>- 미리 학습된 모델의 가중치를 새로운 모델에 적용</li><li>- 사전학습 된 모델이란, 내가 풀고자 하는 문제와 비슷하면서 사이즈가 큰 데이터로 이미 학습이 되어 있는 모델</li></ul> |
| <b>Domain Adaptation</b>              | <ul style="list-style-type: none"><li>- 학습 시 도메인 부분은 약하게 학습하여 어느 도메인이나 적용이 가능하게 하는 방법</li></ul>                                                     |
| <b>Layer Re-use</b>                   | <ul style="list-style-type: none"><li>- 기존 모델의 일부 Layer 를 재사용하여 부족한 Data 의 Domain 모델 구축에 활용</li><li>- 다른 레이어를 고정시키고 일부분 layer 를 조정</li></ul>        |

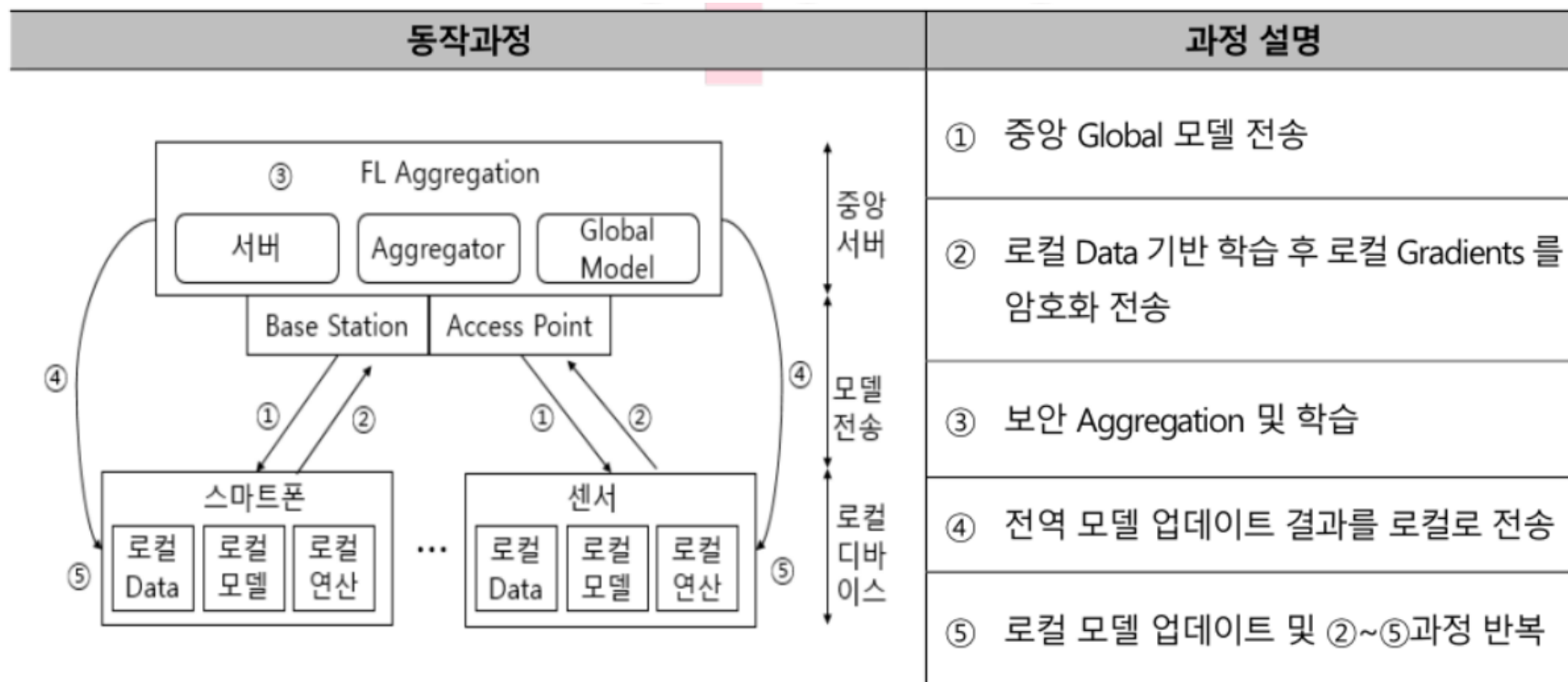
- 전이학습은 이미 잘 훈련된 모델이 있고, 특히 해당 모델과 유사한 문제를 해결시 유용함.

### (3) 연합학습(Federated Learning)



### (3) 연합학습(Federated Learning)

- 데이터를 직접 공유하지 않고, **FedSGD, FedAVG**를 통해 각각의 디바이스에서 학습한 **가중치**를 수집하여 재배포하는 분산형 머신러닝 기법
- 최신의 AI Model을 다운 → 각 로컬 디바이스에서 학습 → 가중치 서버로 전달 → 모델 다시 배포



전역 모델 기준 로컬 학습 결과로 중앙 모델을 업데이트하고 결과를 다시 로컬로 전송하는 과정을 반복

## (4) 메타러닝(Meta Learning)

### Learning To Learn, 메타러닝 개요

| 개념                                                                                 | 도입배경                        |
|------------------------------------------------------------------------------------|-----------------------------|
| 인공지능 개발시 사람이 신경 쓸 부분을 최소화하고,<br><u>적은 데이터만으로 빠른 학습이 진행되도록</u><br>"학습하는 법을 학습"하는 기법 | - 학습 시 데이터 최소화              |
|                                                                                    | - 학습 시 사람 개입 최소화            |
|                                                                                    | - Few shot learning 기반 알고리즘 |

- Meta Learning 의 목적은 학습하는 방법을 학습, 즉 새로운 Task 에 대한 학습의 성능을 높이는 것 임
- few shot learning 은 소량의 데이터(few-shot)만으로도 좋은 학습 결과가 나오는 학습 방법으로, 메타 러닝은 few show learning 을 위한 기반 알고리즘 임

## (4) 메타러닝(Meta Learning)

---

### ① Metric based learning

- 두 이미지의 특징을 한 공간에 나타내고 거리를 통해 similarity 를 계산
- 삼 네트워크(Siamese Neural Network)

### ② Model based learning

- 메모리를 이용하여 거리 측정하는 순환 신경망
- MANN(Memory-Augmented Neural Network)

### ③ Optimization based learning

- 모델 파라미터 최적화
- MAML(Model-Agnostic Meta-Learning)

## (5) 딥페이크(Deepfake)

### □ 딥페이크 개념

- 딥러닝(Deep Learning)을 이용해 원본 이미지나 동영상 위에 다른 영상을 중첩하거나 결합하여 원본과는 다른 가공 콘텐츠를 생성하는 합성기술

### □ 딥페이크의 이점, 사용하는 이유

- 딥 페이크 기술을 사용하여 개발된 목소리와 유사성은 영화 속에서 창조적인 효과를 얻거나 사람이 못하거나 하기 힘들 일을 대체하고 도와주는 것에도 사용됨. |예를 들어, 최근의 스타워즈 영화에서, 이 기술은 죽은 주인공들을 대체하거나 젊은 시절에 등장했던 캐릭터들을 보여주기 위해 사용. 또한 소매업체들은 또한 고객들이 옷을 가상으로 입어볼 수 있도록 하기 위해 이 기술을 사용했습니다.

### □ 딥페이크 문제점, 위험성

- 영화나 상업과 같은 온순하고 합법적인 응용 프로그램에도 불구하고, 딥페이크는 안타깝게도 성착취물에 많이 사용됨. 일부 연구는 온라인의 많은 가짜 콘텐츠가 포르노이고, 가짜 포르노로 인해 피해를 입는 사람이 생기고 있음.



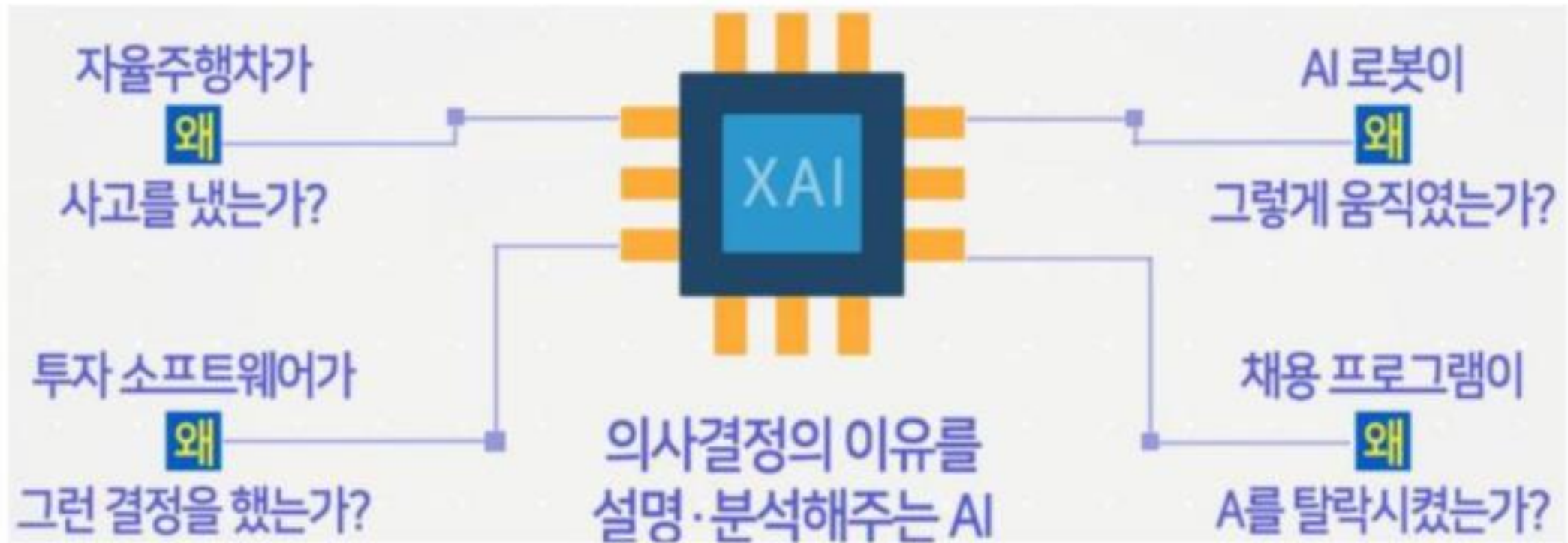
## (5) 딥페이크(Deepfake)

| 구분   | 주요 기술                                    | 설명                                                                       |
|------|------------------------------------------|--------------------------------------------------------------------------|
| GAN  | 내쉬균형<br>Mini-max problem                 | - 동영상의 모든 패털을 모델링하거나, 이미지에서 물체의 3D 모델을 재구성 처리                            |
| LSTM | Input Gate<br>Forget Gate<br>Output gate | - 현재 프레임과 이전 프레임의 차를 표현하는 residual 프레임을 CNN 과 LSTM 기반의 RNN 을 이용하여 벡터화 처리 |

- 딥페이크 해결하기 위해 성폭력처벌법 제 14 조의 2(허위 영상물 등의 배포 등) 에 '특정 인물의 신체 등을 대상으로 한 영상물 등을 성적 욕망 또는 수치심을 유발할 수 있는 형태로 편집하는 행위 등' 관련 조항 추가

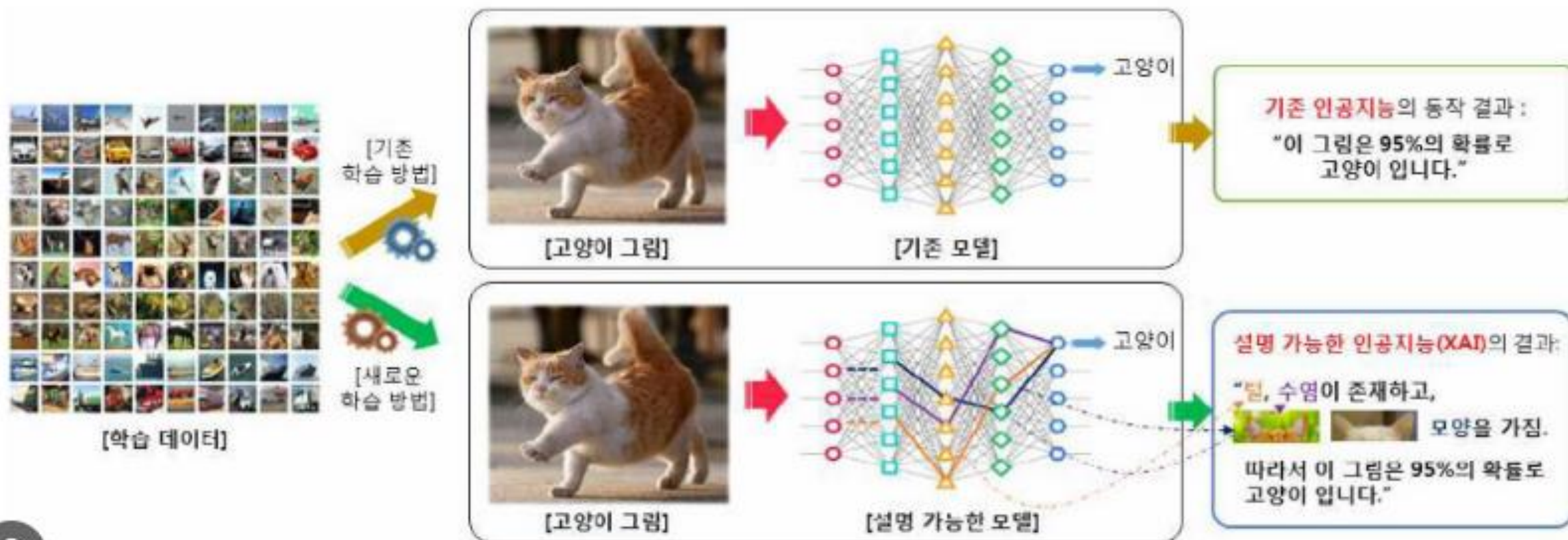
## (6) XAI(eXplainable Artificial Intelligence)

- 인공지능 동작과정을 이해하고, 결과물의 생성되는 과정을 설명 가능하게 해주는 기술
- 즉 해당 SW가 왜 그렇게 동작을 했는지 알려주는 기술



## (6) XAI(eXplainable Artificial Intelligence)

- 인공지능 동작과정을 이해하고, 결과물의 생성되는 과정을 설명 가능하게 해주는 기술
- 즉 해당 SW가 왜 그렇게 동작을 했는지 알려주는 기술



감사합니다!



# 인공지능(AI)

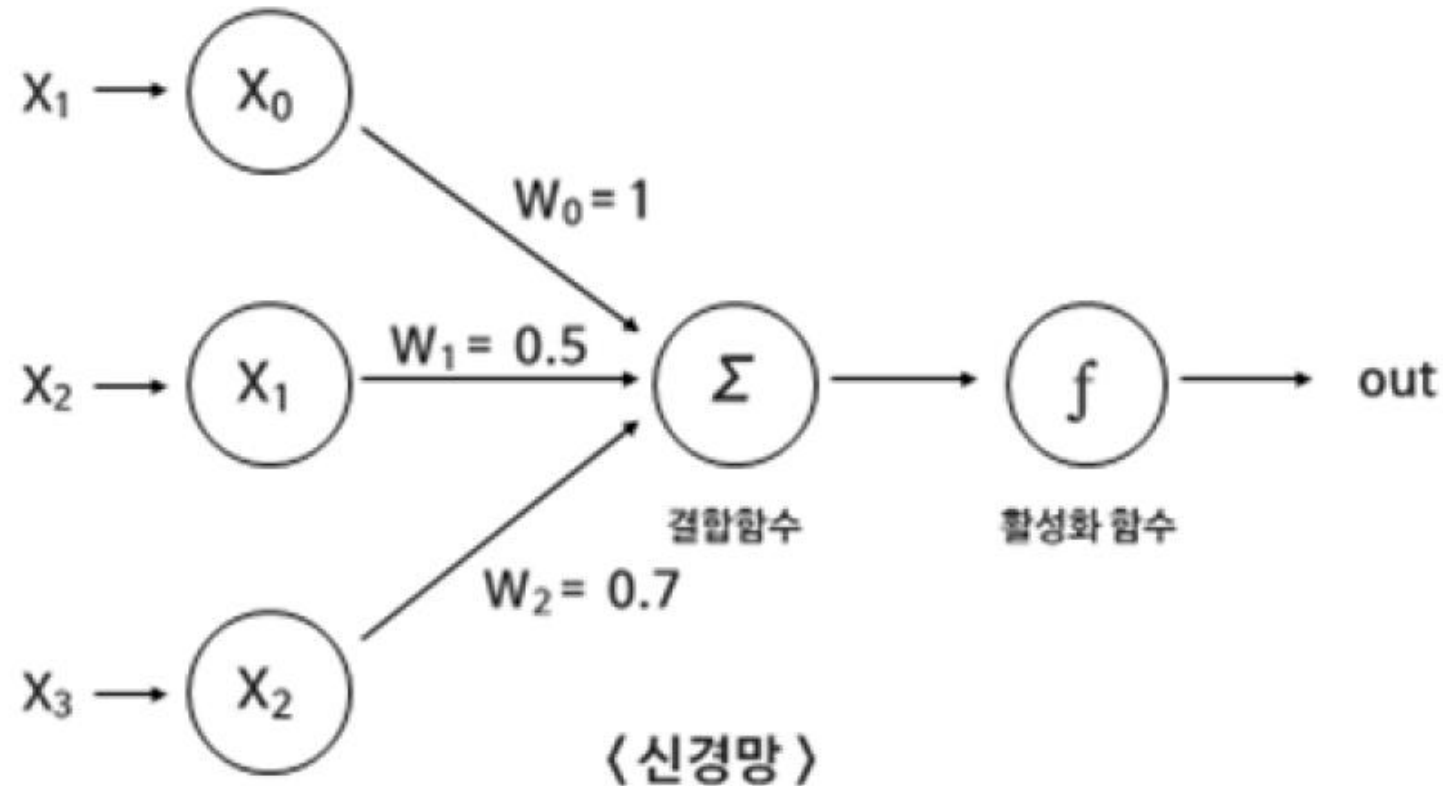


## 7. 인공지능경망

1. 신경망역사(Artificial Neural Network)
2. 퍼셉트론(Perceptron)
3. 활성화함수(Activation Function)
4. 오차 역전파(Back-propagation)
5. 비용함수(Cost Function)
6. 경사하강법(Gradient Descent)

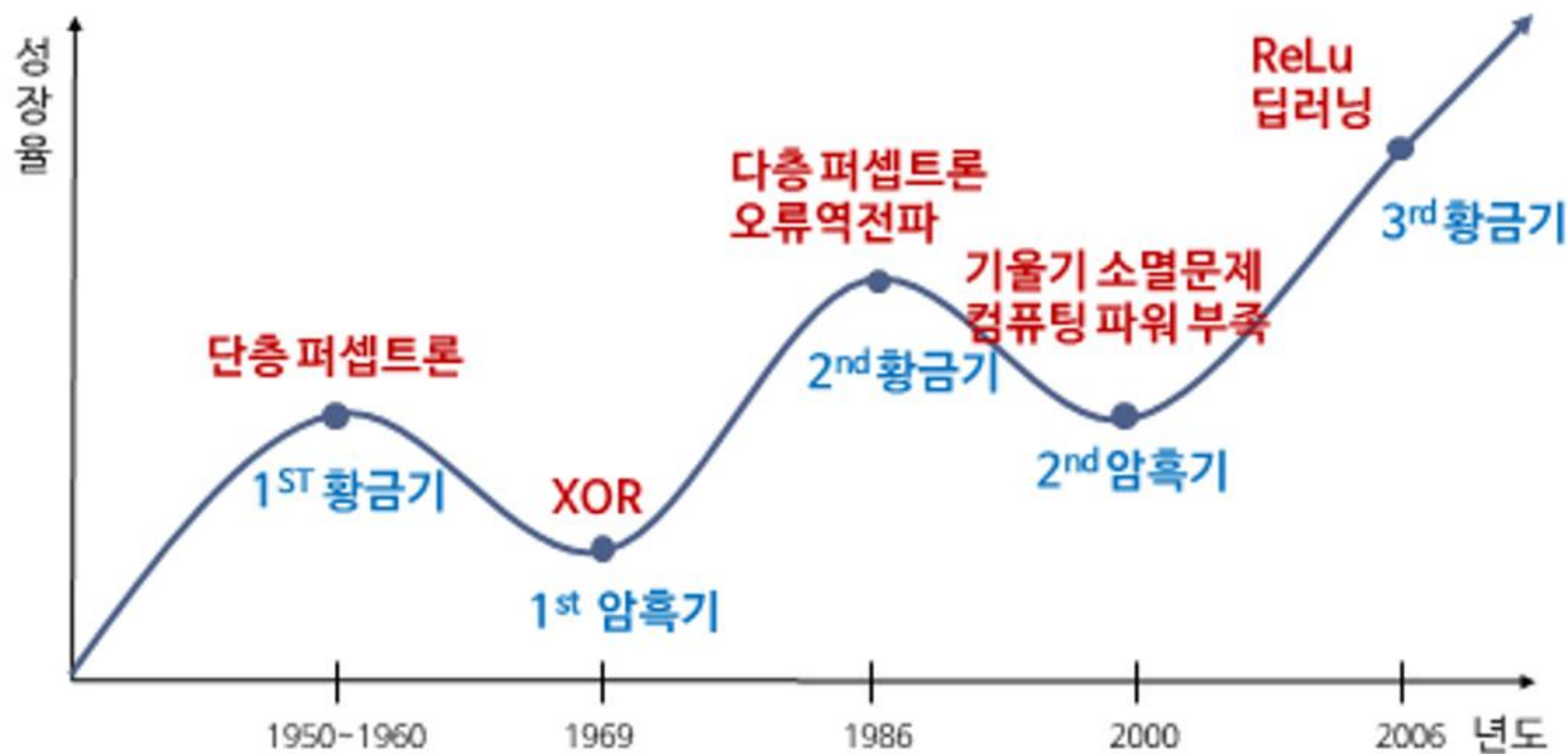
## (1) 신경망역사(Artificial Neural Network)

- 인간의 뇌를 모방해서 그 기반으로 결과를 예측하는 기법
- 입력값과 가중치를 설정하고, 활성화 함수를 통해 결과값을 출력하는 전체 과정을 의미
- 인공지능 분야에서 신경망이란 보통 인공신경망을 지칭하며 그 둘을 따로 구분하지 않고 신경망이라고 총칭



## (1) 신경망역사(Artificial Neural Network)

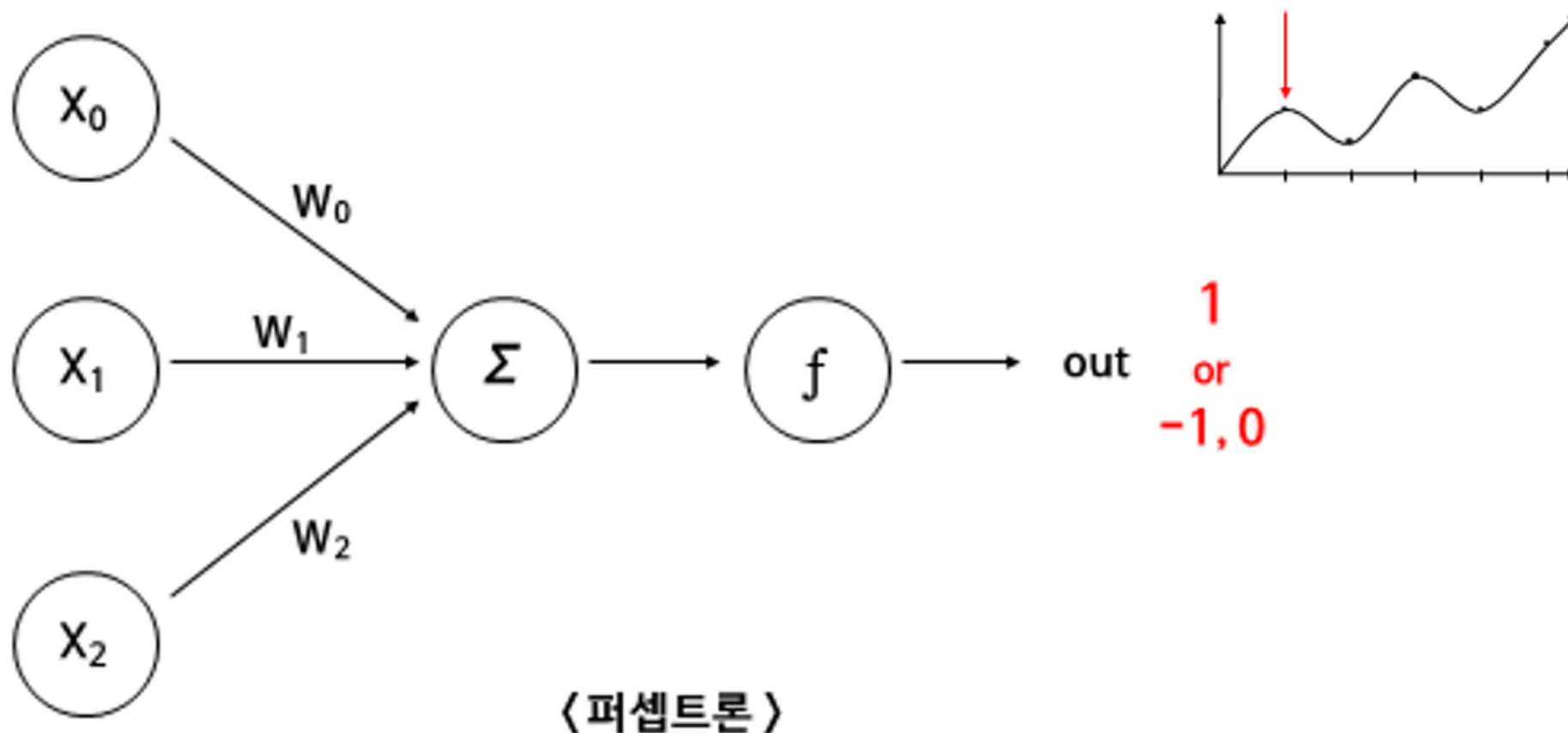
- 1차 황금기 ~ 3차 황금기 : 파란만장한 신경망





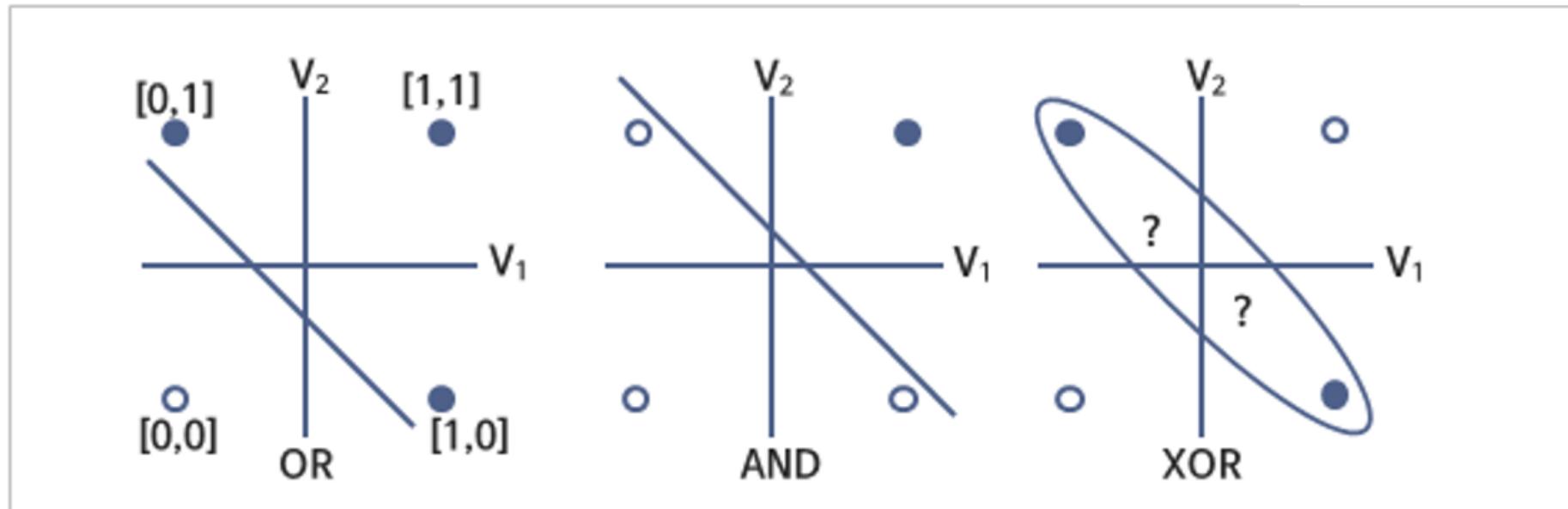
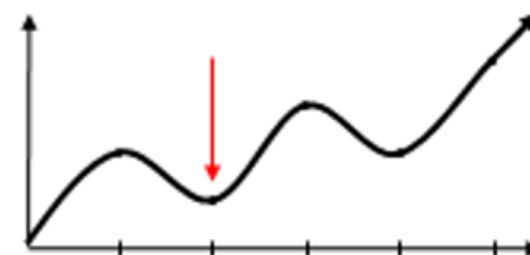
## (1) 신경망역사(Artificial Neural Network)

- 1958년 : frank rosenblatt에 의해 퍼셉트론 제안



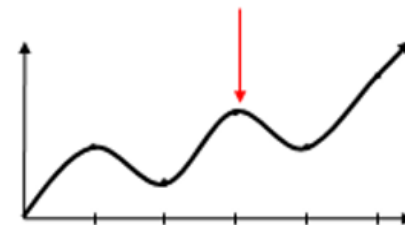
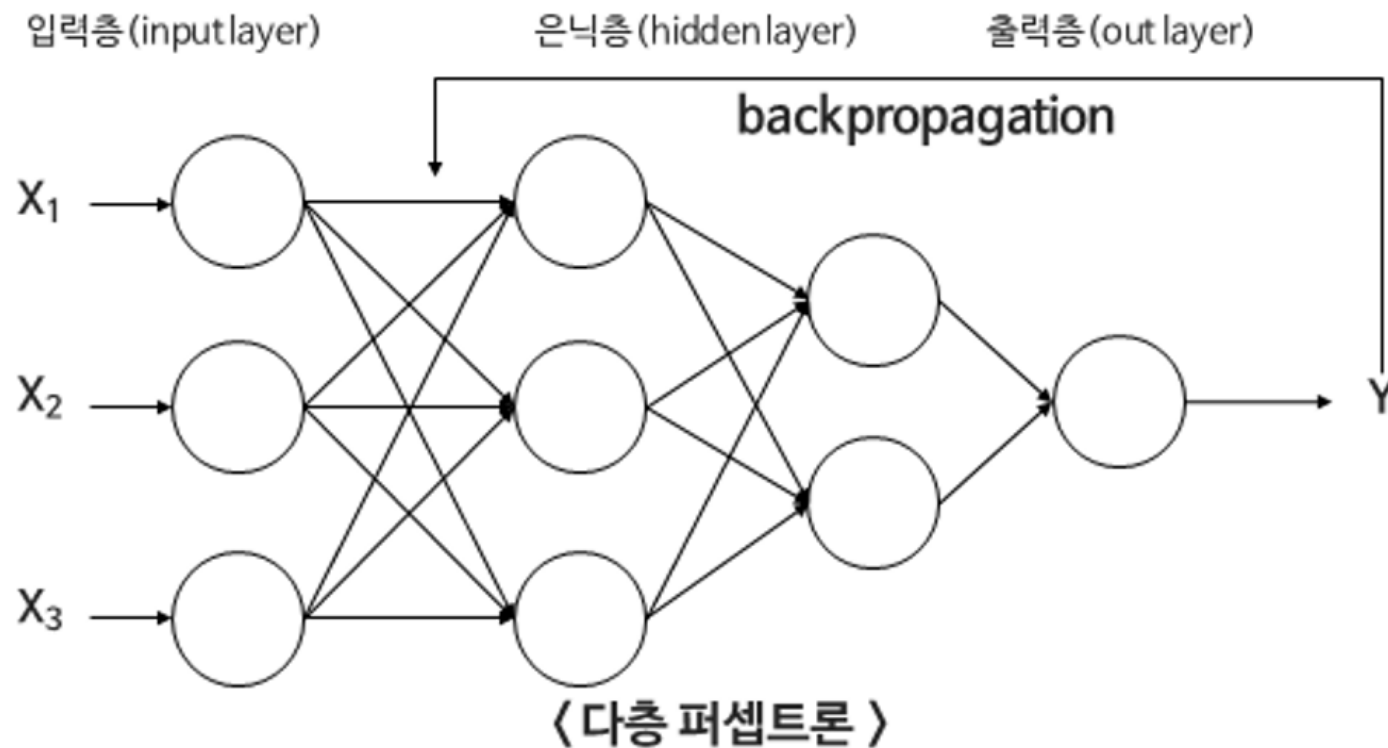
## (1) 신경망역사(Artificial Neural Network)

- 1969년(10년뒤) : marvin minsky 교수에 의해 XOR 문제 증명



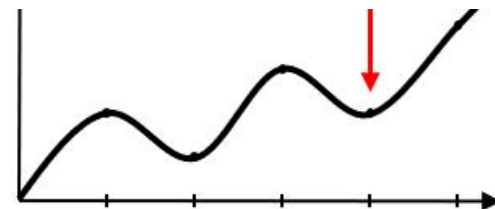
## (1) 신경망역사(Artificial Neural Network)

- 1986년(20년뒤) : Backpropagation + 다층퍼셉트론 등장
- XOR 문제 해결

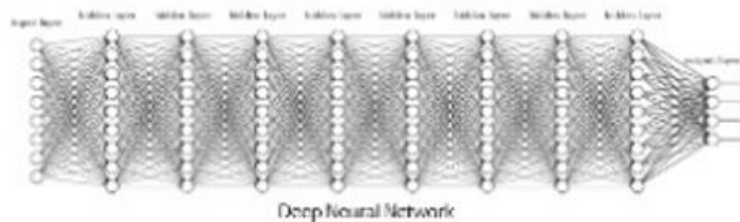


## (1) 신경망역사(Artificial Neural Network)

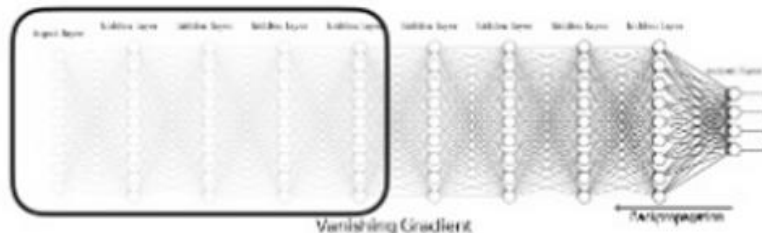
- 2000년 : Vanishing gradient problem 발견
- 데이터 증가, 컴퓨팅 파워 부족



**Vanishing gradient 현상:** 레이어가 깊을 수록 업데이트가 사라져간다.  
그래서 fitting이 잘 안됨(underfitting)

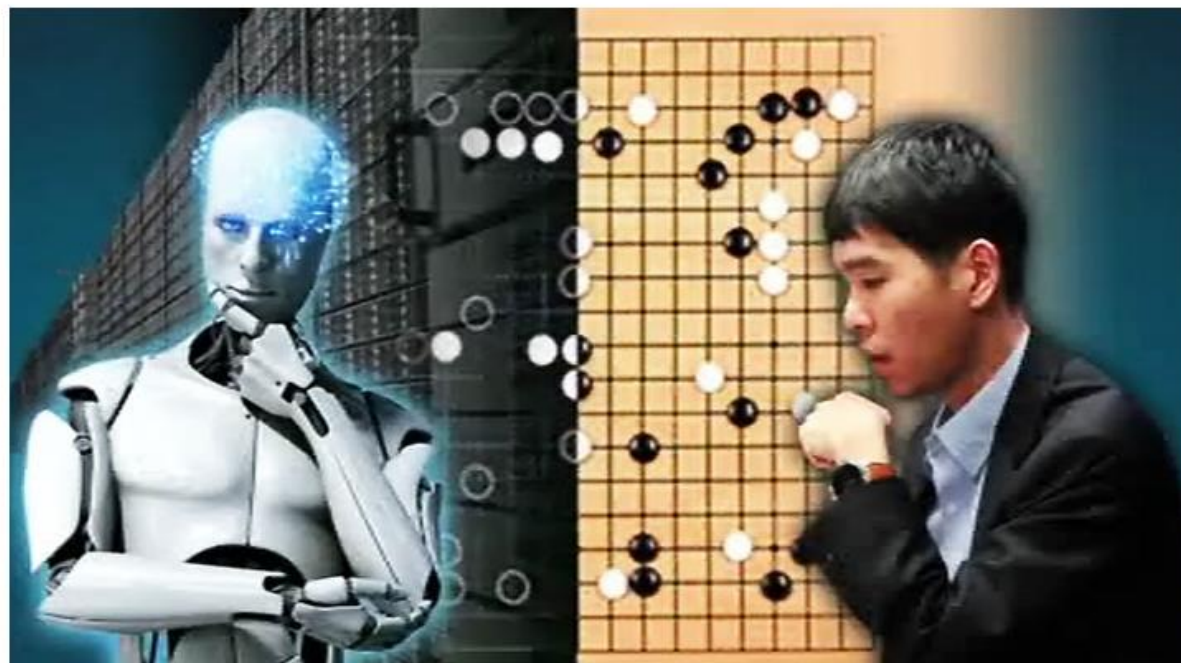
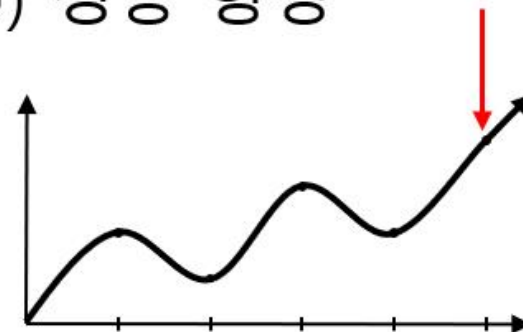


**학습이 잘 안됨**



## (1) 신경망역사(Artificial Neural Network)

- 2006년 : 딥러닝 등장, 컴퓨팅 파워(CPU, GPGPU) 성능 향상



## (2) 퍼셉트론(Perceptron)

### □ 신경망 계층구조 상세설명

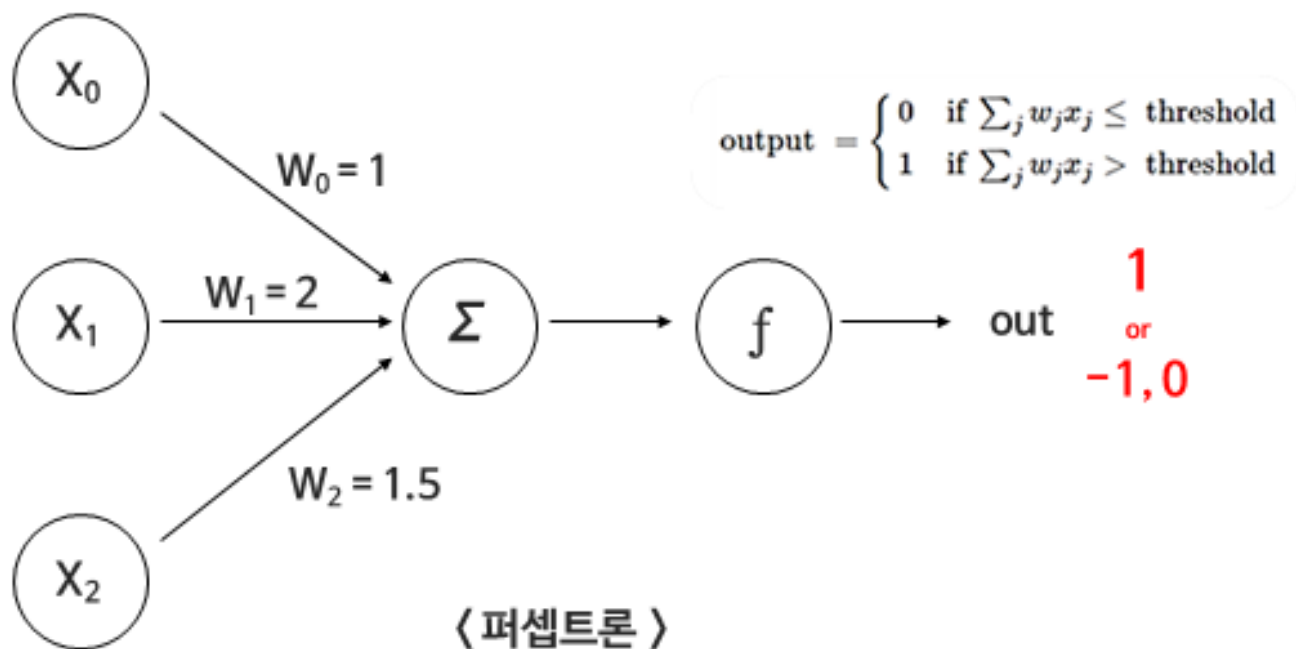
| 계층                    | 설명                                                                                      |
|-----------------------|-----------------------------------------------------------------------------------------|
| 입력층<br>(Input Layer)  | - 초기에 데이터가 셋팅되는 층을 의미                                                                   |
| 은닉층<br>(Hidden Layer) | - 데이터가 드러나지 않고 가려져 있는 층을 의미<br>- 즉, 우리 눈에 보이지 않기 때문에 Black Box 와 같다고 하여 Hidden layer 부름 |
| 출력층<br>(Output Layer) | - 그래프의 실행시, 파라미터를 저장하고 갱신하는데 사용되고, 메모리 상에서 텐서를 저장하는 버퍼 역할                               |

- 신경망 내부에 결합함수, 활성화함수, Backpropagation 등이 존재하나, 신경망을 구성하는 계층은 3 단계로 구성됨

## (2) 퍼셉트론(Perceptron)

### □ 퍼셉트론 개념

- 신경망 모형 중 하나로, 하나의 샘플이 어떤 클래스에 속하는지 예측 및 분류하는 알고리즘
- 인간의 두뇌를 모방하여 각 노드들이 서로 연결되고, 각 노드는 학습을 통해 가중치를 조정해 나가는 알고리즘
- 유형 : 단층 퍼셉트론, 다층 퍼셉트론

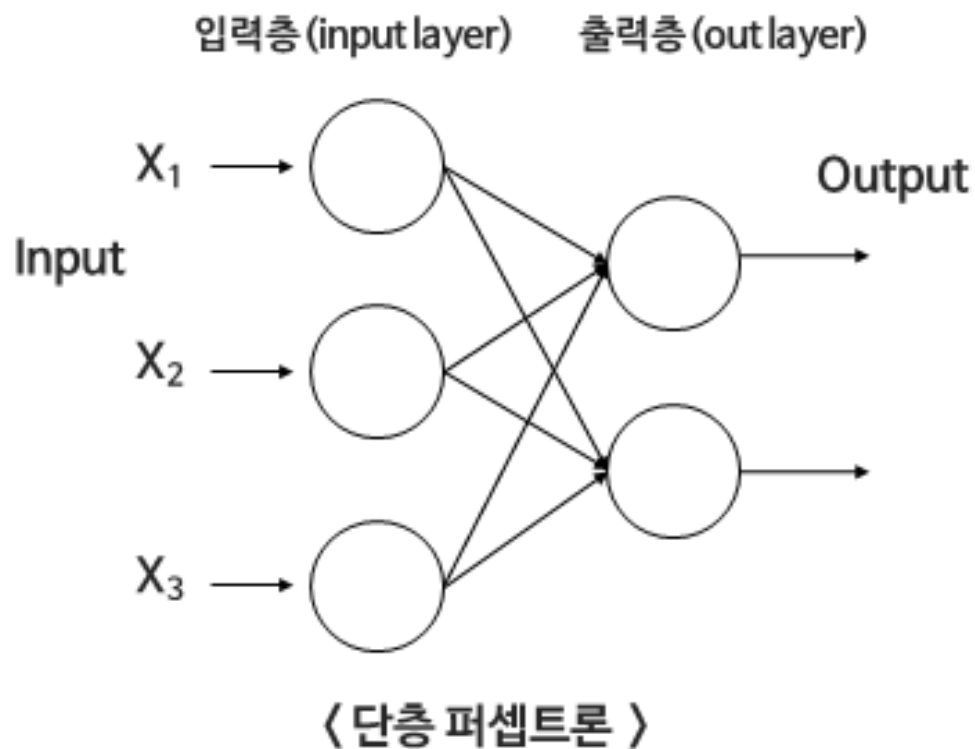




## (2) 퍼셉트론(Perceptron)

### ① 단층 퍼셉트론

- 입력층과 출력층으로 구성된 심플한 모형
- 은닉층 無
- 은닉층이 없 간단한 연산(AND, OR)은 가능하나 복잡한 연산(XOR) 불가

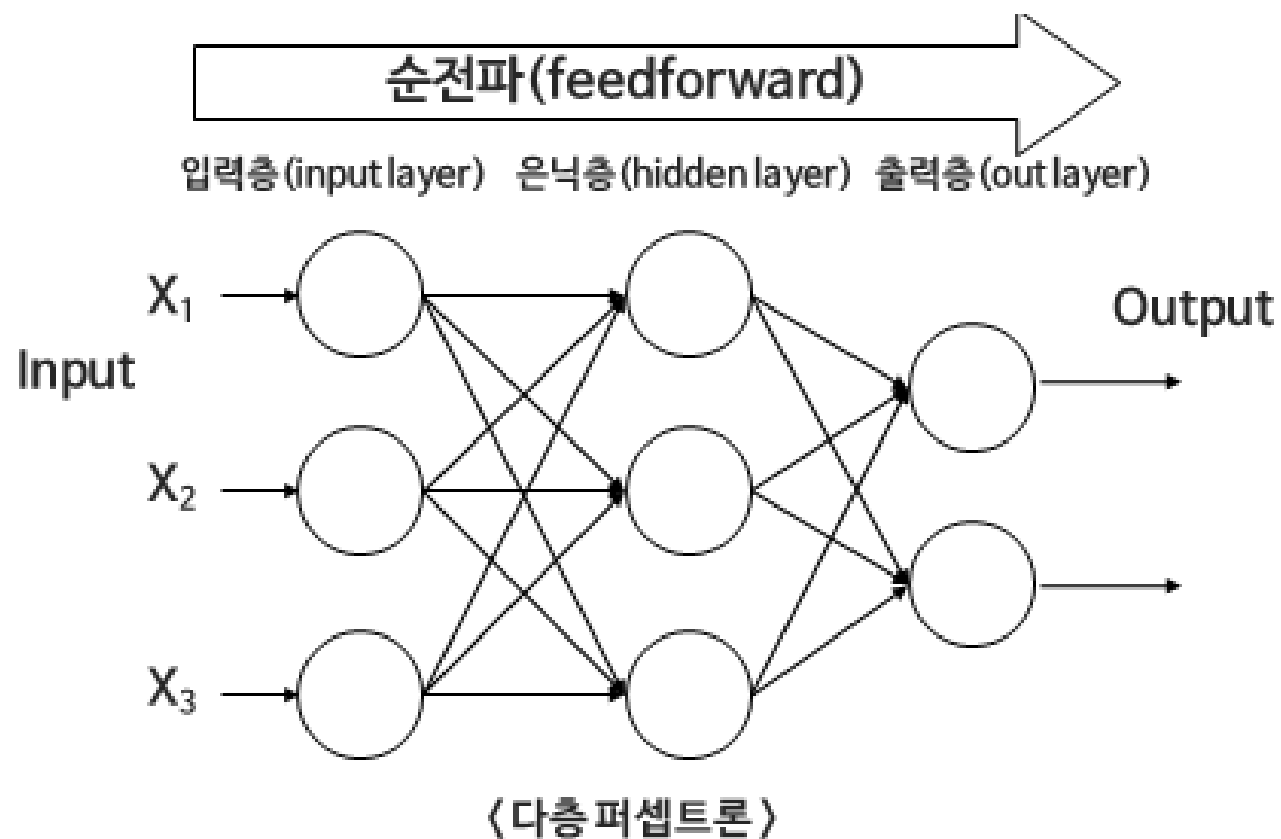




## (2) 퍼셉트론(Perceptron)

### ② 다층 퍼셉트론 (MLP)

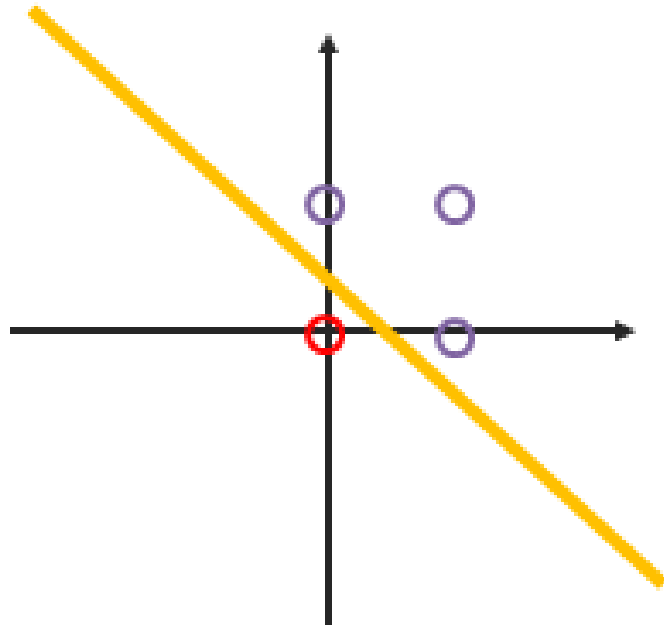
- 입력층, 은닉층(다수), 출력층 구조
- 은닉층 추가



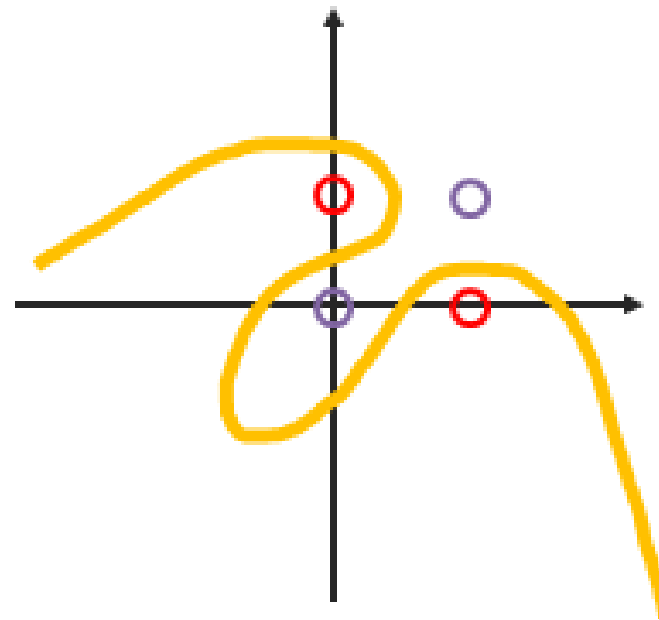
## (2) 퍼셉트론(Perceptron)

### ② 다층 퍼셉트론

- 다수의 퍼셉트론을 연결하여 XOR 문제 해결
- 비선형 분류 가능



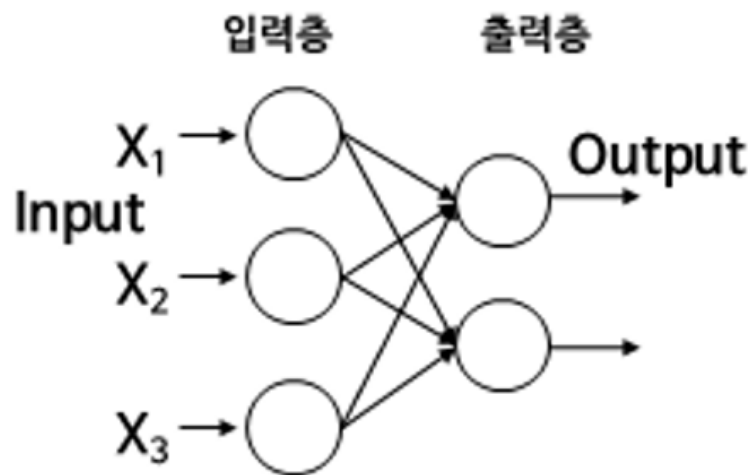
OR 게이트



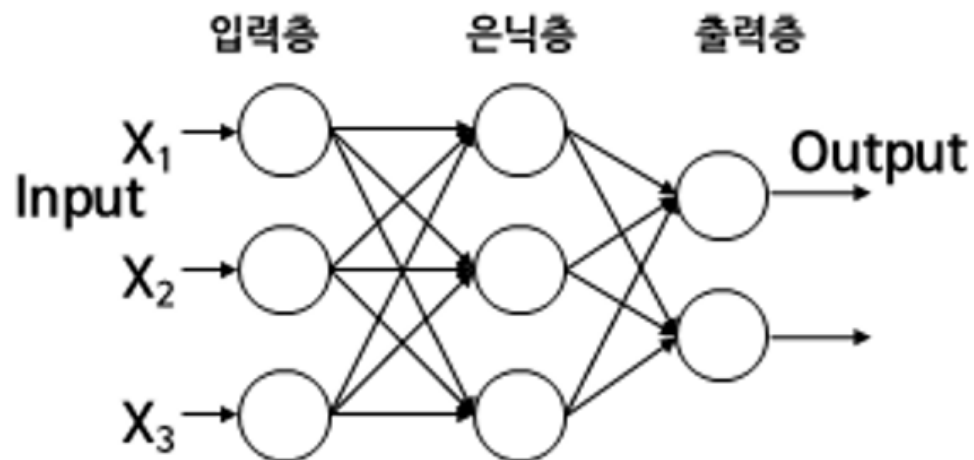
XOR 게이트

## (2) 퍼셉트론(Perceptron)

- 단층 퍼셉트론 : 입력층 + 출력층, XOR 문제
- 다층 퍼셉트론 : 입력층 + 은닉층 + 출력층 + 오류역전파



〈단층 퍼셉트론〉

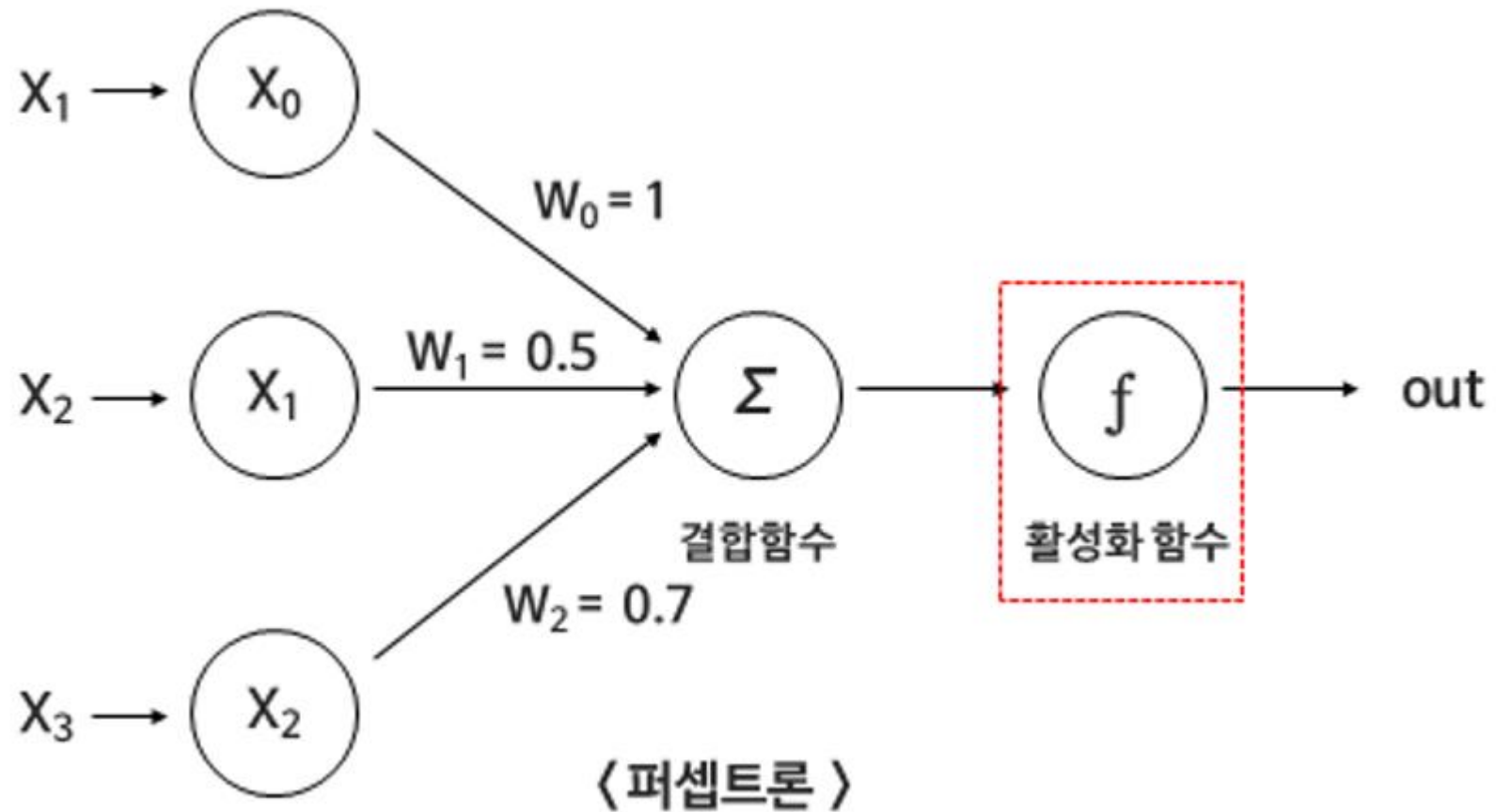


〈다층 퍼셉트론〉

### (3) 활성화 함수(Activation Function)

#### □ 활성화 함수 개념

- 결합함수로부터 전달받은 결과값이 임계값 보다 크면 1, 그렇지 않으면 0 또는 -1 을 출력하는 함수



### (3) 활성화함수(Activation Function)

#### □ 활성화 함수 필요성

- 신경망에 너무 약한 신호가 입력될 경우 그 신호가 미세하여 별 의미가 없고, 반대로 너무 강한 신호를 전달할 경우에도 그 값이 너무 편향되어 좋은 값이 아님
- 또한 우리가 만든 신경망 모델이 이러한 미세한 입력값에 지나치게 민감하다면 모델의 성능이 현저히 저하될 것이기 때문에 특정 수준 이상의 입력값 만을 전달하는 것이 더욱 효율적일 임

#### □ 활성화 함수 유형 많은 이유

- 활성화 함수 유형이 많은 이유는 아직 우리 뇌의 신경들이 어떻게 서로 연결되고, 정확히 어떤 규칙을 통해서 동작하는지 아직 다 밝혀내지 못함
- 즉, 우리가 원하는 모델에 다양한 함수를 적용해보고, 그 결과값을 통해 가장 효율적인 활성화 함수를 선택 필요

### (3) 활성화함수(Activation Function)

**Sigmoid**

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



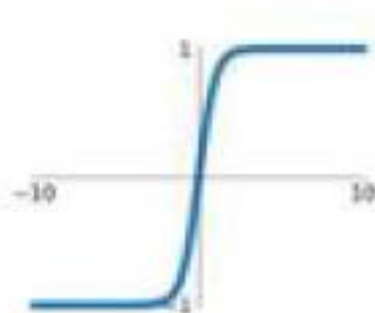
**Leaky ReLU**

$$\max(0.1x, x)$$



**tanh**

$$\tanh(x)$$



**ReLU**

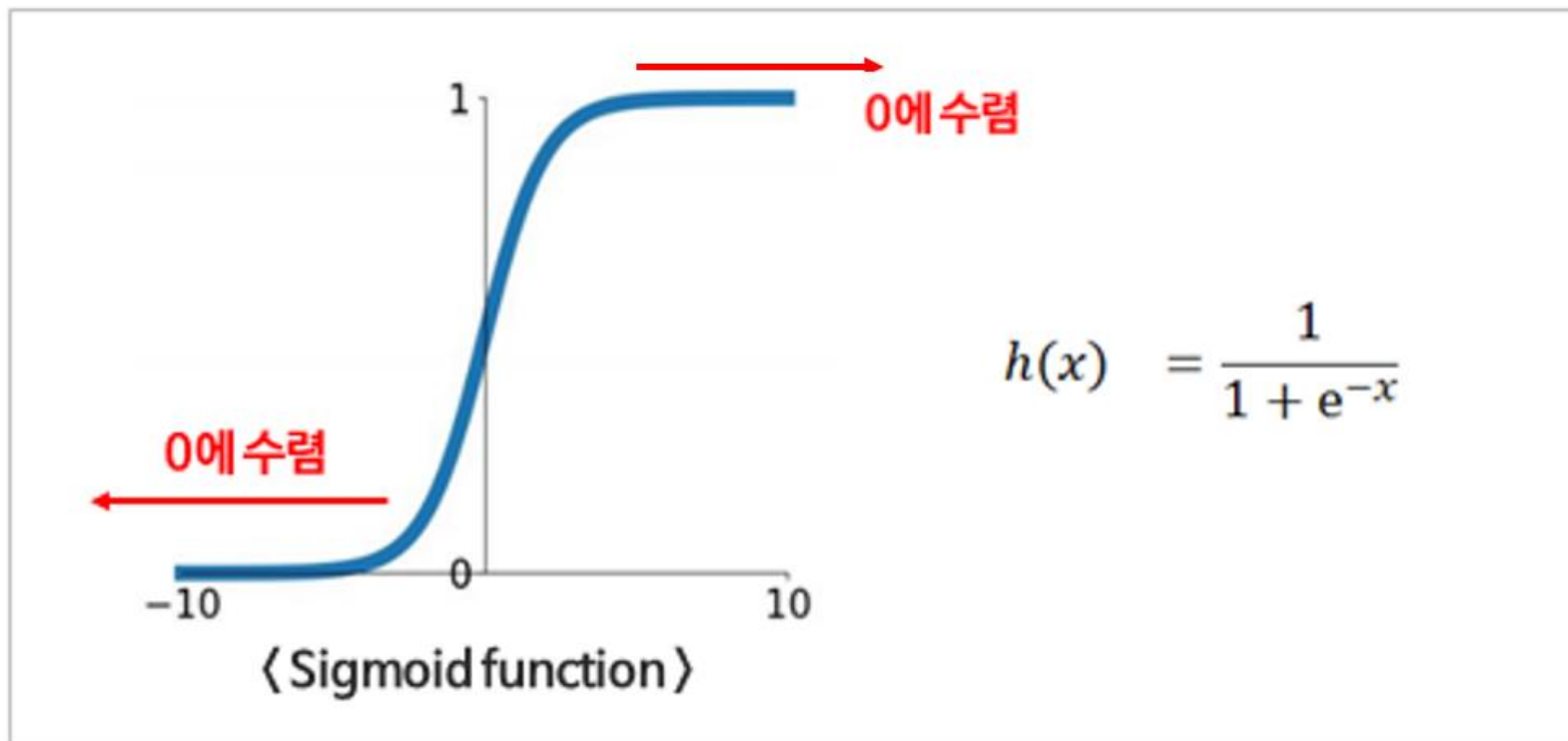
$$\max(0, x)$$



### (3) 활성화함수(Activation Function)

#### ① Sigmoid Function

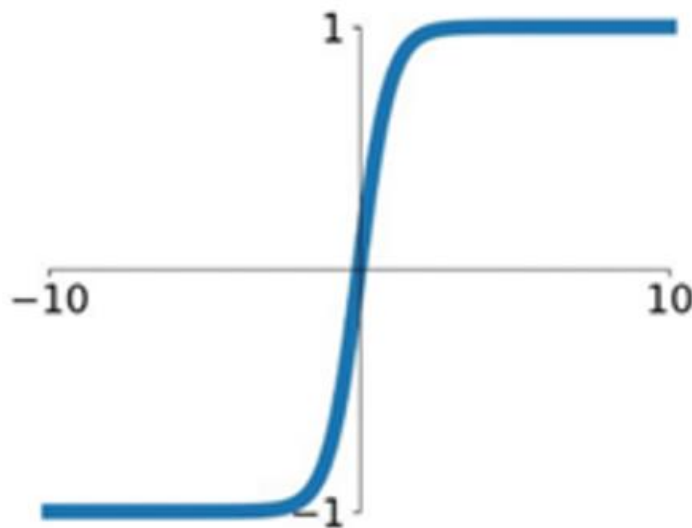
- 0~1까지의 값을 출력하는 S자형, 비선형 함수
- Logistic 함수라고도 부름
- Vanishing Gradient 문제



### (3) 활성화함수(Activation Function)

#### ② tanh Function

- sigmoid function 개선된 함수
- Sigmoid보다 출력 범위가 더 넓고 더 빠르게 학습
- Vanishing Gradient 문제



〈 tanh function 〉

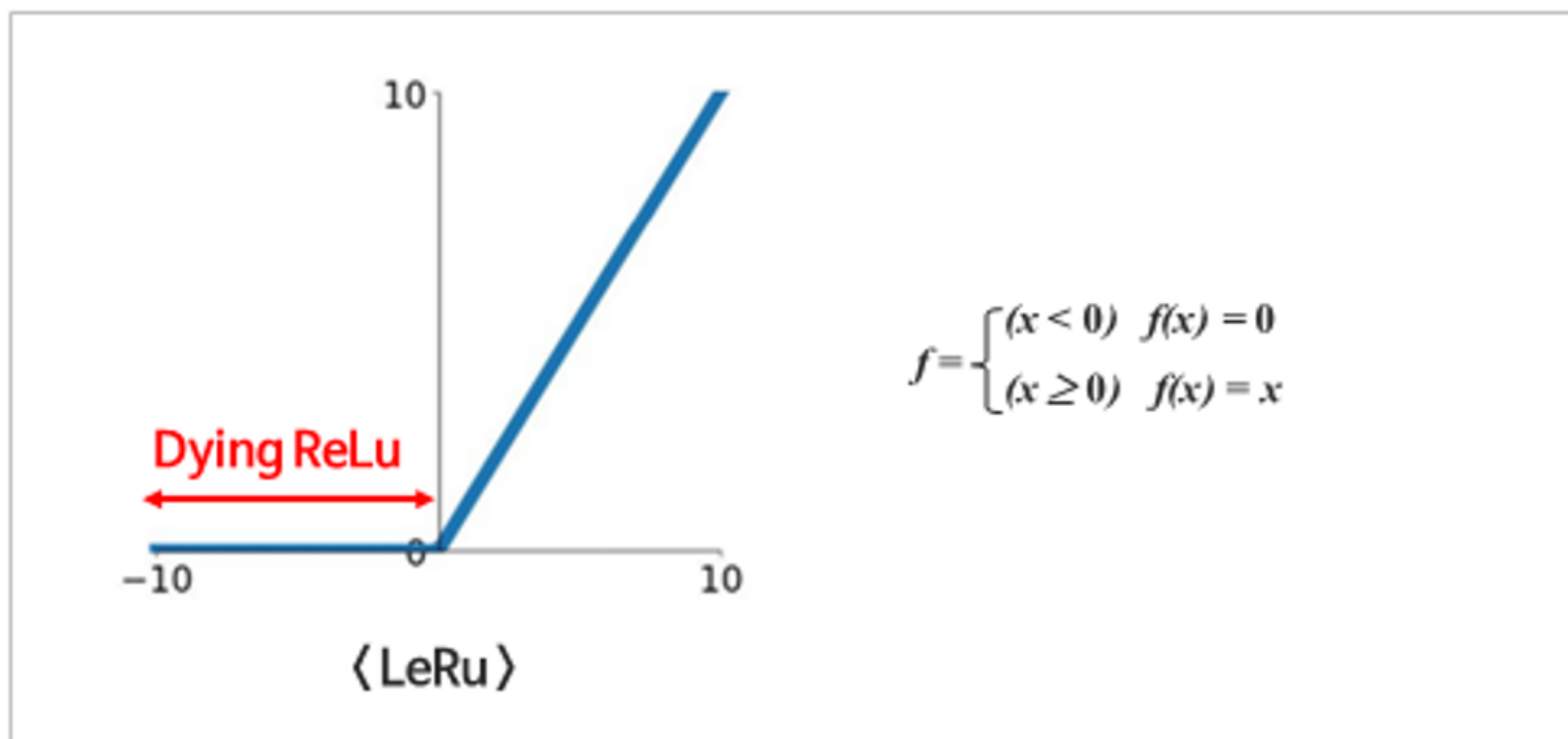
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



### (3) 활성화함수(Activation Function)

#### ③ ReLu Function

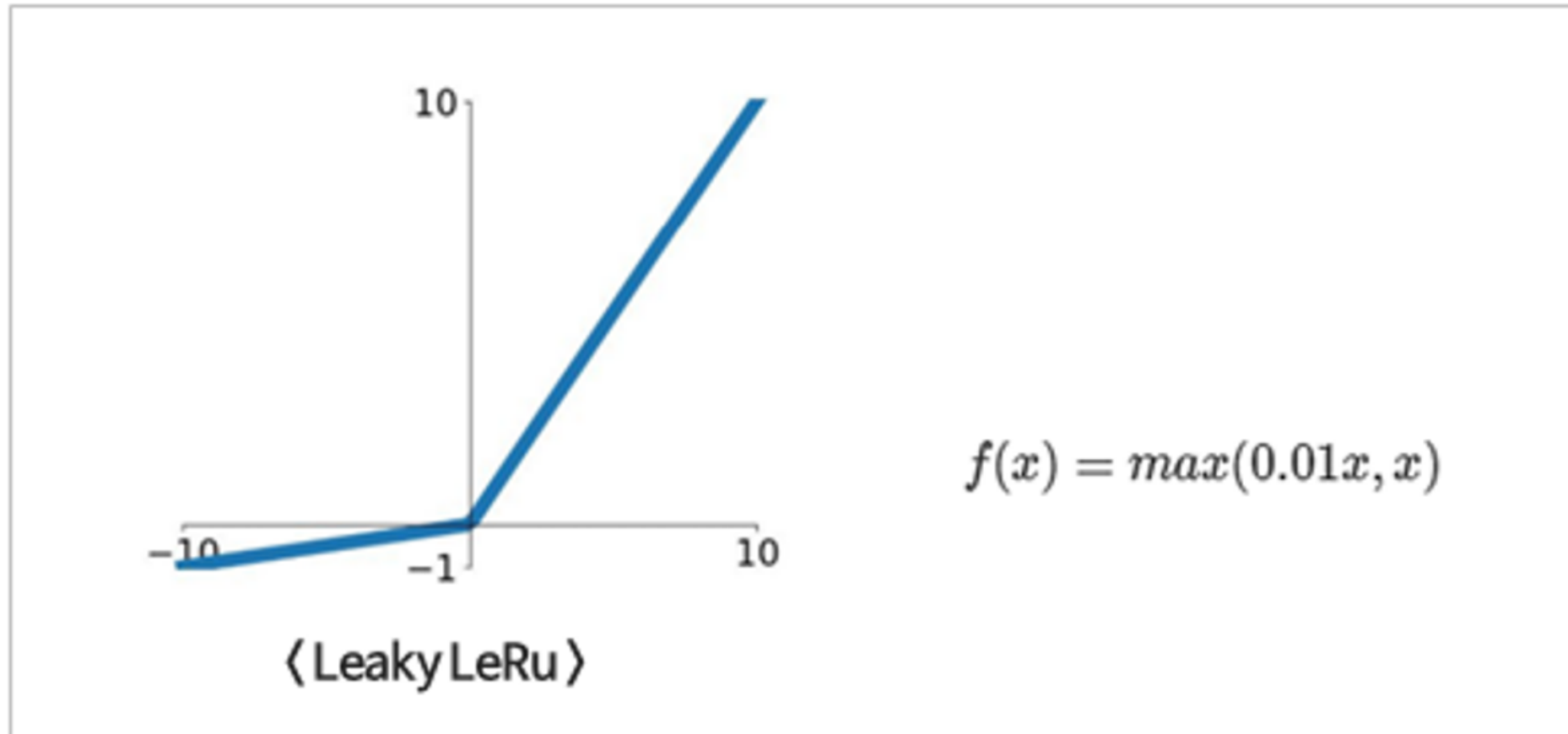
- Vanishing gradient 문제 해결
- Sigmoid function보다 6배 빠른 학습 시간
- Dying ReLu 현상



### (3) 활성화함수(Activation Function)

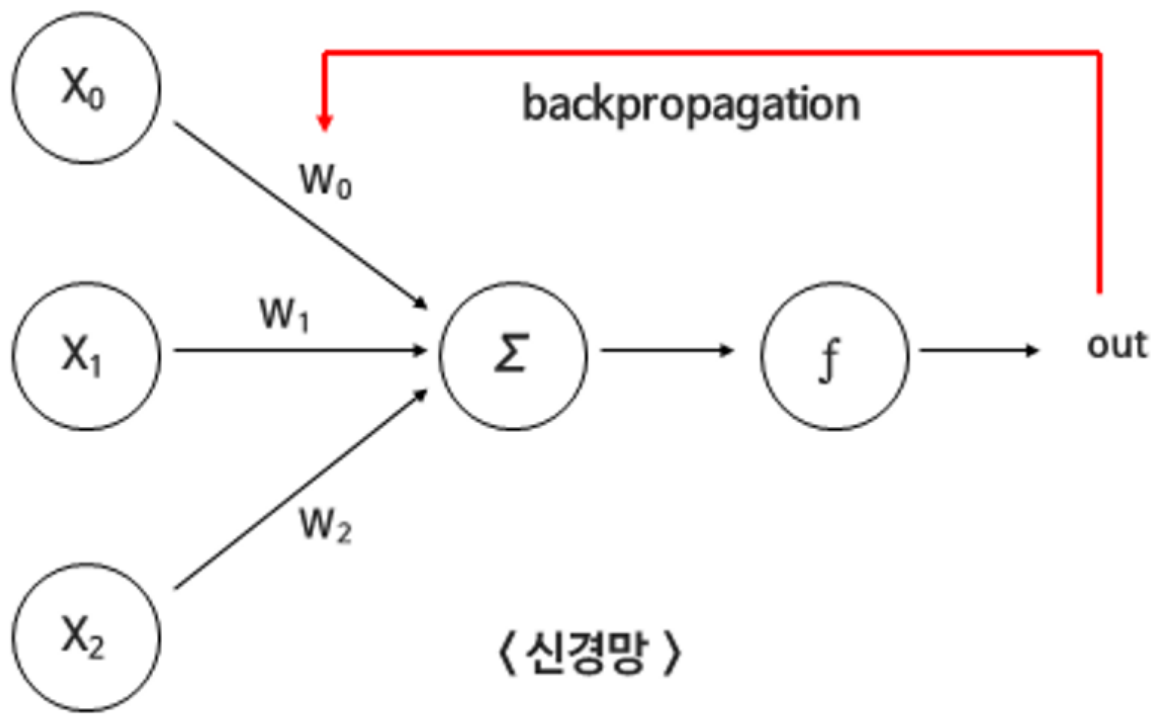
#### ④ Leaky ReLu Function

- 입력값이 음수가 되더라도 0이 아닌 값을 유지 : Dying ReLU 해결



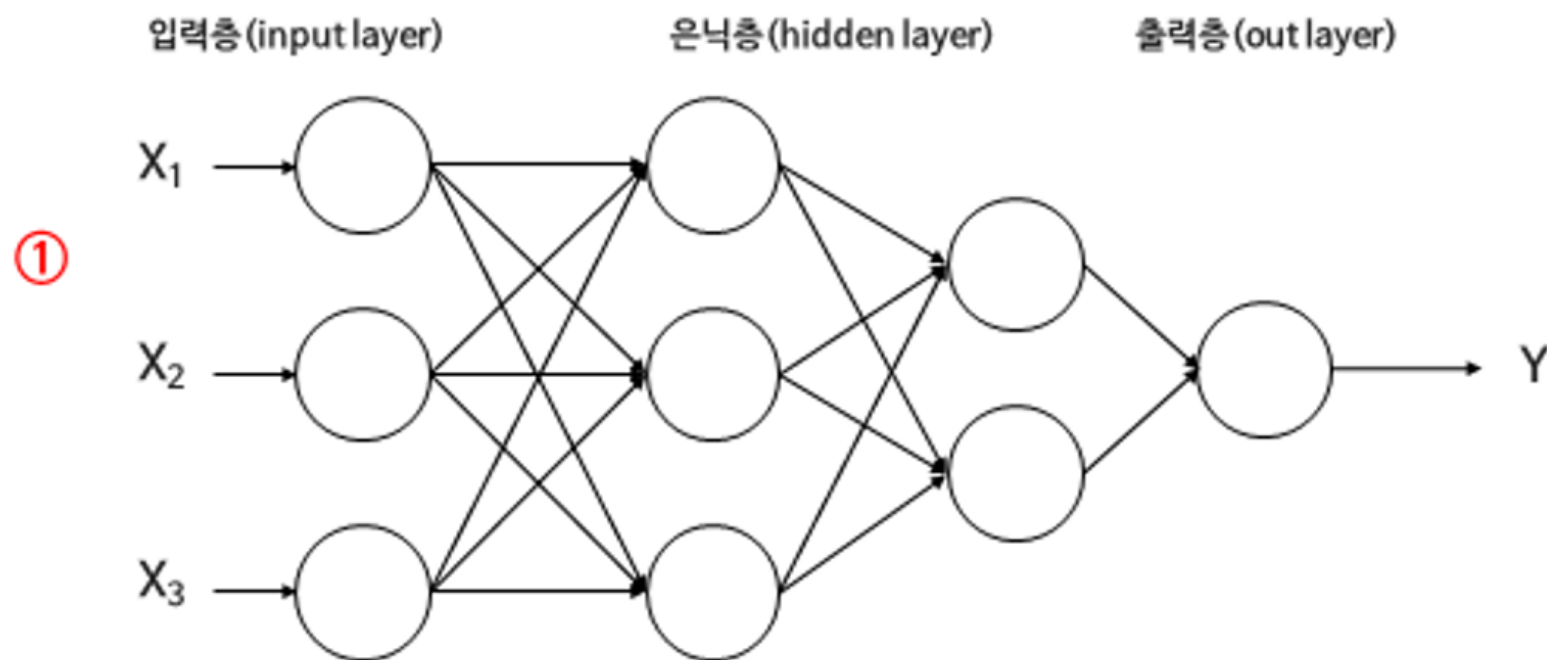
#### (4) 오류 역전파(Back-propagation)

- 출력값이 원하는 결과와 다를 경우, 가중치를 갱신하여 오차를 최소화 시키도록 반복 학습하는 알고리즘
- Cost function, 경사 하강법(Gradient Descent)을 이용



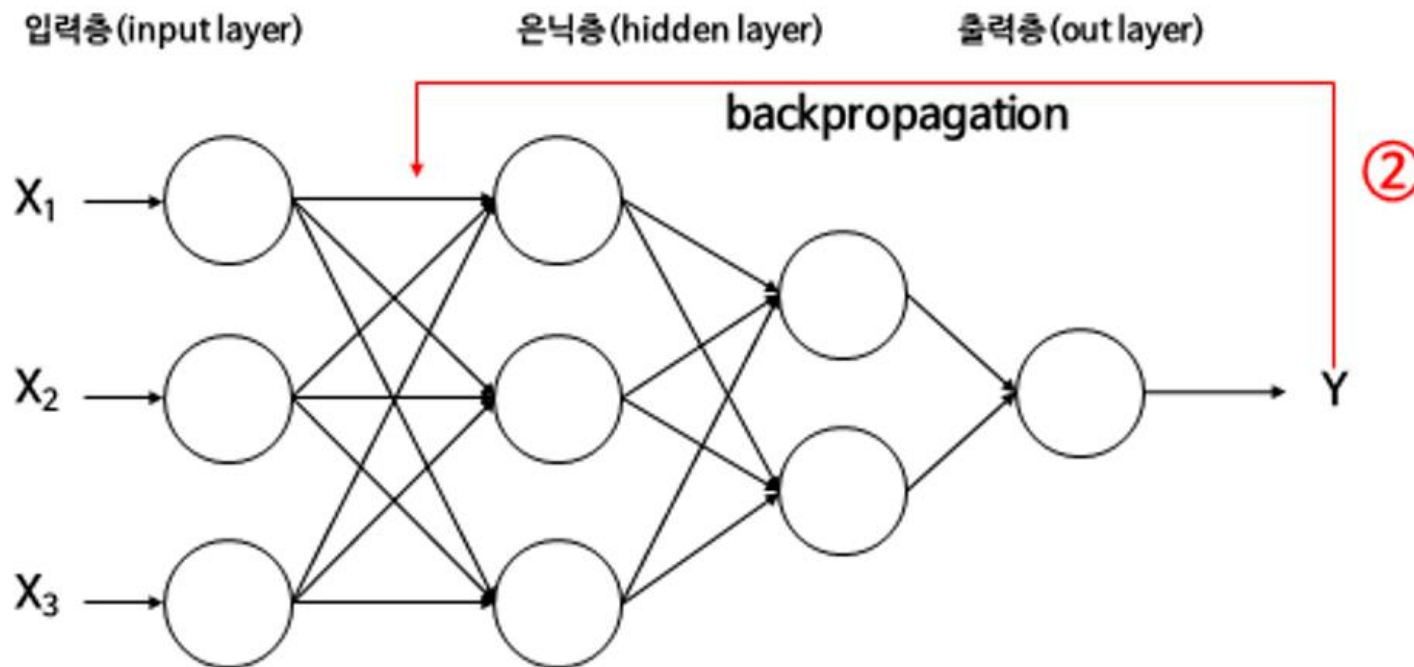
#### (4) 오류 역전파(Back-propagation)

- feedforward 방향으로 입력값과 가중치를 계산하여 그 결과값을 출력

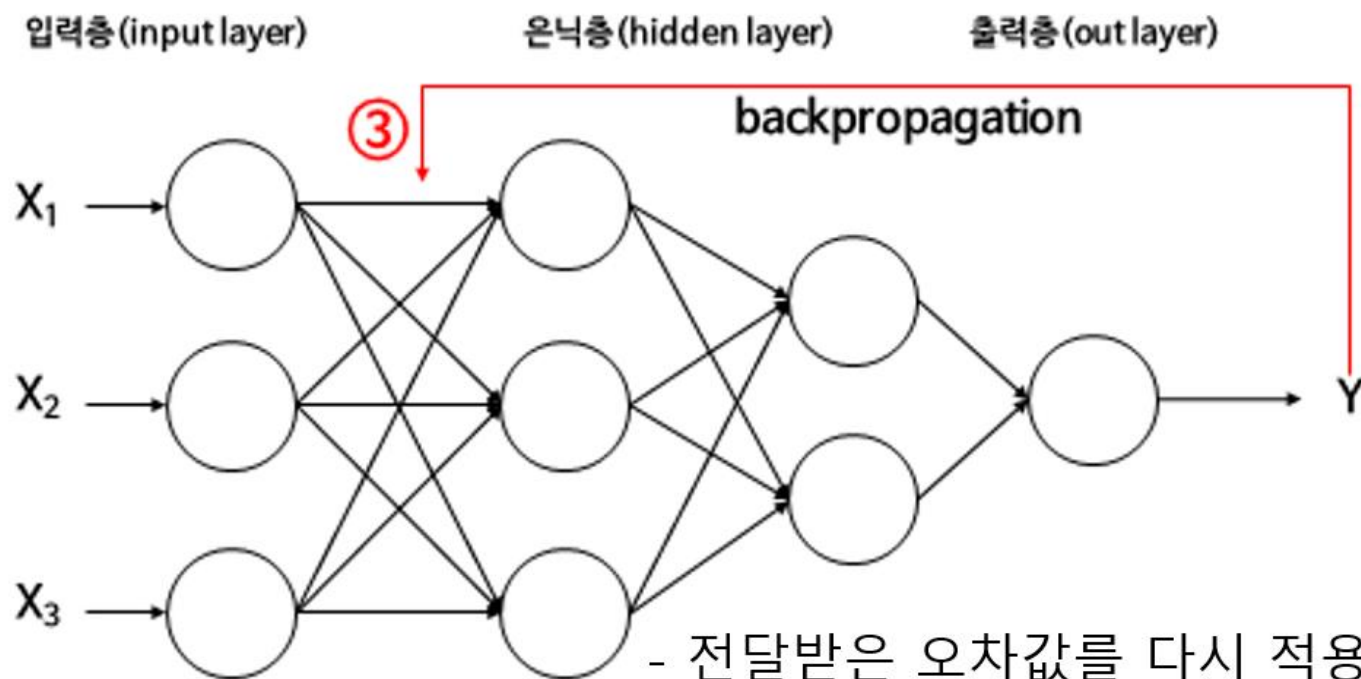


#### (4) 오류 역전파(Back-propagation)

- 출력값과 예측값이 서로 다를 경우 그 둘간의 오차값을 계산하여 이전 노드로 오차값을 다시 전달



#### (4) 오류 역전파(Back-propagation)



- 전달받은 오차값을 다시 적용하여 가중치 값을 update
- 오차가 최소화가 될 때까지 이 과정을 반복

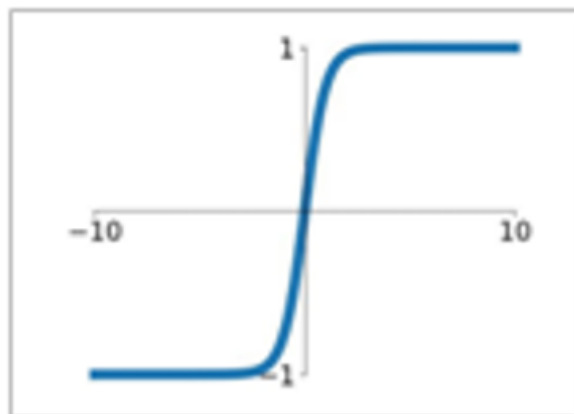
#### (4) 오류 역전파(Back-propagation)

- **Vanishing Gradient 문제**
  - Layer가 많아질수록 기울기(오차)가 0으로 수렴하는 현상



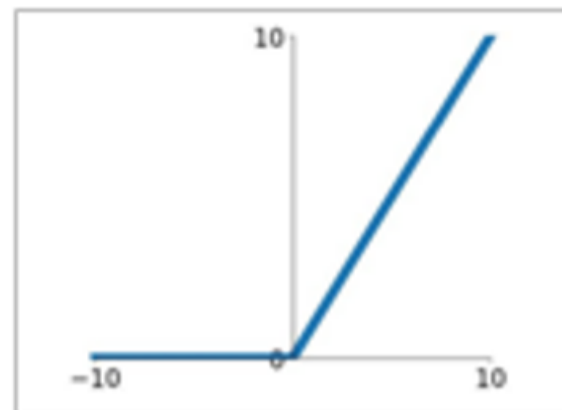
#### (4) 오류 역전파(Back-propagation)

- **Vanishing Gradient 문제 해결**
- Sigmoid 활성화 함수에서 발생하는 현상
- ReLu, Tanh 활성화 함수 사용



Sigmoid function

변경

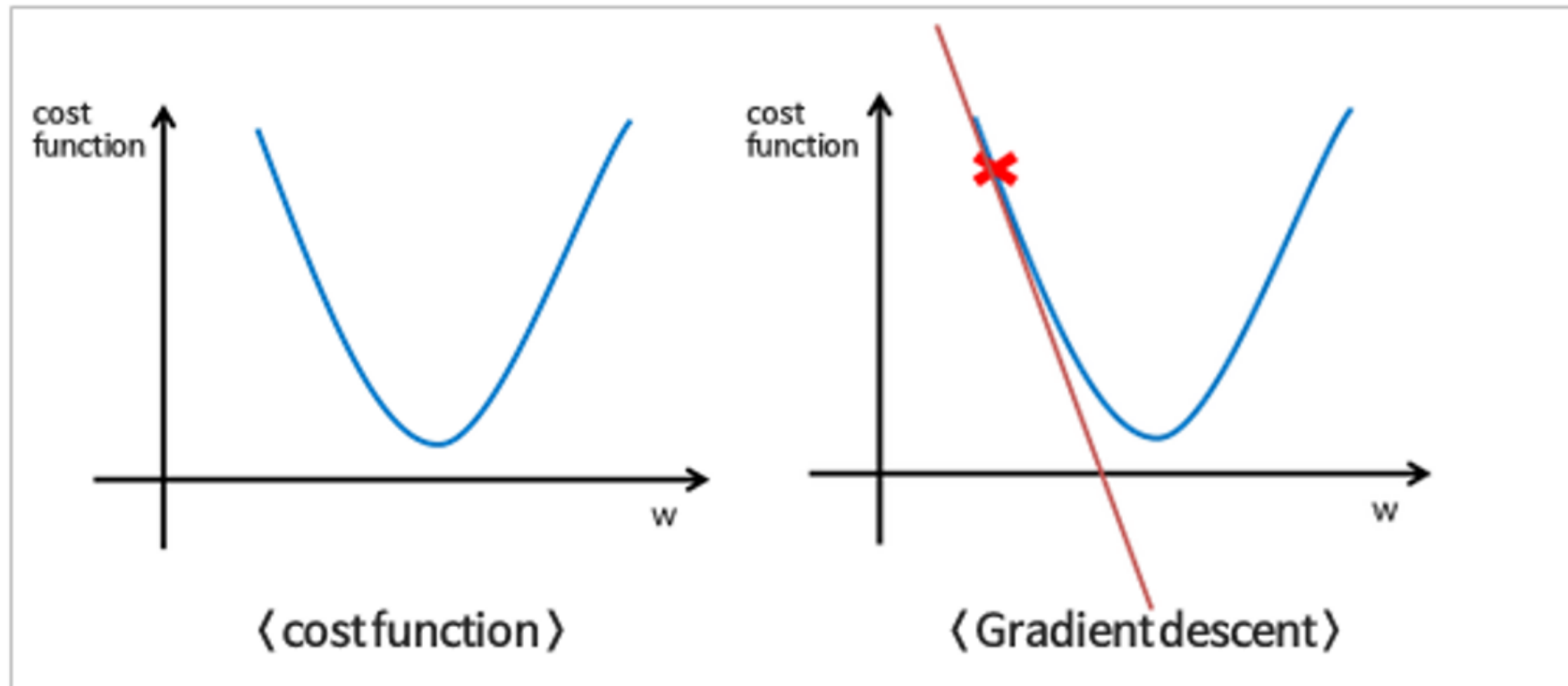


ReLU



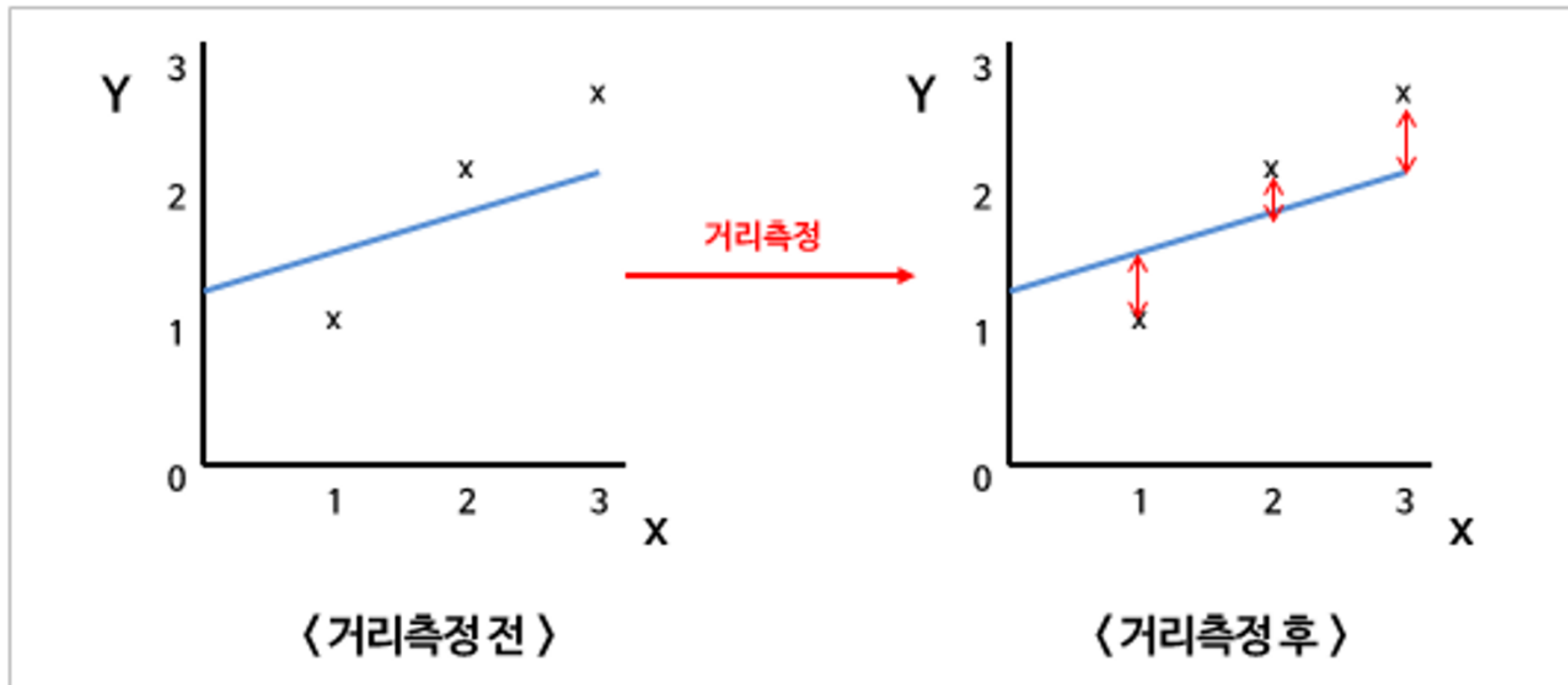
## (5) 비용함수(Cost Function)

- 실제값과 예측값이 얼마나 차이 나는지(오차) 측정하기 위한 함수
- 즉, 오차가 좋은 모델
- Lost function이라고도 함



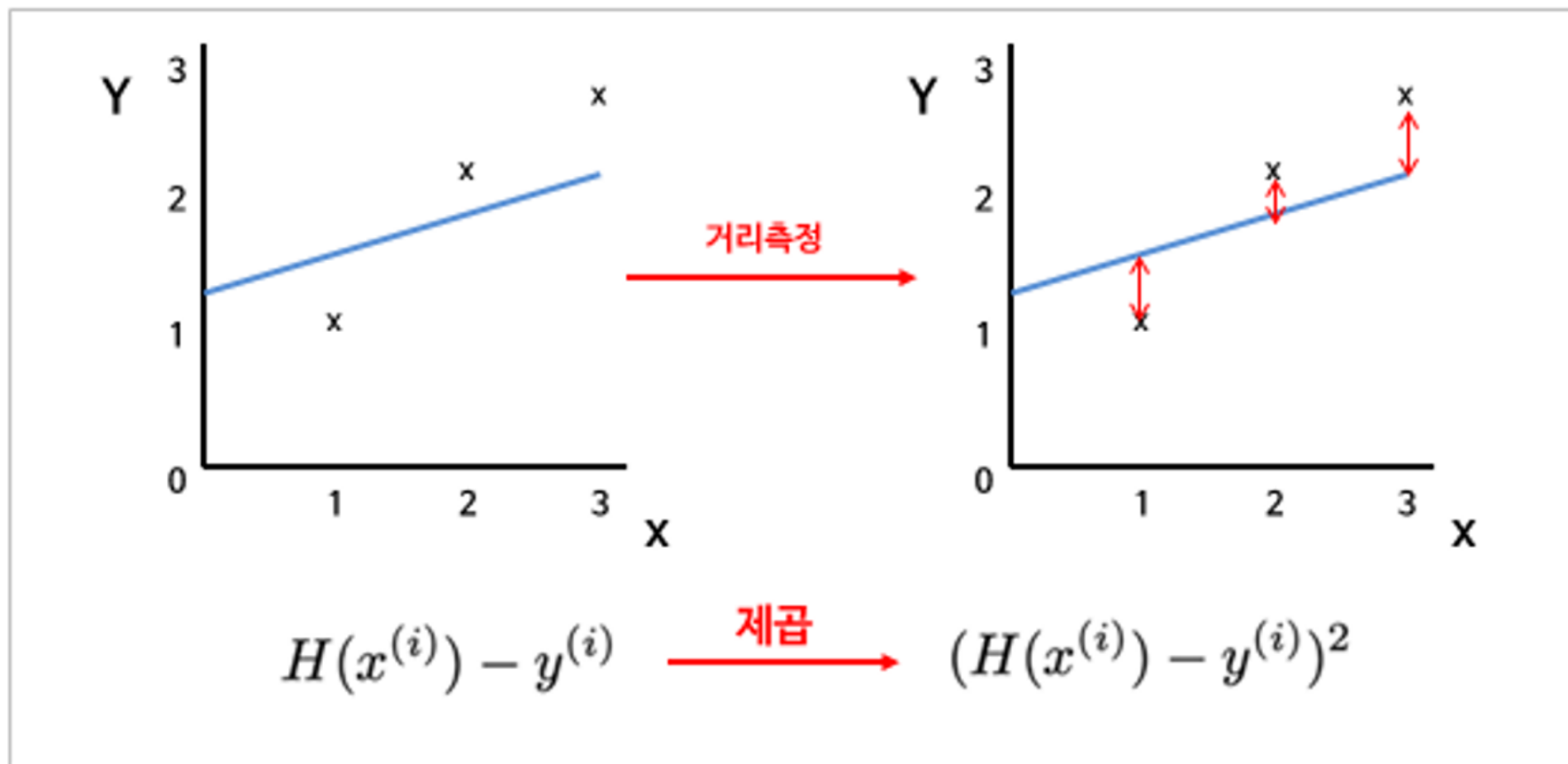
## (5) 비용함수(Cost Function)

- 직선 : 예측값      x점 : 실제값



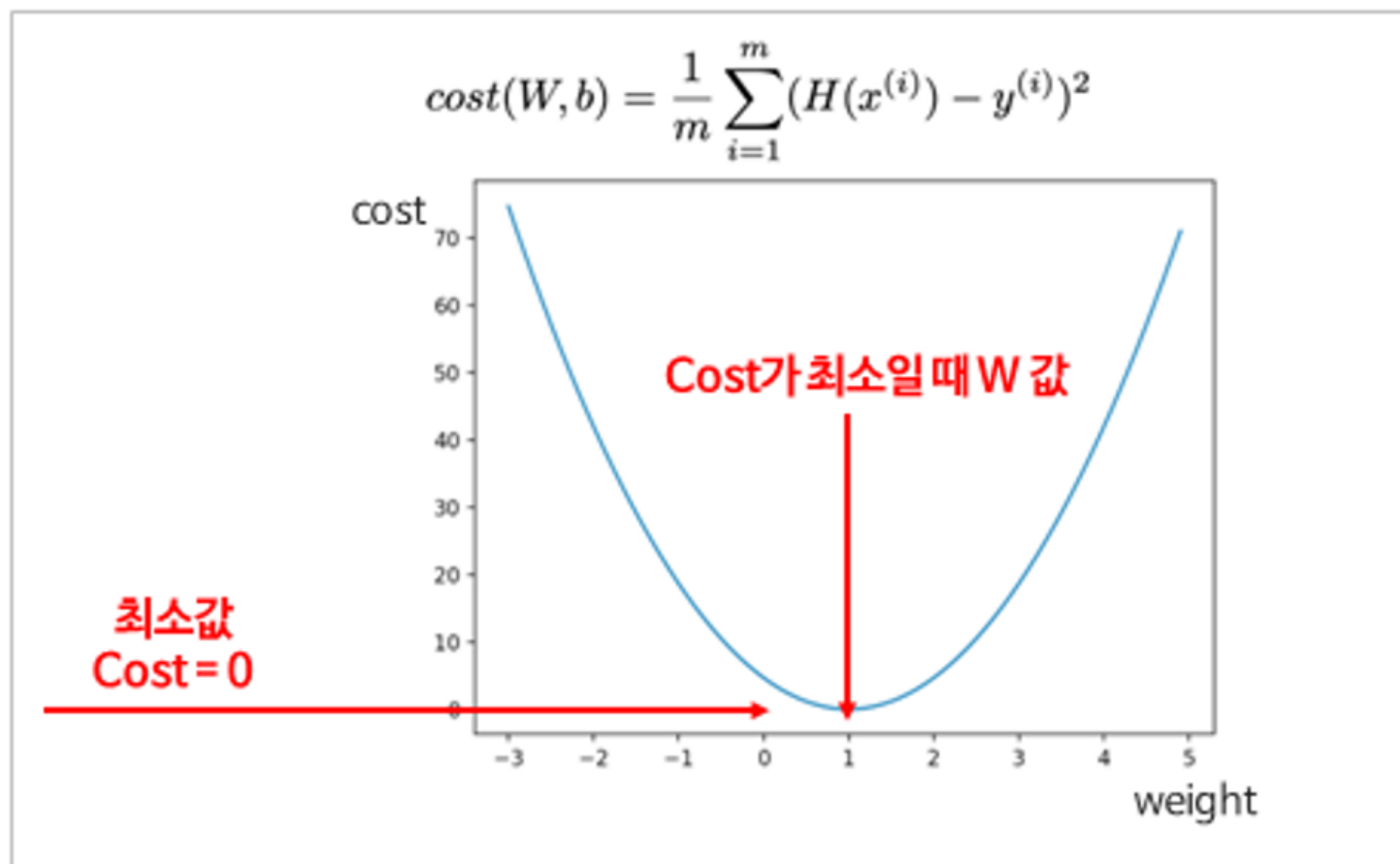
## (5) 비용함수(Cost Function)

- 직선 : 예측값      x점 : 실제값



## (5) 비용함수(Cost Function)

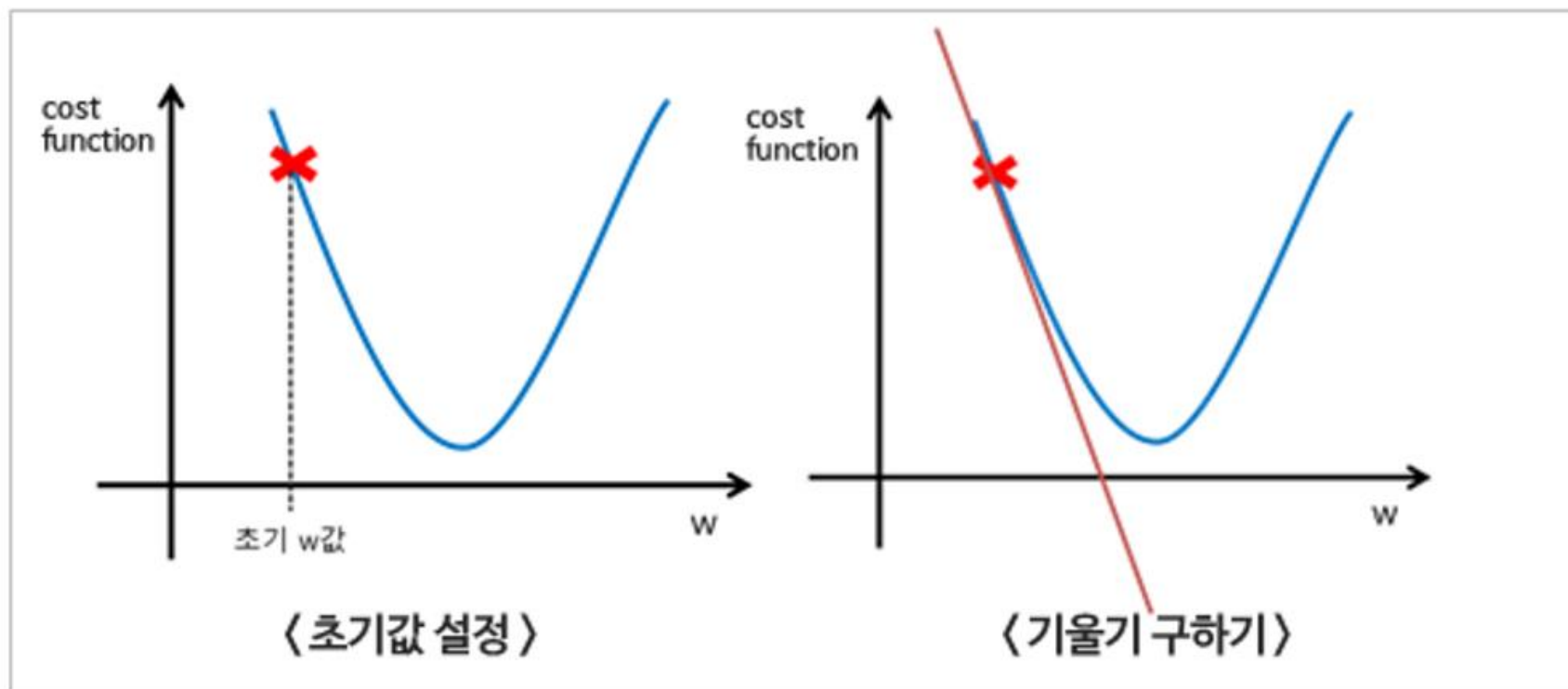
- X축 : 가중치(weight), Y축 : 거리(cost)  
=> cost의 값이 최소화가 되는 weight값을 찾는 것이 목표



## (6) 경사하강법 (Gradient descent)

- Cost function의 최소값을 찾는 알고리즘
- 초기값 설정 : Random

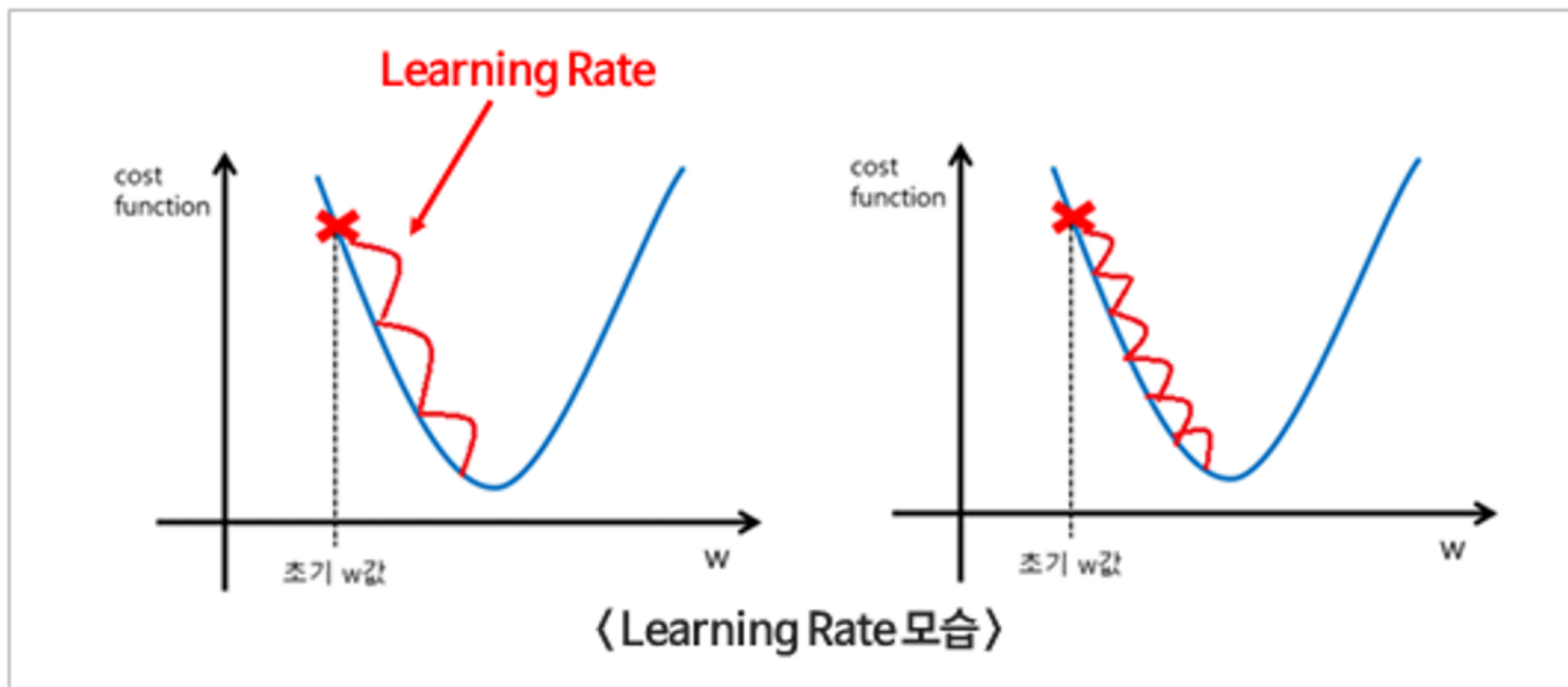
$$W := W - \alpha \frac{\partial}{\partial W} \text{cost}(W)$$



## (6) 경사하강법 (Gradient descent)

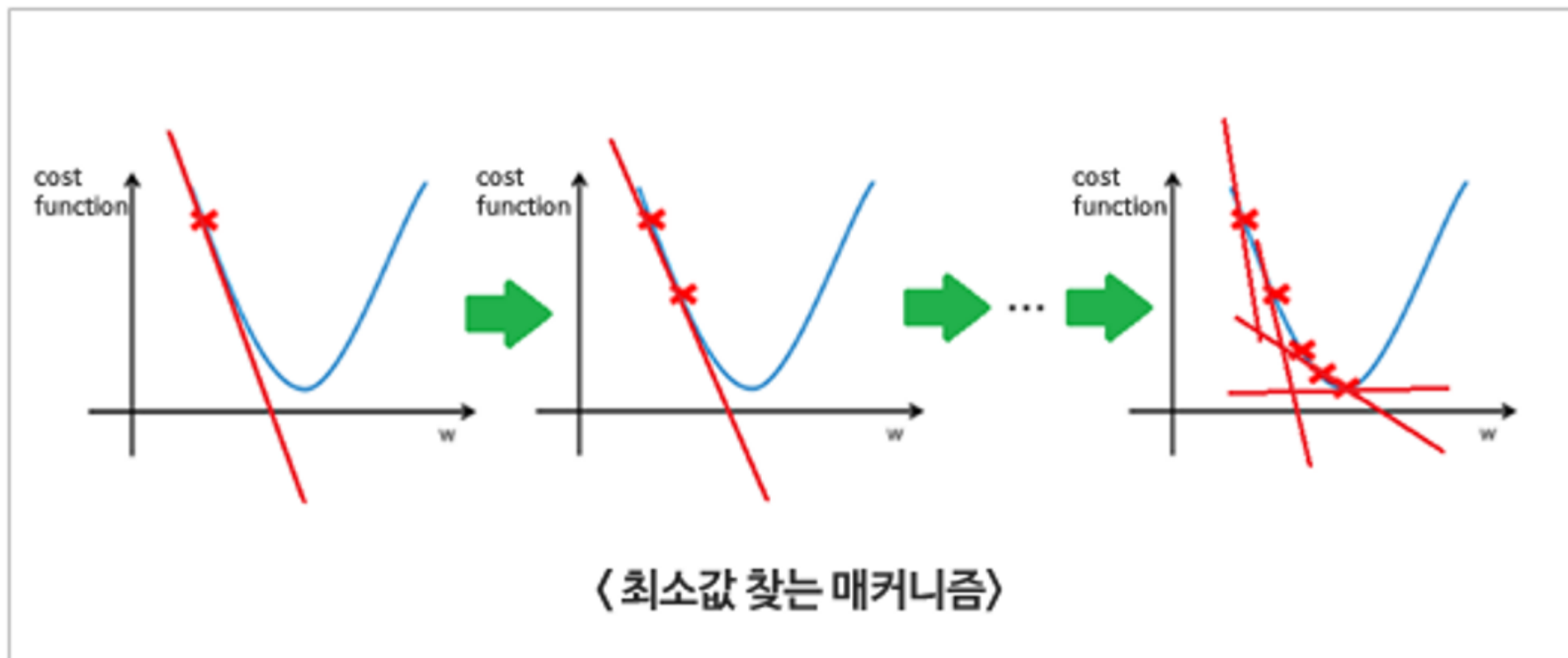
- Learning Rate : 경사면을 따라 내려가는 거리

$$W := W - \alpha \frac{\partial}{\partial W} cost(W)$$



## (6) 경사하강법 (Gradient descent)

- 현재 위치에서 미분 하여 기울기에 따라 이동  
→ 최소 cost 값 찾기



감사합니다!